Great Ideas in Computer Architecture (Machine Structures)

Mihir Rao

Contents

1 Great Ideas in Computer Architecture, Intro **6**
1.1 Machine structures . . . 6
1.2 Six great ideas . . . 6
1.3 Why architecture is exciting today . . . 9

2 Number Representation **10**
2.1 Bits can represent anything . . . 10
2.2 Bases and conversions . . . 10
2.3 Unsigned numbers and overflow . . . 11
2.4 Negative numbers . . . 12

3 C Intro: Basics **14**
3.1 Stored-program computers and C . . . 14
3.2 Compilation . . . 14
3.3 C versus Java . . . 16
3.4 C syntax . . . 16

4 C Intro: Pointers, Arrays, Strings **19**
4.1 Declarations and undefined behavior . . . 19
4.2 Addresses and pointers . . . 19
4.3 Using pointers effectively . . . 20
4.4 Arrays . . . 21
4.5 Pointers to pointers . . . 22
4.6 Function pointer example . . . 22

5 C Memory Management **23**
5.1 Dynamic memory allocation . . . 23
5.2 Linked list example . . . 25
5.3 Memory locations . . . 25
5.4 Memory management . . . 26
5.5 When memory goes bad . . . 27

6 Floating Point **29**
6.1 Fixed point . . . 29
6.2 Floating point representation . . . 29
6.3 IEEE 754 . . . 30
6.4 Special numbers . . . 30
6.5 Examples . . . 31
6.6 Floating point arithmetic . . . 31
6.7 Other representations . . . 32

7 RISC-V Intro **34**
7.1 Instruction set architectures . . . 34

7.2 Registers . . . 34
7.3 Addition and subtraction . . . 35
7.4 Immediates . . . 36

8 RISC-V lw, sw, Decisions I **37**
8.1 Storing data in memory . . . 37
8.2 Data transfer instructions . . . 37
8.3 Decisions . . . 38

9 RISC-V Decisions II **40**
9.1 Logical instructions . . . 40
9.2 Machine programs . . . 40
9.3 Function calls . . . 41

10 RISC-V Procedures **43**
10.1 A function call example . . . 43
10.2 Nested calls and register conventions . . . 44
10.3 Memory allocation . . . 45

11 RISC-V Instruction Formats I **47**
11.1 The stored-program computer . . . 47
11.2 Instructions as numbers . . . 47
11.3 R-format . . . 48
11.4 I-format . . . 48
11.5 S-format . . . 49

12 RISC-V Instruction Formats II **50**
12.1 Conditional branches and PC-relative addressing . . . 50
12.2 B-format . . . 50
12.3 U-format and long immediates . . . 51
12.4 J-format and JALR . . . 52

13 Compilation, Assembly, Linking, Loading **53**
13.1 Interpretation and translation . . . 53
13.2 Compiler and assembler . . . 53
13.3 Linker . . . 55
13.4 Loader . . . 56
13.5 Example: Hello world . . . 57

14 Intro to Synchronous Digital Systems **59**
14.1 Switches . . . 59
14.2 Transistors . . . 60
14.3 Signals and waveforms . . . 61
14.4 Types of circuits . . . 62

15 State, State Machines **63**
15.1 The accumulator . . . 63
15.2 Flip-flops . . . 63
15.3 Accumulator timing . . . 64
15.4 Maximum clock frequency . . . 65
15.5 Pipelining for performance . . . 65
15.6 Finite state machines . . . 66
15.7 The general model . . . 67

16 Combinational Logic **68**
16.1 Truth tables . . . 68
16.2 Logic gates . . . 68
16.3 From truth tables to gates . . . 69
16.4 Boolean algebra . . . 70
16.5 Canonical forms . . . 71

17 Combinational Logic Blocks **72**
17.1 Data multiplexors . . . 72
17.2 Arithmetic and logic unit . . . 73
17.3 Adder/subtractor . . . 73
17.4 Overflow . . . 74
17.5 Subtractor . . . 75

18 Single-Cycle CPU Datapath I **76**
18.1 One instruction per cycle . . . 76
18.2 Datapath elements . . . 77
18.3 R-type instructions: add and sub . . . 78
18.4 I-type arithmetic: addi . . . 79

19 Single-Cycle CPU Datapath II **81**
19.1 Loads . . . 81
19.2 Stores . . . 81
19.3 Branches . . . 82
19.4 Jumps . . . 83
19.5 Upper immediates . . . 84
19.6 The complete datapath . . . 84

20 Single-Cycle CPU Control **86**
20.1 Control and status registers . . . 86
20.2 Datapath control examples . . . 87
20.3 Instruction timing . . . 87
20.4 Control logic design . . . 89

21 Pipelining I **91**
21.1 Instruction timing in the single-cycle datapath . . . 91
21.2 Performance measures . . . 92
21.3 The iron law of processor performance . . . 92
21.4 Energy efficiency . . . 93
21.5 Pipelining: the laundry analogy . . . 94

22 Pipelining II **95**
22.1 Pipelining the RISC-V datapath . . . 95
22.2 The pipelined datapath . . . 96
22.3 Hazards . . . 97
22.4 Structural hazards . . . 97
22.5 Data hazards . . . 98

23 Pipelining III **100**
23.1 Load data hazards . . . 100
23.2 Control hazards . . . 101
23.3 Superscalar processors . . . 101
23.4 Pipelining and ISA design . . . 102

24 Caches I **103**
24.1 Binary prefixes . . . 103
24.2 Why memory is slow . . . 103
24.3 Memory caching and the hierarchy . . . 104
24.4 Locality . . . 105

25 Caches II **106**
25.1 Direct-mapped caches . . . 106
25.2 Tag, index, and offset . . . 107
25.3 Memory access with a cache . . . 108
25.4 Terminology and the valid bit . . . 108

26 Caches III **110**
26.1 Reading from a direct-mapped cache . . . 110
26.2 Multiword blocks in hardware . . . 111
26.3 Write hits . . . 111
26.4 Block size . . . 112
26.5 The three Cs and full associativity . . . 112

27 Caches IV **114**
27.1 N-way set-associative caches . . . 114
27.2 Block replacement . . . 115
27.3 Average memory access time . . . 116
27.4 Actual CPUs and caching in general . . . 117

28 OS and Virtual Memory Intro **118**
28.1 From a processor to a computer . . . 118
28.2 What the OS needs from hardware . . . 118
28.3 Booting and launching applications . . . 119
28.4 Supervisor mode and syscalls . . . 119
28.5 Interrupts, exceptions, and traps . . . 120
28.6 Multiprogramming and the need for virtual memory . . . 121

29 Virtual Memory I **122**
29.1 Virtual memory concepts . . . 122
29.2 Physical memory and storage . . . 122
29.3 The memory manager . . . 123
29.4 Paged memory . . . 123
29.5 Page faults . . . 124

30 Virtual Memory II **126**
30.1 Hierarchical page tables . . . 126
30.2 Translation lookaside buffers . . . 127
30.3 Virtual memory performance . . . 128

31 I/O **130**
31.1 Interacting with devices . . . 130
31.2 Processor and I/O speeds . . . 130
31.3 Polling . . . 130
31.4 Interrupts and DMA . . . 131
31.5 Networks . . . 133

32 Flynn Taxonomy, SIMD Instructions **134**
32.1 Matrix multiplication as a reference problem . . . 134
32.2 Flynn's taxonomy . . . 135

32.3 SIMD architectures . . . 136
32.4 Intel SSE intrinsics . . . 137

33 Thread-Level Parallelism I **139**
33.1 Improving performance . . . 139
33.2 Multicore processors . . . 139
33.3 Threads . . . 141
33.4 Multithreading . . . 141

34 Thread-Level Parallelism II **143**
34.1 Parallel programming languages . . . 143
34.2 OpenMP . . . 143
34.3 Example: computing π . . . 144
34.4 Data races . . . 145
34.5 Synchronization and locks . . . 146

35 Thread-Level Parallelism III **147**
35.1 OpenMP loops and data races . . . 147
35.2 Hardware synchronization . . . 147
35.3 Synchronization in OpenMP . . . 148
35.4 Shared memory and caches . . . 149
35.5 Cache coherence . . . 150
35.6 False sharing and coherence misses . . . 151

36 MapReduce, Spark **152**
36.1 Amdahl's law . . . 152
36.2 Request-level and data-level parallelism . . . 152
36.3 MapReduce . . . 153
36.4 Spark . . . 154

37 Data Centers, Cloud Computing (WSC) **157**
37.1 Eras of computer hardware . . . 157
37.2 Warehouse-scale computers . . . 157
37.3 Performance in a WSC . . . 158
37.4 Power and power usage effectiveness . . . 159

38 Dependability: Parity, ECC, RAID **161**
38.1 Dependability via redundancy . . . 161
38.2 Dependability metrics . . . 161
38.3 Error detection and parity . . . 162
38.4 Hamming ECC . . . 163
38.5 RAID . . . 164

Lecture 1: Great Ideas in Computer Architecture, Intro

The subject is the hardware-software interface: what a programmer needs to know about the machine to achieve the highest possible performance. C is used because it is closer to the underlying hardware than languages such as Snap!, Python, or Java. That lets us discuss hardware features in higher-level terms, and it lets the programmer explicitly harness the hardware's parallelism.

1.1 Machine structures

The classic view of a computer is a stack of layers. Software consists of applications (such as a browser), the operating system, the compiler, and the assembler; the instruction set architecture (ISA) is the boundary; hardware consists of the processor, memory, and I/O system, implemented by datapath and control, digital design, circuit design, transistors, and fabrication. CS 61C spans the layers around the ISA, from the compiler and assembler down through datapath and control to digital design. Today the machines range from personal mobile devices and network edge devices (phones, cars, drones) to warehouse-scale computers in datacenters.

The modern view adds that software and hardware must *harness parallelism* at every level to achieve high performance:

- *parallel requests*, assigned to computers (a web search);
- *parallel threads*, assigned to cores (lookups, ads);
- *parallel instructions*, more than one instruction at a time (five pipelined instructions);
- *parallel data*, more than one data item at a time (adding four pairs of words at once);
- *hardware descriptions*, in which all logic gates work in parallel at the same time.

The matching hardware hierarchy runs from warehouse-scale computers and smartphones, through a computer (cores, memory and caches, input/output), a core (execution units, functional blocks), and main memory, down to logic gates.

1.2 Six great ideas

1. Abstraction (layers of representation and interpretation).
2. Moore's law.
3. The principle of locality and the memory hierarchy.
4. Parallelism.

5. Performance measurement and improvement.
6. Dependability via redundancy.

Abstraction. A program passes through levels of representation. A high-level language program (C) is translated by a COMPILER into an assembly language program (RISC-V), which an ASSEMBLER turns into a machine language program: 32-bit binary words. The machine language is interpreted by hardware described by an architecture (block diagrams), which is implemented as logic circuits (circuit schematics). For example, the C statements that swap `v[k]` and `v[k+1]`,

```
temp = v[k];      lw x3, 0(x10)
v[k] = v[k+1];    lw x4, 4(x10)
v[k+1] = temp;    sw x4, 0(x10)
                  sw x3, 4(x10)
```

become the four RISC-V instructions on the right, each of which is then encoded as a 32-bit number. Anything can be represented as a number: data or instructions.

Moore's law. In 1965 Gordon Moore (Intel cofounder) noted that reduced cost is one of the big attractions of integrated electronics, and that this advantage grows as technology moves toward larger and larger circuit functions on a single chip. He predicted that the number of transistors per chip would double every two years. From about 1970 to 2005, the transistor counts of memory chips (from 1K to 2G bits) and of microprocessors (from the 4004 to the Itanium 2) followed this exponential trend.

Locality and the memory hierarchy. Jim Gray's storage latency analogy measures how far away data is by scaling access times (in ns) to human distances and times:

Storage	Latency (ns)	Analogy	Time
Registers	1	my head	1 min
On-chip cache	2	this room	
On-board cache	10	this campus	10 min
Memory	100	Sacramento	1.5 hr
Disk	10^6	Pluto	2 years
Tape or optical robot	10^9	Andromeda	2,000 years

The MEMORY HIERARCHY is therefore a pyramid. At the top, inside the CPU core, are the registers: extremely fast, extremely expensive, tiny capacity. Below them are the CPU caches (level 1, 2, and 3): faster, expensive, small capacity. Next is physical memory, random-access memory (RAM) built from DRAM chips such as DDR3/4/5 or HBM/HBM2/3: fast, reasonably priced, medium capacity. At the bottom, backing virtual memory, are solid-state (flash) drives and magnetic disks: cheap with large capacity.

Parallelism. Parallelism appears in several forms. PIPELINING splits the execution of each instruction into stages and overlaps the stages of successive instructions (Figure 1): in time slot 3, instruction 1 is being executed, instruction 2 is being decoded, and instruction 3 is being fetched from memory. In a multithreaded program, a sequential preamble calls `fork()` to start several worker threads, a `join()` waits for them all, and sequential post-processing follows. Independent tasks can simply be handed to different workers, as when three people each process one urgent folder.

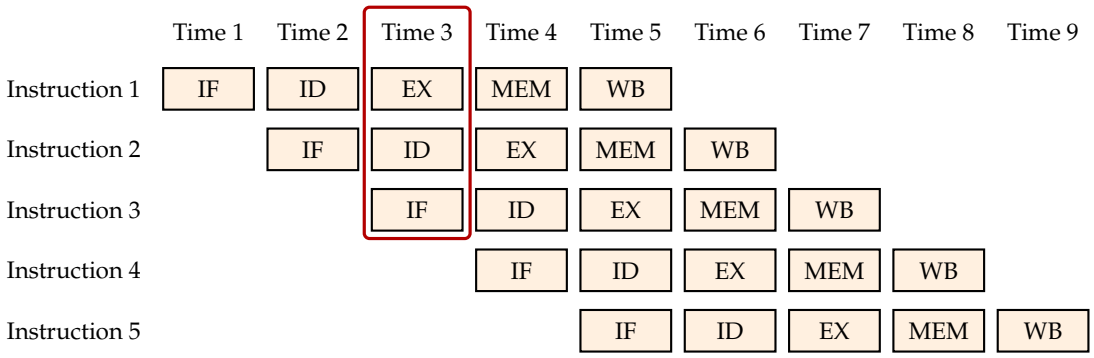


Figure 1: Five instructions in a pipeline. Each passes through instruction fetch (IF), instruction decode (ID), execute (EX), memory access (MEM), and write back (WB), one stage per time slot. The boxed column is time slot 3.

Note. *Amdahl's law* limits the benefit of parallelism. If a fraction x of the code must execute sequentially and the rest is spread over N CPU cores, then

$$\text{performance increase ratio} = \frac{1}{x + \dfrac{1-x}{N}}.$$

With $x = 0$ the ratio grows linearly in N; with $x = 10\%$, 20%, or 50% it flattens as cores are added, since it can never exceed $1/x$ (for example $10\times$ when $x = 10\%$). There is no significant throughput improvement if the fraction of code that can execute in parallel is low.

Performance measurement and improvement. Performance comes from matching the application to the underlying hardware to exploit locality, parallelism, and special hardware features such as specialized instructions (for example, for matrix manipulation). One must distinguish LATENCY from THROUGHPUT: how long it takes to set a problem up and complete it (or how many tasks can be completed in a given time), versus how much faster it executes once it gets going. Latency is all about the time to finish.

Dependability via redundancy. Redundancy keeps a failing piece from making the whole system fail. If three units each compute $1 + 1$ and one of them fails and answers 1, the other two still agree on 2, and the majority answer (2 of 3) is correct. Increasing transistor density reduces the cost of redundancy. The idea applies at every scale: redundant datacenters, so that losing one keeps an

Internet service online; redundant disks (Redundant Arrays of Independent Disks, RAID), so that losing a disk loses no data; and redundant memory bits (Error Correcting Code, ECC, memory), so that losing a bit loses no data.

1.3 Why architecture is exciting today

Computing has moved through eras with their own applications: mainframes (scientific computing, data processing), personal computers (the web), smartphones (apps), and now healthcare and wellness, telemedicine, education, autonomous driving, gaming, VR/AR entertainment, and visualization, backed by the cloud. The number of deployed devices keeps growing but there is no single killer application, so needs and architectures diversify; machine learning is common in most domains.

At the same time the constraints have changed: power limits and the end of Moore's law. Long-term trends show transistor counts still rising while clock frequency, typical power, and application performance have levelled off, and the number of cores per chip has grown instead. Each domain therefore needs a heterogeneous system of multiple processor cores, GPUs, accelerators, interfaces, and memory (the Apple A13 Bionic is an example).

The old conventional wisdom was that Moore's law plus Dennard scaling gave faster, cheaper, lower-power general-purpose computers every year (in the glory days, a 1% performance improvement per week), so it was pointless to compete by designing parallel or specialized computers: by the time the design was finished, the next generation of general-purpose machines would beat it. The new conventional wisdom is domain-specific hardware, for example Google's TPU3, a specialized engine for neural-network training deployed in the cloud in systems of 1024 chips delivering more than 100 petaFLOPs.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 1 (Fall 2020).

Lecture 2: Number Representation

2.1 Bits can represent anything

The real world is analog. To import analog information we must do two things: SAMPLE it (a CD asks a music signal how loud it is 44,100 times per second) and QUANTIZE each sample (find where it lies on a 16-bit "yardstick" with 65,536 tick marks). Digital data need not be born analog, however; a ray-traced image, for example, is computed directly.

Bits can represent anything. The 26 letters need 5 bits ($2^5 = 32$); upper and lower case plus punctuation need 7 bits (stored in 8), the ASCII code; a standard code covering all the world's languages (Unicode) uses 8, 16, or 32 bits. Logical values use 0 for false and 1 for true; two bits can name colors (red 00, green 01, blue 11); bits can also encode locations, addresses, and commands. The rule to memorize:

N bits can represent at most 2^N things.

2.2 Bases and conversions

In base 10 the digits are 0–9 and $3271 = 3271_{10} = (3 \times 10^3) + (2 \times 10^2) + (7 \times 10^1) + (1 \times 10^0)$. In base 2 the digits are 0 and 1 (binary digits, or BITS); the numeral 1101 is written 1101_2 or `0b1101`, and

$$1101_2 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 8 + 4 + 0 + 1 = 13.$$

In base 16 (HEXADECIMAL) the digits are 0–9 and A–F (worth 10–15), and `0xA5` $= A5_{16} = (10 \times 16^1) + (5 \times 16^0) = 165$.

To convert a decimal number n to another base, write the columns (powers of the base) and go from left to right: if the column value is at most n, write how many copies of it fit in n and subtract that many; otherwise write 0; stop when n reaches 0. Thus 13 in the columns $8, 4, 2, 1$ gives 1101 ($13 - 8 = 5$, $5 - 4 = 1$, $1 - 1 = 0$), and 165 in the columns $4096, 256, 16, 1$ gives 00A5 (10 sixteens leave 5).

Binary and hex convert by table lookup, because one hex digit is exactly four bits:

Hex	Binary	Hex	Binary	Hex	Binary	Hex	Binary
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

For binary to hex, left-pad with 0s to whole 4-bit groups and look each group up: `0b11110` = `0b00011110` = `0x1E`. For hex to binary, look up each digit and drop leading 0s: `0x1E` = `0b00011110` = `0b11110`.

Four bits form a NIBBLE: one hex digit, 16 things. Eight bits form a BYTE: two hex digits, 256 things. A color is usually a byte each of red, green, and blue (0–255), so `#D0367F` names one color.

Decimal is best for humans doing arithmetic. Hex is best for a human reading long strings of binary, at 4 bits per symbol, but terrible for arithmetic on paper. Binary is what computers use: to a computer a number is always binary, however it is written, so $32_{\text{ten}} = 32_{10} =$ `0x20` $= 100000_2 =$ `0b100000`. The subscripts "ten", "hex", and "two" resolve ambiguity. C can print and read all of these forms; the program below prints `Decimal: 1234`, `Hex: 4d2`, and `Octal: 2322`, and each literal line reports 1234:

```c
#include <stdio.h>

int main() {
  const int N = 1234;

  printf("Decimal: %d\n", N);
  printf("Hex:     %x\n", N);
  printf("Octal:   %o\n", N);

  printf("Literals (not supported by all compilers):\n");
  printf("0x4d2         = %d (hex)\n", 0x4d2);
  printf("0b10011010010 = %d (binary)\n", 0b10011010010);
  printf("02322         = %d (octal, prefix 0 - zero)\n", 02322);
}
```

2.3 Unsigned numbers and overflow

With number representations we want to add, subtract, multiply, divide, and compare. Binary addition is so simple that circuits can do it: $1010_2 + 0111_2 = 10001_2$ is $10 + 7 = 17$. Subtraction works as in decimal.

Bit patterns are only representatives of numbers (strictly speaking, NUMERALS). A numeral really has infinitely many digits, almost all equal (00 . . . 0 or 11 . . . 1) except for a few on the right; we just do not show the leading ones. If the result of an operation cannot be represented in the rightmost bits the hardware keeps, an OVERFLOW has occurred. Unsigned 5-bit numbers run from 00000 (0) up the "binary odometer" to 11111 (31); one more wraps around to 00000. In C these are `unsigned int` and C18's `uint`N`_t`.

2.4 Negative numbers

Sign and magnitude. The obvious solution makes the leftmost bit the sign (0 for +, 1 for −) and the remaining bits the magnitude. Its shortcomings: the arithmetic circuit is complicated (special steps depending on whether the signs agree); there are two zeros, `0x00000000` = +0 and `0x80000000` = −0; and incrementing the binary odometer sometimes increases the value and sometimes decreases it. Sign and magnitude is therefore used only in signal processors.

One's complement. Negate by complementing every bit: $7_{10} = 00111_2$ and $-7_{10} = 11000_2$. Positive numbers have leading 0s and negative numbers leading 1s, and −00000 = 11111. In N bits there are $2^{N-1} - 1$ positive and $2^{N-1} - 1$ negative numbers, plus two zeros. The arithmetic is still somewhat complicated and the two zeros remain (`0x00000000` = +0, `0xFFFFFFFF` = −0), so although some computers used it, one's complement was abandoned for a better solution.

Two's complement. In one's complement the negative mappings "overlap" the positive ones (the two zeros); shifting the negative mappings by one fixes this. To negate, complement the bits and then add 1. As before, leading 0s mean ≥ 0 and leading 1s mean < 0, but now $1\ldots1111$ is −1, not −0. This is TWO'S COMPLEMENT, C's `int` and C18's `int`N`_t` (a "signed integer"); it makes the hardware simple. Each bit is worth its place value, except that the leftmost bit's weight is negative:

$$d_{31} \times \left(-2^{31}\right) + d_{30} \times 2^{30} + \cdots + d_2 \times 2^2 + d_1 \times 2^1 + d_0 \times 2^0.$$

Example: In a nibble, $1101_2 = 1 \times (-2^3) + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = -8 + 4 + 0 + 1 = -3$. Negating twice returns to the start: from x = 1101 (−3), complementing gives 0010 and adding 1 gives 0011 (+3); complementing that gives 1100 and adding 1 gives 1101 (−3) again.

For $N = 5$, the two's complement values run from −16 (10000) to 15 (01111): 2^{N-1} non-negatives, 2^{N-1} negatives, one zero, and hence $2^{N-1} - 1 = 15$ positives.

Bias encoding. Another way to center a range on zero (think of an electrical signal from 0 V to 31 V) is to store a number as unsigned and add a fixed BIAS: value = unsigned + bias. For N bits the bias is chosen as $-(2^{N-1} - 1)$, which is −15 for $N = 5$: 00000 is −15, 01111 is 0, and 11111 is 16. There is one zero and 16 positives, and the binary odometer counts upward through the values in order.

The table compares the five integer encodings on a few 5-bit patterns. One's complement and sign and magnitude have the most problems; unsigned (C18's `uint`N`_t`) and two's complement (C99's `int`N`_t`) are universal.

Bits	Unsigned	Sign and magnitude	One's complement	Two's complement	Bias −15
00000	0	+0	+0	0	−15
00001	1	1	1	1	−14
01111	15	15	15	15	0
10000	16	−0	−15	−16	1
10001	17	−1	−14	−15	2
11110	30	−14	−1	−2	15
11111	31	−15	−0	−1	16

Two design lessons recur. Design decisions are often made to keep the hardware simple. And there is no free lunch: numbers are infinite but computers are finite, so every encoding has limits, and exceeding them produces errors.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 2 (Fall 2020).

Lecture 3: C Intro: Basics

3.1 Stored-program computers and C

ENIAC (University of Pennsylvania, 1946) was the first electronic general-purpose computer. It was blazingly fast for its time, multiplying two 10-digit decimal numbers in 2.8 ms, but setting up a new program took two to three days because it was programmed with patch cords and switches. (Until then, a "computer" was usually a person who did calculations.) EDSAC (Cambridge, 1949) was the first general STORED-PROGRAM computer: programs were held as numbers in memory, using 35-bit two's complement words. This is the revolution: the program is the same type of data that the computer computes on, so bits are not just the numbers being manipulated but also the instructions for manipulating them.

C, in Kernighan and Ritchie's words, "is not a 'very high-level' language, nor a 'big' one, and is not specialized to any particular area of application. But its absence of restrictions and its generality make it more convenient and effective for many tasks than supposedly more powerful languages." It made possible the first operating system not written in assembly language, UNIX, which was therefore portable. We use C because it lets programs exploit features of the underlying architecture: memory management, special instructions, and parallelism. C and its derivatives (C++, Objective-C, C#) remain among the most popular languages after more than 40 years, although a new performance-critical project should consider Rust ("C but safe": by the time C code is correct with all necessary checks, it should be no faster than Rust) or Go (practical concurrent programming for multicore processors).

The key C concepts are pointers, arrays, and their implications for memory management. They are all *unsafe*: if a program contains an error in these areas it might not crash immediately, but instead leave the program in an inconsistent (and often exploitable) state.

3.2 Compilation

C compilers map C programs directly into architecture-specific machine code (strings of 1s and 0s). Java instead converts programs to architecture-independent bytecode, which may then be compiled by a just-in-time (JIT) compiler, and Python environments convert programs to bytecode at runtime. These approaches differ mainly in *when* the program is converted to low-level machine instructions (the "levels of interpretation"). For C it is generally a two-part process: compiling each `.c` file to a `.o` object file, then LINKING the `.o` files into an executable (Figure 2). Assembling also happens, but it is hidden by default.

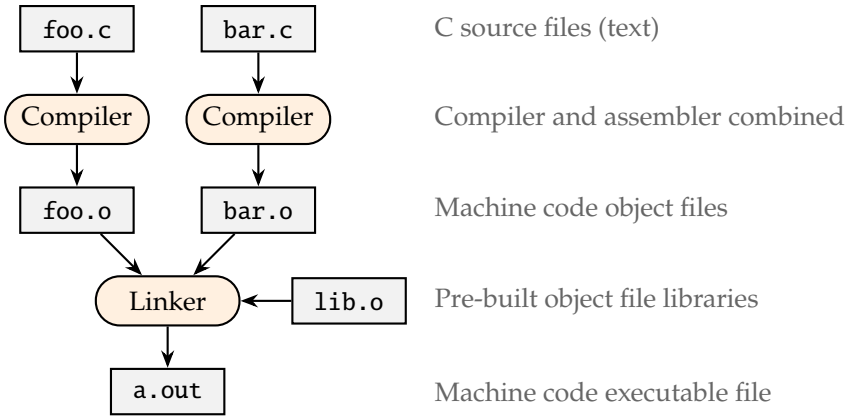


Figure 2: Simplified C compilation. Each source file is compiled separately (the compiler and assembler are combined here), and the linker joins the object files with pre-built libraries into one executable.

Compilation has advantages. Compile times are reasonable, because tools such as `make` recompile only modified files. Run-time performance is excellent, generally much faster than Scheme or Java for comparable code, because the compiler optimizes for a given architecture. (These days much performance also lives in libraries: plenty of scientific computing is done in Python, using good libraries for GPU resources, driving many machines at once, or calling low-level C code through Cython.)

It also has disadvantages. Compiled files, including the executable, are specific to the processor type (MIPS, x86, RISC-V) and the operating system (Windows, Linux, macOS), so the executable must be rebuilt on each new system ("porting" the code). The change, compile, run cycle can be slow during development; `make` rebuilds only the changed pieces and can compile in parallel (`make -j`), but the linker is sequential, so Amdahl's law applies.

The C preprocessor. Source files first pass through a macro processor, CPP, before the compiler sees them (`foo.c` becomes `foo.i`). CPP replaces each comment with a single space and acts on commands that begin with `#`: `#include "file.h"` inserts `file.h` into the output, `#include <stdio.h>` looks for the file in the standard location (but there is no actual difference), `#define PI (3.14159)` defines a constant, and `#if`/`#endif` conditionally include text. The `-save-temps` option to `gcc` shows the preprocessed result.

Preprocessor macros are often used as small "functions", but they are not functions: `#define` only performs string replacement on the program text. With the macro on the first line below, the statement on the second line expands to the third, which may call `foo` twice, an error if `foo(z)` has a side effect:

```
#define min(X,Y) ((X)<(Y)?(X):(Y))
next = min(w, foo(z));
next = ((w)<(foo(z))?(w):(foo(z)));
```

3.3 C versus Java

	C	Java
Type of language	Function oriented	Object oriented
Programming unit	Function	Class (abstract data type)
Compilation	`gcc hello.c` creates machine language code	`javac Hello.java` creates Java virtual machine bytecode
Execution	`a.out` loads and executes the program	`java Hello` interprets the bytecodes
Storage	Manual (`malloc`, `free`)	`new` allocates and initializes; garbage collection frees automatically
Comments	`/* ... */` (C99 also `//`)	`/* ... */` or `//` to end of line
Constants	`#define`, `const`	`final`
Preprocessor	Yes	No
Variable declaration	At the beginning of a block (C99: as in Java)	Before you use it
Naming convention	`sum_of_squares`	`sumOfSquares`
Accessing a library	`#include <stdio.h>`	`import java.io.File;`

The operators are nearly identical: arithmetic (`+ - * / %`), assignment (=) and augmented assignment (`+= -= *= /= %= &= |= ^= <<= >>=`), bitwise logic (`~ & | ^`), shifts (`<< >>`), boolean logic (`! && ||`), equality (`== !=`), grouping (`()`), order relations (`< <= > >=`), increment and decrement (`++ --`), member selection (`.` and `->`, which differ slightly from Java because C has both structures and pointers to structures), and conditional evaluation (`? :`).

ANSI C has been updated. C99 (compile with `gcc -std=c99`; then `__STDC_VERSION__` is `199901`) added declarations in `for` loops, Java-like `//` comments, variable-length non-global arrays, explicit integer types in `<inttypes.h>`, and booleans in `<stdbool.h>`. C11 (`gcc -std=c11`, version `201112L`; its bug-fix revision C18 is `-std=c17`, `201710L`) added multithreading support, Unicode strings and constants, removal of `gets()`, type-generic macros (dispatch based on type), complex values, static assertions, and exclusive create-and-open.

To accept command-line arguments, `main` is declared `int main(int argc, char *argv[])`. Here `argc` is the number of strings on the command line, counting the executable itself, so `argc` is 2 for `sort myFile`; `argv` is a pointer to an array holding the arguments as strings.

3.4 C syntax

In C, 0 (an integer), `NULL` (a pointer), and the false value of C99's `stdbool.h` evaluate to false; everything else is true (as in Scheme, where only `#f` is false).

Every variable must be declared with the type of data it will hold, and the type cannot change (`int var = 2;`):

Type	Description	Examples
`int`	Integers, including negatives; at least 16 bits, can be larger	`0, 78, -217, 0x7337`
`unsigned int`	Unsigned integers	`0, 6, 35102`
`float`	Floating point decimal	`0.0, 3.14159, 6.02e23`
`double`	Equal or higher precision floating point	`0.0, 3.14159, 6.02e23`
`char`	Single character	`'a', 'D', '\n'`
`long`	Longer `int`; size ≥ `sizeof(int)`, at least 32 bits	`0, 78, -217, 301720971`
`long long`	Even longer `int`; size ≥ `sizeof(long)`, at least 64 bits	`31705192721092512`

An `int` should be the integer type the target processor works with most efficiently. The only guarantees are `sizeof(long long)` ≥ `sizeof(long)` ≥ `sizeof(int)` ≥ `sizeof(short)`, with `short` at least 16 bits and `long` at least 32 bits; all of them could be 64 bits. So `sizeof(int)` is 16, 32, or 64 bits depending on the computer, whereas Java's `int` is always 32 bits and Python's plain ints are at least 32 bits (its long ints are unbounded). This is why the fixed-width types `int`N`_t` and `uint`N`_t` are preferred.

A `const` is assigned a typed value once, in its declaration, and cannot change during the whole execution (`const float golden_ratio = 1.618;`); any standard type can be made constant. An `enum` is a group of related integer constants, as in `enum color {RED, GREEN, BLUE};`.

Functions declare their return type to the left of the name (any C variable type, or `void` when nothing is returned) and the types of their parameters, as in `int number_of_people() { return 3; }`. Variables and functions *must* be declared before they are used. `typedef` defines new type names, and a `struct` groups variables, accessed with dot notation:

```
typedef uint8_t BYTE;
BYTE b1, b2;

typedef struct {
  int length_in_seconds;
  int year_recorded;
} SONG;

SONG song1;
song1.length_in_seconds = 213;
song1.year_recorded     = 1994;
```

Control flow within a function is remarkably close to Java's. A statement is either a single statement or a `{}` block, and the forms are `if (`*expr*`)` *stmt* (optionally followed by `else` *stmt*), `while (`*expr*`)` *stmt*, `do` *stmt* `while (`*expr*`);` and `for (`*initialize*`;` *check*`;` *update*`)` *stmt*. A chain of `if`/`else` without braces is ambiguous, so always use `{}` blocks; omitting them has caused real bugs. In a `switch (`*expr*`)` with `case` labels and a `default`, execution keeps falling through the following cases until a

`break`. C also has `goto`, which can produce spectacularly bad code; do not use it.

A first complete program prints a table of sines:

```c
#include <stdio.h>
#include <math.h>

int main(void) {
  int    angle_degree;
  double angle_radian, pi, value;

  printf("Compute a table of the sine function\n\n");
  pi = 4.0*atan(1.0);  /* could also just use pi = M_PI */
  printf("Value of PI = %f \n\n", pi);
  printf("Angle\tSine\n");
  angle_degree = 0;    /* initial angle value */
  while (angle_degree <= 360) {  /* loop til angle_degree > 360 */
    angle_radian = pi * angle_degree / 180.0;
    value = sin(angle_radian);
    printf("%3d\t%f\n", angle_degree, value);
    angle_degree += 10;          /* increment the loop index */
  }
  return 0;
}
```

It prints `Value of PI = 3.141593` and then the rows `0 0.000000`, `10 0.173648`, `20 0.342020`, . . . , `90 1.000000`, and so on up to 360°.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 3 (Fall 2020).

Lecture 4: C Intro: Pointers, Arrays, Strings

4.1 Declarations and undefined behavior

Variable declarations in C resemble Java's, with a few important differences. In ANSI C every declaration must appear before the variable is used, at the beginning of a block: `{ int a = 0, b = 10; ...` is correct, whereas `for (int i=0; ...` is incorrect in ANSI C but correct in C99 and later. A variable may be initialized in its declaration; if it is not, it holds garbage, and its contents are undefined.

Much of C has UNDEFINED BEHAVIOR: the program may run one way on one computer and another way on another, or even differently each time it is executed. Such bugs are often called HEISENBUGS: they seem random and hard to reproduce, and they seem to disappear or change when debugged (in contrast to repeatable BOHRBUGS).

4.2 Addresses and pointers

Think of memory as a single huge array. Each cell has an ADDRESS and stores a VALUE; addresses are unsigned numbers, since there are no negative addresses. Do not confuse the address of a memory location with the value stored there. For now the abstraction is an unbounded memory numbered from 0.

Definition: A POINTER is a variable that contains the address of a variable; it "points to" that memory location.

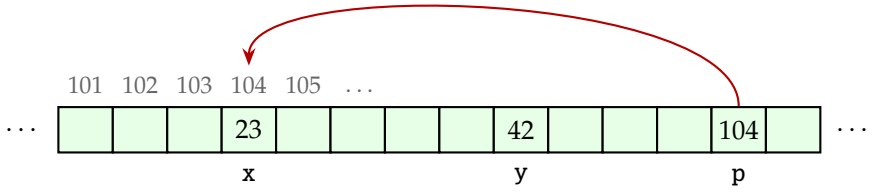


Figure 3: Memory as an array of cells. The variable `x` at address 104 holds 23, `y` holds 42, and the pointer `p` holds 104, the address of `x`.

The syntax has two operators. In `int *p;` the `*` declares that `p` holds the address of an `int`. The ADDRESS OPERATOR `&` gets the address of a variable, so `p = &y;` assigns the address of `y` to `p`. The DEREFERENCE OPERATOR `*` gets the value at an address, so `z = *p;` assigns the value pointed to by `p` to `z`. Thus the `*` is used in two different ways, and a dereference on the left of an assignment changes the variable pointed to:

```c
int *p, x;   // p and x hold garbage
x = 3;
p = &x;      // p now points to x
printf("p points to %d\n", *p);   // prints 3
*p = 5;      // dereference on the left of =: x is now 5
```

Java and C pass parameters *by value*: a function gets a copy of each argument, so changing the copy cannot change the original. With `void addOne(int x) { x = x + 1; }`, after `int y = 3; addOne(y);` the value of `y` is still 3. To let a function change a value, pass its address: with `void addOne(int *p) { *p = *p + 1; }`, the call `addOne(&y);` makes `y` equal to 4.

Declaring a pointer allocates space only for the pointer, not for anything to point to, and local variables in C are not initialized. So `void f() { int *ptr; *ptr = 5; }` stores 5 at whatever garbage address `ptr` happens to hold.

Pointers are worth this danger. Passing a large struct or array by pointer is easier and faster than copying the whole thing, and pointers generally allow cleaner, more compact code. But they are probably the single largest source of bugs in C, especially with dynamic memory management, where they cause dangling references and memory leaks.

4.3 Using pointers effectively

Pointers can point to any data type (`int`, `char`, a struct, and so on), but normally a pointer points to only one type. The generic pointer type `void *` can point to anything; use it sparingly, since it invites bugs and security problems. There are even pointers to functions: `int (*fn)(void *, void *) = &foo;` declares `fn` as a pointer to a function that accepts two `void *` pointers and returns an `int`, initially pointing to `foo`, and `(*fn)(x, y)` then calls it.

For structures, dot notation reaches a field of a struct and arrow notation a field through a pointer to a struct:

```c
typedef struct { int x; int y; } Point;
Point p1, p2, *paddr;
int h = p1.x;      // dot notation
p2.y = p1.y;
h = paddr->x;      // arrow notation,
h = (*paddr).x;    // equivalent to this
p1 = p2;           // struct assignment works too
```

The pointer of all 0s is special: the `NULL` pointer. Reading or writing through a null pointer should crash the program. Since 0 is false, tests are easy: `if (!p) { /* p is a null pointer */ }` and `if (q) { /* q is not a null pointer */ }`.

Modern machines are BYTE-ADDRESSABLE: memory consists of 8-bit cells, each with a unique address. A C pointer is just an abstracted memory address, and its declared type tells the compiler how many

bytes to fetch on each access through it; a 32-bit integer, for example, occupies four consecutive bytes (Figure 4). We also want WORD ALIGNMENT: some processors do not allow 32-bit values to be accessed except on 4-byte boundaries, and others are just very slow on unaligned accesses.

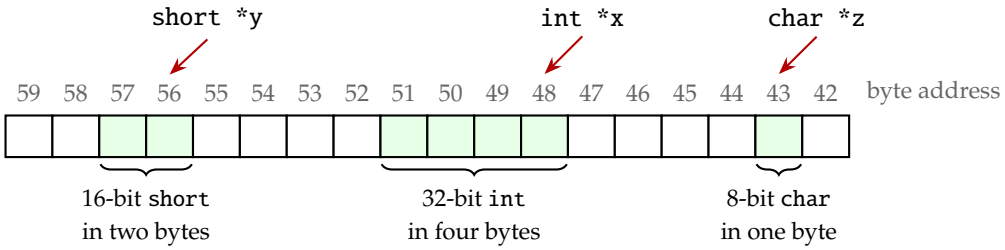


Figure 4: Objects of different sizes in byte-addressed memory. Each pointer holds the address of its object's lowest byte: `y` points to the `short` at 56, `x` to the `int` at 48, and `z` to the `char` at 43.

4.4 Arrays

`int ar[2];` declares a two-element integer array, which is just a block of memory; `int ar[] = {795, 635};` declares and fills one; and `ar[num]` returns element `num`. Arrays are almost identical to pointers: `char *string` and `char string[]` are nearly identical declarations, differing in subtle ways (incrementing, and declaring filled arrays). The key concept is that *an array variable is a "pointer" to its first element*. Consequently `ar[0]` is the same as `*ar` and `ar[2]` is the same as `*(ar+2)`, and pointer arithmetic can be used to access arrays conveniently.

POINTER ARITHMETIC scales by the size of the pointed-to type: `pointer + n` adds `n*sizeof(`*type pointed to*`)` to the address, and `pointer - n` subtracts the same amount.

A declared array is allocated only while its scope is valid, so returning one is incorrect:

```
char *foo() {
  char string[32]; ...;
  return string;   // incorrect: string dies when foo returns
}
```

An array of size n is accessed with indices 0 to $n - 1$. Declare the size once, in a variable, and use that variable for both the declaration and the loop bound, rather than repeating the number:

```
int ARRAY_SIZE = 10;
int i, a[ARRAY_SIZE];
for (i = 0; i < ARRAY_SIZE; i++) { ... }
```

This keeps a single source of truth (indirection instead of two copies of the number 10). A C array does not know its own length, and bounds are not checked. We can therefore accidentally access past the end of an array, and we must pass both an array and its size to any procedure that traverses it. The resulting segmentation faults and bus errors are very difficult to find.

4.5 Pointers to pointers

Since parameters are passed by value, a function cannot change a pointer it is given:

```
void IncrementPtr(int *p) { p = p + 1; }
int A[3] = {50, 60, 70};
int *q = A;
IncrementPtr(q);
printf("*q = %d\n", *q);   // *q = 50
```

Only the copy `p` moved. The fix is to pass a pointer to the pointer, a HANDLE, declared `int **h`:

```
void IncrementPtr(int **h) { *h = *h + 1; }
int A[3] = {50, 60, 70};
int *q = A;
IncrementPtr(&q);
printf("*q = %d\n", *q);   // *q = 60
```

4.6 Function pointer example

A function pointer lets one routine apply any function to an array. Here `mutate_map` replaces each element by `(*fp)` of it, and `print_array` prints the `n` elements on one line:

```
int x2 (int n) { return 2*n; }
int x10(int n) { return 10*n; }
void mutate_map(int A[], int n, int(*fp)(int)) {
  for (int i = 0; i < n; i++)
    A[i] = (*fp)(A[i]);
}
int main(void) {
  int A[] = {3,1,4}, n = 3;
  print_array(A, n);        // 3 1 4
  mutate_map(A, n, &x2);
  print_array(A, n);        // 6 2 8
  mutate_map(A, n, &x10);
  print_array(A, n);        // 60 20 80
}
```

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 4 (Fall 2020).

Lecture 5: C Memory Management

5.1 Dynamic memory allocation

The operator `sizeof()` gives the size in bytes of a type or variable. Assuming the size of an object is misleading and bad style (many years ago an `int` was 16 bits, and programs were written with that assumption), so use `sizeof(`*type*`)`. `sizeof` also knows the size of arrays, including arrays whose size is determined at run time: after `int ar[3];` (or `int n = 3; int ar[n];`), `sizeof(ar)` is 12 with 4-byte `int`s.

To allocate room for something new to point to, use `malloc()` with a typecast and `sizeof`: `ptr = (int *) malloc(sizeof(int));` makes `ptr` point to a space somewhere in memory of `sizeof(int)` bytes. The TYPECAST `(int *)` simply tells the compiler what will go into that space. Since `malloc` is almost never used for one variable, the usual form is `ptr = (int *) malloc(n*sizeof(int))`, which allocates an array of `n` integers.

Once `malloc()` is called, the memory contains garbage, so do not use it until its value is set. After dynamically allocating space we must dynamically free it with `free(ptr)`. Even though the program frees all memory on exit (or when `main` returns), do not be lazy: `main` may later be turned into a subroutine. Two mistakes make a program crash or behave strangely later, causing bugs that are very hard to find: `free()`ing the same memory twice, and calling `free()` on something not obtained from `malloc()`. The runtime does not check for these mistakes, because memory allocation is so performance-critical that there is no time to do so. The usual result is a corrupted memory allocator, which shows up much later in an unrelated part of the code.

`realloc(p, size)` resizes a previously allocated block at `p`. If `p` is `NULL` it behaves like `malloc`; if `size` is 0 it behaves like `free`. It returns the new address of the block, which has likely moved. Always check each result for `NULL`:

```
1 int *ip = (int *) malloc(10*sizeof(int));
2 ip = (int *) realloc(ip, 20*sizeof(int));  // first 10 elements retained
3 realloc(ip, 0);                            // identical to free(ip)
```

Arrays are not implemented as one might think. In

```c
void foo() {
  int *p, *q, x;
  int a[4];
  p = (int *) malloc(sizeof(int));
  q = &x;
  *p = 1;   // p[0] would also work here
  printf("*p:%u, p:%u, &p:%u\n", *p, p, &p);
  *q = 2;   // q[0] would also work here
  printf("*q:%u, q:%u, &q:%u\n", *q, q, &q);
  *a = 3;   // a[0] would also work here
  printf("*a:%u, a:%u, &a:%u\n", *a, a, &a);
}
```

the pointers `p` and `q` are variables with their own addresses, but the array name `a` is not (Figure 5). With the layout shown, the output is `*p:1, p:40, &p:12`, then `*q:2, q:20, &q:16`, then `*a:3, a:24, &a:24`: the value and the address of `a` are the same. As Kernighan and Ritchie put it, "an array name is not a variable."

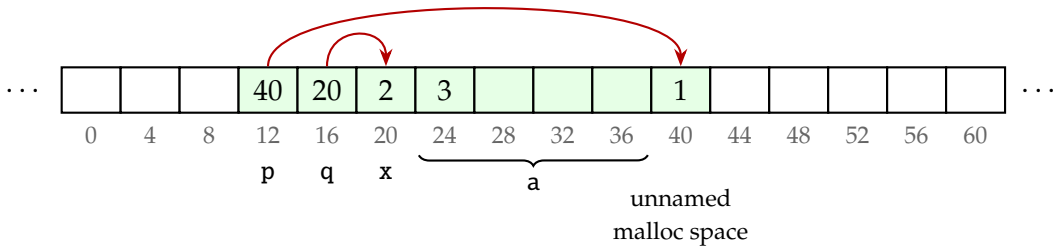


Figure 5: Memory for `foo()`, with byte addresses below the cells. `p` (at 12) points to the `malloc`'ed `int` at 40, `q` (at 16) points to `x` (at 20), and the 16-byte array `a` starts at 24.

In short: pointers and arrays are virtually the same, C knows how to increment pointers, and C is efficient but offers little protection (array bounds are not checked and variables are not automatically initialized). Handles change pointers, and dynamically allocated heap memory must be deallocated manually with `free()`. The cost of efficiency is more overhead for the programmer.

5.2 Linked list example

Structures, pointers, `malloc()`, and `free()` together implement a linked list of strings:

```
struct Node {
  char *value;
  struct Node *next;
};
typedef struct Node *List;

/* Create a new (empty) list */
List ListNew(void) { return NULL; }
```

```
/* add a string to an existing list */
List list_add(List list, char *string) {
  struct Node *node =
    (struct Node*) malloc(sizeof(struct Node));
  node->value =
    (char*) malloc(strlen(string) + 1);
  strcpy(node->value, string);
  node->next = list;
  return node;
}
```

Adding `"abc"` first allocates a node whose two fields hold garbage, then allocates `strlen(string)+1` bytes (the extra byte holds the terminating null character), still garbage, for the node's own copy of the string. `strcpy` copies `"abc"` into that space, `node->next` is set to the old list, and the new node is returned as the head of the longer list (Figure 6).

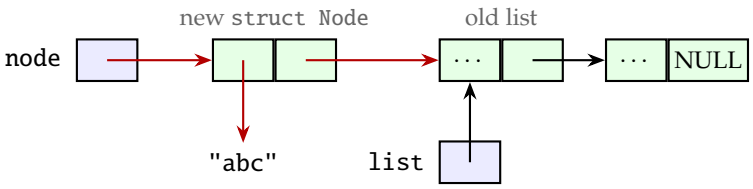


Figure 6: `list_add(list, "abc")` just before it returns. The pointers it sets are emphasized: `node` points to the new node, whose `value` is a heap copy of the string and whose `next` is the old list.

5.3 Memory locations

A structure declaration does not allocate memory; a variable declaration does. Memory for data is obtained in three ways: by declaring a local variable (`int i; struct Node list; char *string; int ar[n];`), by "dynamic" allocation at run time with an allocation function (`ptr = (struct Node *) malloc(sizeof(struct Node)*n)`), and by declaring data outside any procedure (before

`main`), which is like a local declaration but with global scope.

C therefore has three pools of memory. STATIC STORAGE holds global variables, essentially permanently, for the entire program run. The STACK holds local variables, parameters, and return addresses (the "stack frame" in C, an "activation record" in Java). The HEAP is dynamic `malloc` storage, whose data lives until the programmer deallocates it. C requires knowing where objects are in memory, otherwise things do not work as expected; Java hides the location of objects.

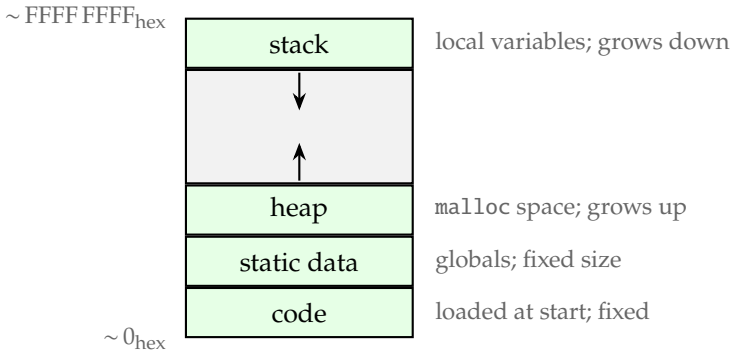


Figure 7: A program's address space. The stack grows down from the top and the heap grows up; for now, assume the operating system prevents accesses to the space between them.

A program's address space has four regions (Figure 7): the *stack* of local variables, which grows downward; the *heap*, the space requested through `malloc()`, which resizes dynamically and grows upward; *static data*, the variables declared outside `main`, which neither grows nor shrinks; and the *code*, loaded when the program starts and unchanged after. A variable declared outside a procedure (global) is allocated in static storage; one declared inside a procedure (local, and `main` is a procedure) is allocated on the stack and freed when the procedure returns.

A STACK FRAME holds a procedure's return "instruction" address, its parameters, and space for its other local variables. Stack frames are contiguous blocks of memory, and the STACK POINTER tells where the top frame is. When a procedure ends its frame is tossed off the stack, freeing the memory for future frames. The stack is thus last in, first out (LIFO): if `main` calls `a(0)`, which calls `b(1)`, which calls `c(2)`, which calls `d(3)`, then four frames are pushed below `main`'s, the stack pointer moving down with each call, and they are popped in the reverse order.

5.4 Memory management

The heap is a large pool of memory that is *not* allocated in contiguous order: back-to-back requests may return blocks very far apart. It is where Java's `new` allocates memory. In C the number of bytes to allocate is given explicitly, and `malloc()` allocates raw, uninitialized memory from the heap; it returns type `void *`, so the result is cast to the right type.

Code and static storage are easy to manage, since they never grow or shrink, and the stack is easy,

since frames are created and destroyed in LIFO order. The heap is tricky, because memory can be allocated and deallocated at any time. We want `malloc()` and `free()` to run quickly, with minimal memory overhead, and we want to avoid FRAGMENTATION (technically, external fragmentation): most free memory in many small chunks, so that a large request cannot be satisfied even though many free bytes exist, because they are not contiguous.

Example: Request R1 for 100 bytes, then R2 for 1 byte, then free R1, then request R3 for 50 bytes. R3 can go into the space R1 freed or after R2; either way, the free memory is now split into pieces around the 1-byte block.

The `malloc`/`free` implementation in Section 8.7 of Kernighan and Ritchie precedes each block of memory with a header holding the size of the block and a pointer to the next block. All free blocks are kept in a circular linked list; the pointer field is unused in an allocated block. `malloc()` searches the free list for a block that is big enough. If none is found, more memory is requested from the operating system, and if that cannot satisfy the request, it fails. `free()` checks whether the blocks adjacent to the freed block are also free. If so, the adjacent free blocks are merged (COALESCED) into a single larger free block; otherwise the freed block is just added to the free list. When several free blocks are big enough for a request, `malloc` can choose by

- BEST FIT: the smallest block that is big enough (the most "snug" free space);
- FIRST FIT: the first block found that is big enough;
- NEXT FIT: like first fit, but resuming the search where the last one finished.

5.5 When memory goes bad

Pointers are probably the single largest source of bugs in software. A DANGLING REFERENCE uses a pointer before `malloc`; a MEMORY LEAK frees memory late or loses the pointer to it. Typical errors:

- *Writing off the end of an array.* After `int *foo = (int *) malloc(sizeof(int) * 100)`, the loop `for (i = 0; i <= 100; ++i) foo[i] = 0;` writes one element too many. It corrupts other parts of the program, including internal C data, and may cause crashes later.

- *Returning a pointer into the stack.* C pointers allow access to deallocated memory:

```c
int *ptr() {
  int y;
  y = 3;
  return &y;
}
main() {
  int *stackAddr, content;
  stackAddr = ptr();
  content = *stackAddr;
  printf("%d", content);   /* 3 */
  content = *stackAddr;
  printf("%d", content);   /* 13451514 */
}
```

  When `ptr` returns, its frame (holding `y`) is popped. The first read still finds 3, but the call to `printf` then places its own frame over the same memory, so the second read returns garbage.

- *Use after free.* After `free(f)`, a call such as `bar(f->a)` may read corrupted data, since something else may have taken over that memory, and writes through `f` corrupt other data, so the program crashes later.

- *Forgetting that `realloc` can move data.* After `struct foo *g = f;` and `f = realloc(f, sizeof(struct foo) * 20)`, the copy `g` may point to invalid memory, so reads through it may be corrupted and writes may corrupt other memory.

- *Freeing the wrong thing.* Calling `free` on something never `malloc`'ed, including a moved pointer, as in `f++; free(f)`, may confuse `malloc` or `free`, corrupting their internal storage or erasing other data.

- *Double free.* Calling `free(f)` twice may cause a use after free (if something else called `malloc()` in between and got that memory) or corrupt `malloc`'s data (since the pointer no longer came from `malloc`).

- *Losing the initial pointer.* If a function sets the global `int *plk` with `plk = malloc(2 * sizeof(int));` and later executes `plk++`, this may be a memory leak, unless a copy of the original `malloc`'ed pointer is kept somewhere else.

Tools such as Valgrind are essential for debugging C code. Valgrind slows a program down by an order of magnitude but adds many checks that catch most (not all) memory errors: memory leaks, misuse of `free`, and writing past the end of arrays. Heap data is the biggest source of bugs in C code.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 5 (Fall 2020).

Lecture 6: Floating Point

6.1 Fixed point

N bits represent 2^N things and no more: unsigned integers 0 to $2^N - 1$ (for $N = 32$, up to 4,294,967,295) or two's complement integers -2^{N-1} to $2^{N-1} - 1$ (for $N = 32$, up to 2,147,483,647). We also want very large numbers (seconds per millennium, $31{,}556{,}926{,}000_{10} = 3.1556926_{10} \times 10^{10}$), very small numbers (the Bohr radius, 0.0000000000529177_{10} m $= 5.29177_{10} \times 10^{-11}$ m), and numbers with both integer and fractional parts (1.5). The solution for the last case also handles the first two.

A BINARY POINT, like a decimal point, marks the boundary between integer and fractional parts; bits to its right are worth $2^{-1}, 2^{-2}, \ldots$ In a 6-bit format `xx.yyyy`,

$$10.1010_2 = 1 \times 2^1 + 1 \times 2^{-1} + 1 \times 2^{-3} = 2.625_{10},$$

and with this *fixed* binary point the representable range is 0 to 3.9375 (almost 4). Addition is straightforward: 01.100 + 00.100 = 10.000 (1.5 + 0.5 = 2.0). Multiplication is a bit more complex: multiplying 01.100 by 00.100 (1.5×0.5) as integers gives 0000110000, and we must remember where the point is to read the answer $0.11_2 = 0.75$.

6.2 Floating point representation

What we really want is to let the binary point *float*, because that makes the most effective use of the limited bits and so gives the most accuracy. For example, $0.1640625 = \ldots 000000.001010100000\ldots_2$; in 5 bits we store 10101 and record that the binary point is two places to the left of the most significant stored bit. Any other choice would lose accuracy. In floating point each numeral carries an exponent field recording where its binary point is, and since the point may lie outside the stored bits, very large and very small numbers become representable.

In decimal scientific notation, as in $6.02_{\text{ten}} \times 10^{23}$, 6.02 is the MANTISSA, 10 the radix (base), and 23 the exponent. The NORMALIZED form has no leading 0s: exactly one nonzero digit left of the decimal point. So 1/1,000,000,000 is 1.0×10^{-9} normalized, while 0.1×10^{-8} and 10.0×10^{-10} are not normalized. Binary scientific notation, as in $1.01_{\text{two}} \times 2^{-1}$, works the same way around a binary point. Arithmetic that supports it is called FLOATING POINT, because the binary point is not fixed as it is for integers; in C such a variable is declared `float`.

A normalized binary number has the form $\pm 1.\text{xxx}\ldots\text{x}_{\text{two}} \times 2^{\text{yyy}\cdots\text{y}_{\text{two}}}$. A 32-bit word holds it in three fields: bit 31 is the sign S, bits 30–23 are an 8-bit exponent (the y's), and bits 22–0 are a 23-bit significand (the x's). This represents magnitudes from about 1.2×10^{-38} to 3.4×10^{38}. A result larger than 3.4×10^{38} in magnitude is an OVERFLOW (the exponent is too large for the 8-bit field); a

nonzero result smaller than 1.2×10^{-38} in magnitude is an UNDERFLOW (the negative exponent is too large for the field).

6.3 IEEE 754

Definition: IEEE 754 single precision (fp32) uses the 1/8/23-bit layout above. The sign bit is 1 for negative and 0 for positive. To pack in more bits, the leading 1 of a normalized number is implicit, so the significand has $1 + 23$ bits of precision in single and $1 + 52$ in double; for normalized numbers the stored significand satisfies $0 \leq \text{significand} < 1$. The exponent is stored in BIASED NOTATION: the bias is subtracted from the field to get the real exponent, 127 for single precision. The value is

$$(-1)^S \times (1 + \text{significand}) \times 2^{(\text{exponent}-127)}.$$

Double precision is identical except for an exponent bias of 1023 (half and quad precision are similar).

Why a biased exponent? The designers wanted floating-point numbers to be usable even without floating-point hardware, for example to sort records by integer comparisons, with a bigger integer exponent field meaning a bigger number. Two's complement would break this, because negative exponents would look bigger. With a bias, the bit patterns are ordered exactly as in sign and magnitude: counting the binary odometer from $00\ldots00$ up to $11\ldots11$ runs from 0 up to +MAX, then from -0 down to $-$MAX. The standard (IEEE 754, Binary Floating-Point Arithmetic) is due largely to William Kahan, who received the 1989 ACM Turing Award.

6.4 Special numbers

Since 0 has no leading 1, exponent value 0 is reserved for it. Zero has all-zero exponent and significand, and either sign is valid: $+0$ is `0 00000000 000...0` and -0 is `1 00000000 000...0`.

In floating point, dividing by 0 should produce $\pm\infty$, not overflow: computing further with ∞ is fine, and a comparison such as $X/0 > Y$ may be valid. IEEE 754 reserves the most positive exponent (255) with an all-zero significand for $\pm\infty$. Kahan's motto "waste not, want not" put the remaining patterns to use. Results such as $\sqrt{-4.0}$ or $0/0$ are NOT A NUMBER (NaN), encoded with exponent 255 and a nonzero significand. NaNs help with debugging because they contaminate: op(NaN, X) = NaN, and the significand can identify which error produced them.

Normalization and the implicit 1 leave a gap around 0. The smallest positive normalized number is $a = 1.0\ldots0_2 \times 2^{-126} = 2^{-126}$ and the next is $b = 1.0\ldots01_2 \times 2^{-126} = (1 + 2^{-23}) \times 2^{-126} = 2^{-126} + 2^{-149}$, so $a - 0 = 2^{-126}$ while $b - a = 2^{-149}$. The fix uses the remaining patterns, exponent 0 with a nonzero significand, for DENORMALIZED numbers (denorms): no implied leading 1, and an implicit exponent of -126. The smallest positive denorm is then $a = 2^{-149}$ and the next is $b = 2^{-148}$, so the representable numbers are evenly spaced down to 0.

Exponent	Significand	Object
0	0	±0
0	nonzero	denorm
1–254	anything	± floating-point number
255	0	±∞
255	nonzero	NaN

6.5 Examples

Example: The pattern 1 `10000001 111 0000 0000 0000 0000 0000` has $S = 1$, exponent 129, and significand $.111_2$, so its value is $(-1)^1 \times (1.111)_2 \times 2^{129-127} = -111.1_2 = -7.5_{10}$.

Example: $\frac{1}{3} = 0.333\ldots_{10} = \frac{1}{4} + \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \cdots = 0.010101\ldots_2 \times 2^0 = 1.010101\ldots_2 \times 2^{-2}$. The sign is 0, the exponent field is $-2 + 127 = 125 = 01111101_2$, and the significand is 0101 . . .; truncated to 23 bits this gives `0 01111101 01010101010101010101010`. (Rounding to nearest, the default mode below, rounds the last bit up, since the discarded bits are 1010 . . ., giving `0x3EAAAAAB`.)

Two methods help read a significand. With *fractions*, $0.340_{10} = 340/1000 = 34/100$ and likewise $0.110_2 = 110_2/1000_2 = 6/8 = 11_2/100_2 = 3/4$; this is less numerical and helps understand what the significand means. With *place values*, $1.6732 = 1 \times 10^0 + 6 \times 10^{-1} + 7 \times 10^{-2} + 3 \times 10^{-3} + 2 \times 10^{-4}$ and $1.1001_2 = 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$; this is quick for calculating a significand's value and is the method to use when translating floating-point numbers.

6.6 Floating point arithmetic

Floating-point addition is *not* associative. With $x = -1.5 \times 10^{38}$, $y = 1.5 \times 10^{38}$, and $z = 1.0$,

$$x + (y + z) = -1.5 \times 10^{38} + (1.5 \times 10^{38} + 1.0) = -1.5 \times 10^{38} + 1.5 \times 10^{38} = 0.0,$$

but $(x + y) + z = 0.0 + 1.0 = 1.0$. The floating-point result only approximates the real result: 1.5×10^{38} is so much larger than 1.0 that $1.5 \times 10^{38} + 1.0$ is still 1.5×10^{38} in floating point.

Precision is the number of bits used to represent a value; accuracy is the difference between a number's actual value and its computer representation. High precision permits high accuracy but does not guarantee it. After `float pi = 3.14`, `pi` uses all 24 bits of the significand (highly precise) yet is only an approximation of π (not accurate). Likewise 0.3 is stored in a `float` as 0.30000001192092895507812 5, and in double precision 0.1 + 0.2 evaluates to 0.30000000000000004.

Math on real numbers must be rounded to fit the result in the significand. The floating-point hardware carries two extra bits of precision and then rounds to the proper value. Rounding also occurs when converting a double to single precision or a floating-point number to an integer. IEEE 754 has four rounding modes (decimal examples):

- round toward $+\infty$: always round up, $2.001 \to 3$, $-2.001 \to -2$;
- round toward $-\infty$: always round down, $1.999 \to 1$, $-1.999 \to -2$;
- truncate: drop the last bits (round toward 0);
- unbiased, the default: round to nearest, but a value exactly midway rounds to the nearest *even* number, so $2.4 \to 2$, $2.6 \to 3$, $2.5 \to 2$, $3.5 \to 4$. Rounding up on half the ties and down on the other half tends to balance out inaccuracies.

Floating-point addition is harder than integer addition, since we cannot just add significands: de-normalize to match the exponents, add the significands, keep the common exponent, then normalize (possibly changing the exponent). If the signs differ, perform a subtraction instead.

In C, `(int)` *floating_point_expression* converts to an integer by truncation, as in `i = (int) (3.14159 * f)`, and `(float)` *integer_expression* converts to the nearest floating-point value, as in `f = f + (float) i`. Neither round trip is the identity:

- `if (i == (int)((float) i)) printf("true");` does not always print `true`, because most large integers have no exact `float` representation. (With `double`, whose significand has 53 bits, every 32-bit `int` is exact and the test always succeeds.)
- `if (f == (float)((int) f)) printf("true");` does not always print `true`: small floating-point numbers (< 1) have no integer representation, and other numbers suffer rounding errors.

6.7 Other representations

Double precision (binary64, C's `double`) uses the next multiple of the word size, 64 bits: 1 sign bit, 11 exponent bits, and a 52-bit significand (20 bits in the first word and 32 in the second). It represents numbers from almost 2.0×10^{-308} to almost 2.0×10^{308}, but its primary advantage is greater accuracy from the larger significand.

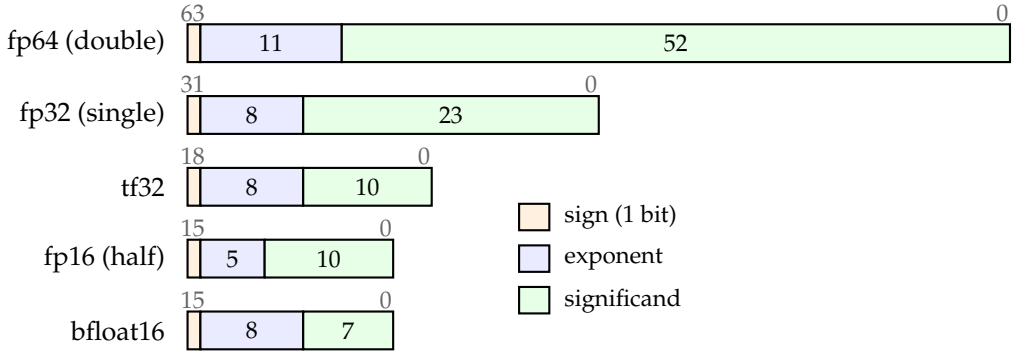


Figure 8: Floating-point formats drawn to scale, with the width of each field in bits and the numbers of the highest and lowest bits.

Quad precision (binary128) has 15 exponent and 112 significand bits, with unbelievable range and precision; octuple precision (binary256) has 19 and 236. Half precision (binary16, fp16) has a 1/5/10 layout. The competing `bfloat16` keeps fp32's 8-bit exponent, and hence fp32's range, with only 7 significand bits; it is used for faster machine learning, as is TF32, with an 8-bit exponent and 10-bit significand (Figure 8).

Domain accelerators choose different formats:

Accelerator	int4	int8	int16	fp16	bf16	fp32	tf32
Google TPU v1		×					
Google TPU v2, v3					×		
Nvidia Volta TensorCore	×	×		×			
Nvidia Ampere TensorCore	×	×	×	×	×	×	×
Nvidia DLA		×	×	×			
Intel AMX		×			×		
Amazon AWS Inferentia		×		×	×		
Qualcomm Hexagon		×					
Huawei Da Vinci		×		×			
MediaTek APU 3.0		×	×	×			
Samsung NPU, Tesla NPU		×					

All these formats fix the number of exponent and significand bits. The UNUM format (John Gustafson) makes them variable and adds a "u-bit" telling whether a number is exact or a range; it "promises to be to floating point what floating point is to fixed point" and claims to save power.

Floating point thus represents numbers with both integer and fractional parts, using the available bits efficiently, and stores approximate values for very large and very small numbers. IEEE 754 is the most widely accepted standard for interpreting them; every computer since about 1997 follows it. In the fp32 formula, the exponent tells the significand how much (2^i) to count by: $\ldots, \frac{1}{4}, \frac{1}{2}, 1, 2, \ldots$

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 6 (Fall 2020).

Lecture 7: RISC-V Intro

7.1 Instruction set architectures

The basic job of a CPU is to execute lots of instructions, the primitive operations it can perform. An instruction is like a sentence: operations (verbs) applied to operands (objects), processed in sequence. Different CPUs implement different sets of instructions, and the set a particular CPU implements is its instruction set architecture (ISA). Examples are ARM (cell phones), Intel x86 (i9, i7, i5, i3), IBM Power, IBM/Motorola PowerPC (old Macs), MIPS, and RISC-V.

The early trend was to add more and more instructions to new CPUs to do elaborate operations; the VAX had an instruction to multiply polynomials. The RISC philosophy (Reduced Instruction Set Computing; Cocke at IBM, Patterson, and Hennessy, 1980s) went against this conventional wisdom: keep the instruction set small and simple, which makes it easier to build fast hardware, and let software do complicated operations by composing simpler ones.

RISC-V is a new open-source, license-free ISA specification, supported by a growing shared software ecosystem and appropriate for all levels of computing, from microcontrollers to supercomputers. It has 32-, 64-, and 128-bit variants; these notes use the 32-bit variant RV32 (the Patterson and Hennessy textbook uses the 64-bit RV64). RISC-V is simple and elegant, avoiding the gritty details of the Intel 80x86, and its adoption is growing exponentially. It started in summer 2010 to support open research and teaching at UC Berkeley, its lineage traces to the RISC-I and RISC-II projects of the 1980s, and it has since moved to the RISC-V Foundation; many commercial and research projects, both open-source and proprietary, are based on it. The instruction set of an architecture is represented by its assembly language, in which each line of code represents one instruction.

7.2 Registers

Unlike high-level languages such as C or Java, assembly cannot use variables, to keep the hardware simple. Its operands are registers: a limited number of special locations built directly into the hardware, and operations can only be performed on them. Because registers are directly in hardware they are very fast, faster than 0.25 ns (light travels 3×10^8 m/s = 30 cm/ns, or 10 cm in 0.3 ns, the clock period of a 3.33 GHz computer). Registers are inside the processor, which is connected to memory and I/O (Figure 9); they form the top of the memory hierarchy (extremely fast, extremely expensive, tiny capacity), the "my head, 1 ns" level of Jim Gray's analogy.

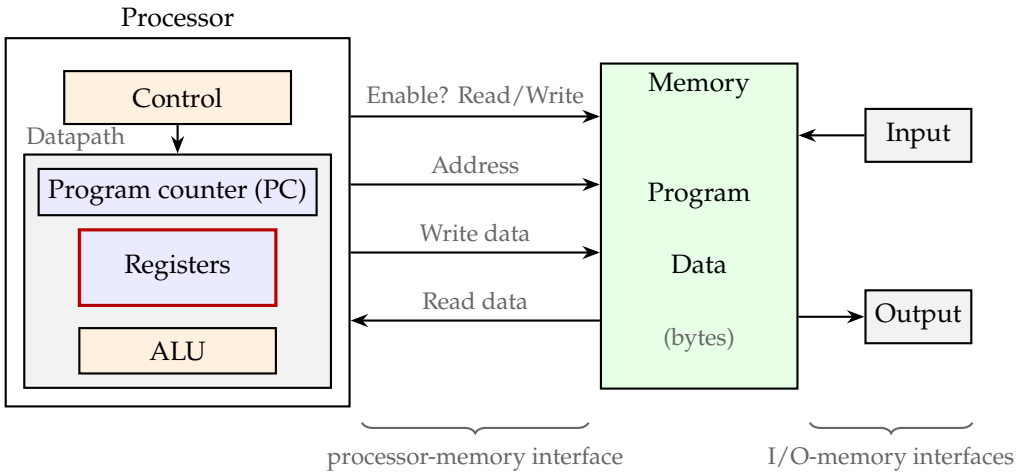


Figure 9: A computer: the processor's control and datapath (program counter, registers, ALU) exchange addresses and data with memory, which holds both program and data; input and output devices connect to memory. The registers are inside the processor.

The drawback of registers being in hardware is that there is a predetermined number of them, so RISC-V code must be put together very carefully to use them efficiently. RISC-V has 32 registers: smaller is faster, but too few is bad (the Goldilocks principle). In RV32 each register is 32 bits wide, and a group of 32 bits is called a WORD. Registers are numbered 0 to 31 and referred to as `x0`–`x31`; each can also be referred to by a name. The register `x0` is special: it always holds the value zero, so only 31 registers can hold variable values.

In C (and most high-level languages) variables are declared first and given a type, as in `int fahr, celsius; char a, b, c, d, e;`, and each variable can only represent a value of its declared type. In assembly language registers have no type: the *operation* determines how register contents are treated. A hash mark `#` starts a RISC-V comment that runs to the end of the line, like C99's `//` (and unlike C's `/* ... */`, which can span many lines).

7.3 Addition and subtraction

In assembly language each statement, an instruction, executes exactly one of a short list of simple commands, and each line contains at most one instruction. Instructions correspond to operations such as =, +, −, ∗, / in C or Java.

The syntax is rigid, one operator and three operands, to keep the hardware simple via regularity. In `add x1, x2, x3`, `add` names the operation, `x1` is the destination that gets the result, and `x2` and `x3` are the first and second sources (source1 and source2). With C variables `a`, `b`, `c` in `x1`, `x2`, `x3`, the instruction `add x1,x2,x3` computes `a = b + c`; with `d`, `e`, `f` in `x3`, `x4`, `x5`, `sub x3,x4,x5` computes `d = e - f`. A single line of C may become several instructions. For `a = b + c + d - e;` with `a`–`e` in `x10`, `x1`, `x2`, `x3`, `x4`:

```
add x10, x1, x2    # a_temp = b + c
add x10, x10, x3   # a_temp = a_temp + d
sub x10, x10, x4   # a = a_temp - e
```

For `f = (g + h) - (i + j);` with `f`, `g`, `h`, `i`, `j` in `x19`–`x23`, use intermediate temporary registers:

```
add x5, x20, x21   # a_temp = g + h
add x6, x22, x23   # b_temp = i + j
sub x19, x5, x6    # f = (g + h) - (i + j)
```

7.4 Immediates

Immediates are numerical constants. They appear often in code, so there are special instructions for them. Add immediate, `addi x3,x4,10`, computes `f = g + 10` with `f`, `g` in `x3`, `x4`; its syntax is like `add` except that the last operand is a number instead of a register. There is no subtract immediate: to limit the operations to the absolute minimum, an operation that can be decomposed into a simpler one is left out, and `addi x3,x4,-10` computes `f = g - 10`.

One immediate, zero, appears so often that register `x0` is hard-wired to 0. So `add x3,x4,x0` computes `f = g`, and since `x0` is defined in hardware, `add x0,x3,x4` does nothing.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 7 (Fall 2020).

Lecture 8: RISC-V lw, sw, Decisions I

8.1 Storing data in memory

So far RV32 has `add rd, rs1, rs2` ($R[rd] = R[rs1] + R[rs2]$), `sub rd, rs1, rs2` ($R[rd] = R[rs1] - R[rs2]$), and `addi rd, rs1, imm` ($R[rd] = R[rs1] + imm$). Registers are very fast but have limited space; memory is a much larger place to hold values, but slower. Moving a value from memory into a register is a LOAD; moving one from a register to memory is a STORE (Figure 9).

Data are typically smaller than 32 bits but rarely smaller than 8 (the `char` type), and everything works out if sizes are multiples of 8 bits. An 8-bit chunk is a BYTE, and 1 word = 4 bytes. Memory addresses are really in bytes, not words, so word addresses are 4 bytes apart. The address of a word is the address of its rightmost, least-significant byte: the least-significant byte gets the smallest address. This is the LITTLE-ENDIAN convention (Figure 10).

bits	31–24	23–16	15–8	7–0
word at 12	15	14	13	12
word at 8	11	10	9	8
word at 4	7	6	5	4
word at 0	3	2	1	0

Figure 10: Byte addresses in little-endian memory. Each row is a 32-bit word, and its address is that of its least-significant byte (outlined).

Endianness is the order in which *bytes* are stored in memory; the bits within a byte are always stored as usual (`0xC2` = `0b1100 0010`). The number 1025 consists of the bytes `00000000 00000000 00000100 00000001` (byte 3 down to byte 0). A BIG-ENDIAN machine stores byte 3 at the lowest address; a little-endian machine stores byte 0 there:

	address +0	address +1	address +2	address +3
Big endian	`00000000`	`00000000`	`00000100`	`00000001`
Little endian	`00000001`	`00000100`	`00000000`	`00000000`

8.2 Data transfer instructions

Registers hold 32 words (128 bytes), while memory (DRAM) holds billions of bytes (2 to 64 GB on a laptop). Physics dictates that smaller is faster: registers are about 50–500 times faster than DRAM in the latency of one access (tens of ns), although subsequent words then come every few ns.

LOAD WORD (`lw`) copies a word from memory into a register. For `int A[100]; g = h + A[3];` with the address of `A[0]` in `x15`, `h` in `x12`, and `g` in `x11`:

```
lw  x10, 12(x15)     # Reg x10 gets A[3]
add x11, x12, x10    # g = h + A[3]
```

Here `x15` is the BASE REGISTER (a pointer to `A[0]`) and 12 is the OFFSET in bytes (3 words of 4 bytes). The offset must be a constant known at assembly time. STORE WORD (`sw`) copies a register to memory. For `A[10] = h + A[3]`:

```
lw  x10, 12(x15)     # Temp reg x10 gets A[3]
add x10, x12, x10    # Temp reg x10 gets h + A[3]
sw  x10, 40(x15)     # A[10] = h + A[3]
```

The addresses `x15` + 12 and `x15` + 40 must be multiples of 4 (word aligned).

Besides word transfers, RISC-V has byte transfers, LOAD BYTE (`lb`) and STORE BYTE (`sb`), with the same format. In `lb x10,3(x11)` the byte at address 3 + `x11` is copied into the low byte of `x10`, and its top bit is copied into all the higher bits of `x10` (SIGN EXTENSION). `lbu` (load byte unsigned) fills the upper bits with 0s instead.

Example: After `addi x11,x0,0x3F5`, `sw x11,0(x5)`, and `lb x12,1(x5)`, register `x12` holds `0x00000003`: the word `0x000003F5` is stored little endian, so address `x5` + 1 holds its second-lowest byte `0x03`, whose sign bit is 0.

A `lw` from a memory location holding a constant followed by an `add` (`lw x10,12(x15); add x12,x12,x10`) could replace `addi`, but it involves a load from memory. Adding an immediate is so common that it deserves its own instruction. So far, then, RV32 has `add`, `sub`, `addi`, and the loads and stores `lw`, `lb`, `lbu`, `sw`, `sb`.

8.3 Decisions

Computers must be able to do something different based on a computation: in programming languages, an `if` statement. The RISC-V counterpart is

```
beq reg1, reg2, L1,
```

which means go to the statement labeled `L1` if the value in `reg1` equals the value in `reg2`, and otherwise go to the next statement (BRANCH IF EQUAL). `bne` IS BRANCH IF NOT EQUAL.

A BRANCH is a change of control flow. A CONDITIONAL BRANCH changes control flow depending on the outcome of a comparison: `beq`, `bne`, branch if less than (`blt`), branch if greater than or equal (`bge`), and their unsigned versions `bltu` and `bgeu`. In `blt reg1, reg2, Label` the branch is taken if `reg1` < `reg2` as signed integers; `bltu` treats the registers as unsigned. There are no `bgt` or `ble` instructions. An UNCONDITIONAL BRANCH always branches: the jump `j label`.

With `f`, `g`, `h`, `i`, `j` in `x10`–`x14`, the C statements on the left compile to the code on the right. The branch condition is often negated, branching *around* the code that should run when the C condition holds:

C	RISC-V

`if (i == j)`
`  f = g + h;`
	`bne x13,x14,Exit`
	`add x10,x11,x12`
	`Exit:`

`if (i == j)`
`  f = g + h;`
`else`
`  f = g - h;`
	`bne x13,x14,Else`
	`add x10,x11,x12`
	`j Exit`
	`Else:sub x10,x11,x12`
	`Exit:`

C has three kinds of loops, `while`, `do...while`, and `for`; each can be rewritten as either of the others, so the same branching method applies to all. There are many ways to write a loop in RISC-V, but the key to decision making is the conditional branch. The loop

```
int A[20]; int sum = 0; for (int i=0; i < 20; i++) sum += A[i];
```

with the address of `A[0]` in `x8` becomes

```
1       add  x9, x8, x0      # x9 = &A[0]
2       add  x10, x0, x0     # sum = 0
3       add  x11, x0, x0     # i = 0
4       addi x13, x0, 20     # x13 = 20
5  Loop: bge  x11, x13, Done
6       lw   x12, 0(x9)      # x12 = A[i]
7       add  x10, x10, x12   # sum +=
8       addi x9, x9, 4       # &A[i+1]
9       addi x11, x11, 1     # i++
10      j    Loop
11 Done:
```

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 8 (Fall 2020).

Lecture 9: RISC-V Decisions II

9.1 Logical instructions

Logical operations act on fields of bits within a word, such as the 8-bit characters in a word, and pack bits into words or unpack them:

Logical operation	C	Java	RISC-V
Bit-by-bit AND	`&`	`&`	`and`
Bit-by-bit OR	`|`	`|`	`or`
Bit-by-bit XOR	`^`	`^`	`xor`
Shift left logical	<<	<<	`sll`
Shift right logical	>>	>>	`srl`

Each comes in two variants: register (`and x5, x6, x7` sets `x5 = x6 & x7`) and immediate (`andi x5, x6, 3` sets `x5 = x6 & 3`). The immediate forms are used for masks: `andi` with `0000 00FF`$_{\text{hex}}$ isolates the least-significant byte, and with `FF00 0000`$_{\text{hex}}$ the most-significant byte. For simplicity there is no logical NOT; use `xor` with all ones (11111111_{two}).

Shift left logical (`sll`, and `slli` with an immediate) shifts bits left, letting them fall off the end and inserting 0s on the right, like C's <<: `slli x11,x12,2` stores in `x11` the value of `x12` shifted left by 2 bits, turning `0000 0002`$_{\text{hex}}$ into `0000 0008`$_{\text{hex}}$. Shifting left by n multiplies by 2^n. Shift right logical (`srl`) is the opposite shift, like C's >>. Shift right arithmetic (`sra`, `srai`) instead inserts copies of the high-order sign bit into the vacated bits. If `x10` holds `1111 1111 1111 1111 1111 1111 1110 0111`$_{\text{two}}$ = -25_{ten}, then `srai x10, x10, 4` gives `1111 1111 1111 1111 1111 1111 1111 1110`$_{\text{two}}$ = -2_{ten}. This is *not* the same as dividing by 2^n: it fails for odd negative numbers, because C's integer division rounds toward 0 ($-25/16$ is -1 in C).

9.2 Machine programs

An assembler converts human-readable assembly code (`foo.S`, `bar.S`) into instruction bit patterns in object files (`foo.o`, `bar.o`), which the linker combines with pre-built object file libraries (`lib.o`) into an executable (`a.out`). A program is stored in memory alongside its data, and one RISC-V instruction occupies 32 bits.

The program counter (PC) is a register internal to the processor that holds the byte address of the next instruction to be executed. Each instruction is fetched from memory; the control unit then executes it using the datapath and memory system, and updates the PC. By default it adds 4 bytes to the PC to move to the next sequential instruction; branches and jumps alter this.

The assembler offers two conveniences. Symbolic register names replace numbers, such as `a0`–`a7` for the argument registers `x10`–`x17` used in function calls and `zero` for `x0`. Pseudo-instructions are shorthand for common idioms: `mv rd, rs` is `addi rd, rs, 0`, `li rd, 13` is `addi rd, x0, 13`, and `nop` is `addi x0, x0, 0`.

9.3 Function calls

Calling a function such as

```
1 int mult(int mcand, int mlier) {   /* really dumb mult function */
2   int product = 0;
3   while (mlier > 0) {
4     product = product + mcand;
5     mlier = mlier - 1;
6   }
7   return product;
8 }
```

from `i = mult(j,k); m = mult(i,i);` takes six fundamental steps:

1. put the arguments in a place where the function can access them;
2. transfer control to the function;
3. acquire the (local) storage resources the function needs;
4. perform the function's task;
5. put the return value in a place where the calling code can access it, restore any registers used, and release local storage;
6. return control to the point of origin, since a function can be called from several points in a program.

Registers are faster than memory, so the RISC-V calling convention uses them: `a0`–`a7` (`x10`–`x17`) are eight argument registers for passing parameters, of which `a0` and `a1` also return values; `ra` (`x1`) is the return address register, used to return to the point of origin; and `s0`–`s1` (`x8`–`x9`) and `s2`–`s11` (`x18`–`x27`) are saved registers.

Instructions are stored in memory just like data, 4 bytes each, so each has an address. Calling `int sum(int x, int y) { return x+y; }` as `sum(a,b)` with `a`, `b` in `s0`, `s1` could be written as follows, with each instruction's address (in decimal) in its comment:

```
     mv   a0, s0          # 1000: x = a
     mv   a1, s1          # 1004: y = b
     addi ra, zero, 1016  # 1008: ra = 1016
     j    sum             # 1012: jump to sum
                          # 1016: next instruction
     ...
sum: add  a0, a0, a1      # 2000
     jr   ra              # 2004: "jump register"
```

The function returns with `jr` (JUMP REGISTER) rather than `j` because `sum` might be called from many places, so it cannot return to a fixed place; the caller must be able to say "return here". Since function calls are very common, a single instruction both jumps and saves the return address: JUMP AND LINK. The two instructions at 1008 and 1012 become `1008 jal sum`, which sets `ra` to 1012 and jumps to `sum`. This makes the common case fast, reduces program size, and means the code need not know where it is in memory.

`jal FunctionLabel` (really "link and jump") forms an address, or link, pointing to the calling site, so that the function can return there: it jumps to the label and simultaneously saves the address of the following instruction in `ra`. `jr ra` is an unconditional jump to the address in a register; the assembler shorthand is `ret`. In fact there are only two instructions: `jal rd, Label` (jump and link) and `jalr rd, rs, imm` (jump and link register). The forms `j`, `jr`, and `ret` are pseudo-instructions; for example, `j Label` is `jal x0, Label`.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 9 (Fall 2020).

Lecture 10: RISC-V Procedures

10.1 A function call example

Recall the six steps of a call: put the arguments in registers, transfer control with `jal`, acquire local storage, do the task, put the return value where the caller can find it (restoring registers and releasing local storage), and return with `ret`. Consider

```c
int Leaf(int g, int h, int i, int j) {
  int f;
  f = (g + h) - (i + j);
  return f;
}
```

with the parameters `g`, `h`, `i`, `j` in `a0`–`a3`, `f` in `s0`, and one temporary register `s1`.

The old values of `s0` and `s1` must be saved before the function uses them and restored when it returns. The ideal place is a STACK, a last-in, first-out (LIFO) queue like a stack of plates: a PUSH places data on the stack and a POP removes it. The stack lives in memory, so a register must point to it: `sp` (`x2`) is the STACK POINTER. By convention the stack grows down from high to low addresses, so a push decrements `sp` and a pop increments it. A STACK FRAME holds a function's return "instruction" address, its parameters (arguments), and space for other local variables. Frames are contiguous blocks of memory, `sp` tells where the bottom of the current frame is, and when a procedure ends its frame is tossed off the stack, freeing the memory for future frames.

```asm
Leaf: addi sp, sp, -8   # adjust stack for 2 items
      sw   s1, 4(sp)    # save s1 for use afterwards
      sw   s0, 0(sp)    # save s0 for use afterwards
      add  s0, a0, a1   # f = g + h
      add  s1, a2, a3   # s1 = i + j
      sub  a0, s0, s1   # return value (g + h) - (i + j)
      lw   s0, 0(sp)    # restore register s0 for caller
      lw   s1, 4(sp)    # restore register s1 for caller
      addi sp, sp, 8    # adjust stack to delete 2 items
      jr   ra           # jump back to calling routine
```

During the call the saved `s1` and `s0` sit just below the caller's part of the stack, with `sp` pointing at the saved `s0`; after the call `sp` is back where it started.

10.2 Nested calls and register conventions

A function that calls another function (including a recursive one) would clobber the values in `a0`–`a7` and `ra`. In

```
int sumSquare(int x, int y) {
  return mult(x,x) + y;
}
```

`ra` holds the address `sumSquare` must jump back to, but the call to `mult` overwrites it. So `sumSquare` must save its return address before calling `mult`, again on the stack.

The CALLER is the calling function and the CALLEE is the function being called. When the callee returns, the caller needs to know which registers may have changed and which are guaranteed unchanged. REGISTER CONVENTIONS are generally accepted rules saying which registers are unchanged after a procedure call (`jal`) and which may change. To reduce expensive loads and stores for spilling and restoring registers, the RISC-V calling convention divides registers into two categories:

1. *Preserved across a call*: the caller can rely on their values being unchanged. These are `sp`, `gp`, `tp`, and the saved registers `s0`–`s11` (`s0` is also the frame pointer `fp`).
2. *Not preserved across a call*: the caller cannot rely on them being unchanged. These are the argument/return registers `a0`–`a7`, `ra`, and the temporaries `t0`–`t6`.

The numbers `x0`–`x31` are what the hardware understands; the symbolic ABI names are the human-friendly names used in assembly code:

Register	ABI name	Description	Saver
`x0`	`zero`	Hard-wired zero	-
`x1`	`ra`	Return address	Caller
`x2`	`sp`	Stack pointer	Callee
`x3`	`gp`	Global pointer	-
`x4`	`tp`	Thread pointer	-
`x5`	`t0`	Temporary/alternate link register	Caller
`x6–x7`	`t1–t2`	Temporaries	Caller
`x8`	`s0/fp`	Saved register/frame pointer	Callee
`x9`	`s1`	Saved register	Callee
`x10–x11`	`a0–a1`	Function arguments/return values	Caller
`x12–x17`	`a2–a7`	Function arguments	Caller
`x18–x27`	`s2–s11`	Saved registers	Callee
`x28–x31`	`t3–t6`	Temporaries	Caller

10.3 Memory allocation

C has two storage classes. AUTOMATIC variables are local to a function and discarded when it exits; STATIC variables exist across exits from and entries to procedures. The stack holds automatic (local) variables that do not fit in registers. The segment of the stack holding a procedure's saved registers and local variables is its PROCEDURE FRAME or ACTIVATION RECORD (Figure 11).

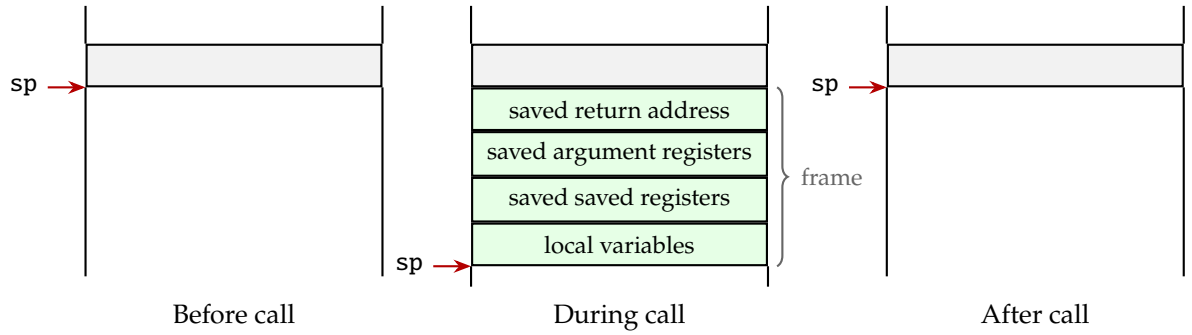


Figure 11: The stack before, during, and after a call. During the call the callee's frame holds, as needed, the saved return address, saved argument registers, saved saved registers, and local variables; after the call `sp` is restored.

Since `sp` always points to the last used space in the stack, a function uses the stack by decrementing `sp` by the amount of space it needs and then filling that space. `sumSquare` compiles to

```
1  sumSquare:
2        addi sp, sp, -8   # space on stack ("push")
3        sw   ra, 4(sp)    # save ret addr
4        sw   a1, 0(sp)    # save y
5        mv   a1, a0       # mult(x,x)
6        jal  mult         # call mult
7        lw   a1, 0(sp)    # restore y
8        add  a0, a0, a1   # mult()+y
9        lw   ra, 4(sp)    # get ret addr
10       addi sp, sp, 8    # restore stack ("pop")
11       jr   ra
12 mult: ...
```

When a C program runs, three important memory areas are allocated: *static* (variables declared once per program, which cease to exist only when execution completes, such as C globals), the *heap* (variables declared dynamically through `malloc`), and the *stack* (space used by a procedure during execution, where register values can be saved). The RV32 convention (RV64 and RV128 have different layouts) is shown in Figure 12. The stack starts in high memory, at $\texttt{bfff_fff0}_{\text{hex}}$, and grows down; it must be aligned on a 16-byte boundary (the examples above are not). Programs (the TEXT SEGMENT) are at the low end, starting at $\texttt{0001_0000}_{\text{hex}}$. The STATIC DATA SEGMENT (constants and other static variables) lies above the text, and the GLOBAL POINTER `gp` points to it (`gp` = $\texttt{1000_0000}_{\text{hex}}$

in RV32). The heap, above static data, holds data structures that grow and shrink, and grows up toward high addresses.

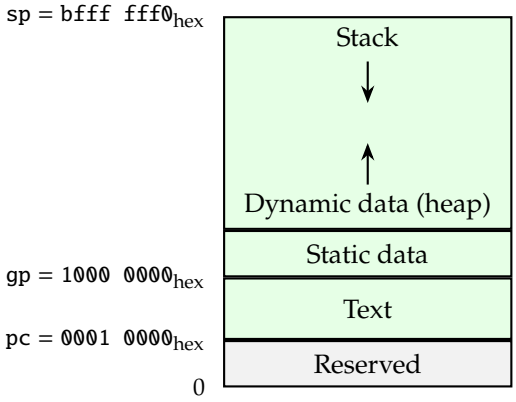


Figure 12: RV32 memory allocation. The stack grows down from `sp` and the heap (dynamic data) grows up above the static data.

The RV32 instructions so far are: arithmetic and logic (`add`, `sub`, `and`, `or`, `xor`, `sll`, `srl`, `sra`); immediates (`addi`, `andi`, `ori`, `xori`, `slli`, `srli`, `srai`); loads and stores (`lw`, `lb`, `lbu`, `sw`, `sb`); and branches and jumps (`beq`, `bne`, `bge`, `blt`, `bgeu`, `bltu`, `jal`, `jalr`).

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 10 (Fall 2020).

Lecture 11: RISC-V Instruction Formats I

11.1 The stored-program computer

ENIAC (University of Pennsylvania, 1946), the first electronic general-purpose computer, multiplied in 2.8 ms but took two to three days to set up a new program, because it was programmed with patch cords and switches. The big idea that followed is the STORED-PROGRAM COMPUTER: instructions are represented as bit patterns, which can be thought of as numbers, so entire programs can be stored in memory to be read or written just like data. A computer can then be reprogrammed in seconds instead of rewired in days. Such machines are known as "von Neumann" computers after the widely distributed 1945 *First Draft of a Report on the EDVAC*, which wrote up discussions of Eckert and Mauchly; the idea was anticipated earlier by Turing and Zuse. EDSAC (Cambridge, 1949) was the first general stored-program electronic computer, holding programs as numbers in memory in 35-bit two's complement words.

Everything has a memory address. Since all instructions and data are stored in memory, everything has an address: instructions and data words alike. Branches and jumps use these addresses, and C pointers are just memory addresses that can point to anything in memory. Unconstrained use of addresses can lead to nasty bugs; avoiding them is up to the programmer in C and limited by language design in Java. One register keeps the address of the instruction being executed, the PROGRAM COUNTER (PC), basically a pointer to memory (Intel calls it the instruction pointer, IP).

Binary compatibility. Programs are distributed in binary form and are therefore bound to a specific instruction set, with different versions for phones and PCs. New machines want to run old programs ("binaries") as well as programs compiled to new instructions, which leads to backward-compatible instruction sets that evolve over time. The choice of the Intel 8088 for the first IBM PC in 1981 is a major reason PCs still use the 80x86 instruction set; a program from a 1981 PC could still run today.

11.2 Instructions as numbers

Most data are words (32-bit chunks): each register is a word, and `lw` and `sw` access memory a word at a time. The computer understands only 1s and 0s, so the assembler string `add x10,x11,x0` means nothing to the hardware. Since data come in words, RISC-V seeks simplicity by making instructions fixed-size 32-bit words as well; the same 32-bit instructions are used in RV32, RV64, and RV128. Each instruction word is divided into FIELDS, each telling the processor something about the instruction. Different fields could be defined for every instruction, but for simplicity RISC-V

defines six basic instruction formats: *R-format* for register-register arithmetic operations, *I-format* for register-immediate arithmetic operations and loads, *S-format* for stores, *B-format* for branches (a minor variant of S-format), *U-format* for 20-bit upper immediate instructions, and *J-format* for jumps (a minor variant of U-format).

11.3 R-format

The R-format divides the 32-bit word into six fields of 7 + 5 + 5 + 3 + 5 + 7 = 32 bits (Figure 13); for example, `opcode` is a 7-bit field in bits 6–0 and `rs2` a 5-bit field in bits 24–20. The `opcode` partially specifies the instruction: it is 0110011_{two} for all R-format register-register arithmetic instructions. Combined with the opcode, `funct7` and `funct3` describe which operation to perform. `rs1` (source register 1) names the register holding the first operand, `rs2` the second, and `rd` (destination register) the register that receives the result. Each register field holds a 5-bit unsigned integer (0–31) naming one of `x0`–`x31`.

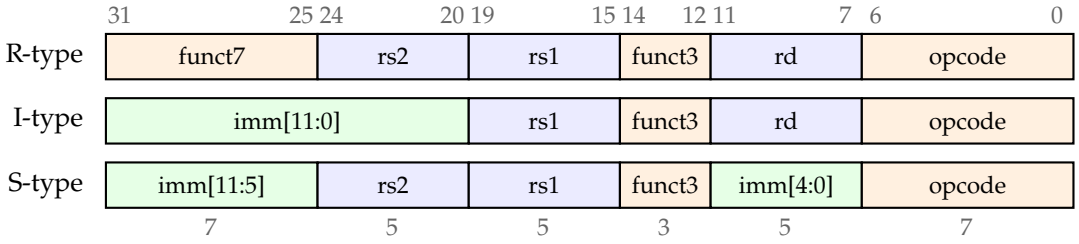


Figure 13: R-, I-, and S-format layouts. Bit positions are shown above and field widths in bits below. The register fields and `funct3` stay in the same place in every format.

Example: `add x18,x19,x10` has `funct7` = 0000000, `rs2` = 10 = 01010, `rs1` = 19 = 10011, `funct3` = 000 (add), `rd` = 18 = 10010, and `opcode` = 0110011 (register-register operation). Likewise `add x4, x3, x2` encodes as `0000000 00010 00011 000 00100 0110011` = $0021\ 8233_{\text{hex}}$.

All ten RV32 R-format instructions share the opcode 0110011, and `funct7` and `funct3` select the operation:

	add	sub	sll	slt	sltu	xor	srl	sra	or	and
funct7	0000000	0100000	0000000	0000000	0000000	0000000	0000000	0100000	0000000	0000000
funct3	000	000	001	010	011	100	101	101	110	111

Two of these are new: `slt` and `sltu` (set less than, signed and unsigned).

11.4 I-format

Instructions with immediates, such as `addi rd, rs1, imm` as opposed to `add rd, rs1, rs2`, need more than a 5-bit field, which only represents numbers up to 31. RISC-V would ideally have one

format, but it must compromise; it defines a new format that is mostly consistent with the R-format. An instruction with an immediate uses at most two registers (one source, one destination), so in the I-format `rs2` and `funct7` are replaced by a 12-bit signed immediate `imm[11:0]`, and the other fields (`rs1`, `funct3`, `rd`, `opcode`) stay as before. The immediate holds values in [−2048, +2047] and is always sign-extended to 32 bits before use in an arithmetic operation.

Example: `addi x15,x1,-50` has `imm` = −50 = 111111001110, `rs1` = 1 = 00001, `funct3` = 000 (add), `rd` = 15 = 01111, and `opcode` = 0010011 (OP-Imm).

The I-format arithmetic instructions share opcode 0010011: `addi` (000), `slti` (010), `sltiu` (011), `xori` (100), `ori` (110), `andi` (111), `slli` (001), and `srli` and `srai` (both 101). The shift-by-immediate instructions use only the lower 5 bits of the immediate as the shift amount `shamt` (shifting by 0–31), and one of the higher immediate bits (`0000000` versus `0100000` in bits 31–25) distinguishes `srli` from `srai`.

Loads are I-type. In a load, the 12-bit signed immediate is an offset added to the base address in `rs1` to form the memory address, much like add immediate but used to create an address rather than a final result. `funct3` gives the width, and the value loaded from memory goes into `rd`. For example, `lw x14, 8(x2)` is `imm` = +8 = 000000001000, `rs1` = 2, `funct3` = 010 (`lw`), `rd` = 14, `opcode` = 0000011 (LOAD). The loads, all with opcode 0000011, are `lb` (000), `lh` (001), `lw` (010), `lbu` (100), and `lhu` (101): `funct3` encodes both the size and the signedness of the loaded data. `lh` (load halfword) loads 16 bits and sign-extends them to fill the 32-bit register; `lhu` zero-extends them. There is no `lwu` in RV32, because copying 32 bits into a 32-bit register needs no extension.

11.5 S-format

A store must read two registers, `rs1` for the base address and `rs2` for the data, and also needs an immediate offset; it writes no register, so it has no `rd`. Since `rs2` and the immediate cannot share the same place, RISC-V moves the low 5 bits of the immediate to where `rd` is in other formats and keeps `rs1` and `rs2` in their usual places: register names are more critical than immediate bits in the hardware design. The S-format is thus `imm[11:5]`, `rs2`, `rs1`, `funct3`, `imm[4:0]`, `opcode`.

Example: `sw x14, 8(x2)` is `0000000 01110 00010 010 01000 0100011`: offset bits 11–5 are 0000000, `rs2` = 14, `rs1` = 2, `funct3` = 010 (`sw`), offset bits 4–0 are 01000, and the opcode is STORE. The two pieces combine to the 12-bit offset 0000000 01000 = 8.

The stores, all with opcode 0100011, are `sb` (000), `sh` (001), and `sw` (010), where `funct3` gives the width.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 11 (Fall 2020).

Lecture 12: RISC-V Instruction Formats II

12.1 Conditional branches and PC-relative addressing

A conditional branch such as `beq x1, x2, Label` reads two registers but writes none (like a store). How should the label be encoded? Branches are typically used for loops (`if-else`, `while`, `for`), which are generally small (fewer than 50 instructions); function calls and unconditional jumps use jump instructions (J-format). Instructions are stored in a localized area of memory (code, or text), so the largest branch distance is limited by the size of the code, and the address of the current instruction is in the PC.

PC-relative addressing therefore uses the immediate field as a two's complement offset from the PC: branches generally change the PC by a small amount, and a 12-bit immediate can specify $\pm 2^{11}$ "unit" addresses from the PC. Bytes would be a poor unit, because instructions are 32 bits (4 bytes) and we never branch into the middle of one. Multiplying the offset by four bytes before adding it to the PC would let one branch reach $\pm 2^{11}$ 32-bit instructions on either side of the PC, four times the reach of a byte offset. In that scheme an untaken branch sets PC = PC + 4 (the next instruction) and a taken branch sets PC = PC + immediate × 4, so the immediate counts instructions, forward (+) or backward (−).

Extensions to the RISC-V base ISA, however, support 16-bit compressed instructions and variable-length instructions that are multiples of 16 bits long. To allow for them, RISC-V scales the branch offset by 2 bytes even when there are no 16-bit instructions. This halves the branch reach and makes half of the possible targets errors on processors that support only 32-bit instructions (as here): conditional branches reach only $\pm 2^{10}$ 32-bit instructions on either side of the PC.

12.2 B-format

The B-format is mostly the same as the S-format, with two register sources (`rs1`, `rs2`) and a 12-bit immediate `imm[12:1]`, but the immediate now represents values from −4096 to +4094 in 2-byte increments. The 12 immediate bits encode even 13-bit signed byte offsets: the lowest bit of the offset is always zero, so it is not stored. Bits 31–25 hold `imm[12|10:5]` and bits 11–7 hold `imm[4:1|11]` (Figure 14). The hardware builds each 32-bit immediate from the instruction bits, and the formats are arranged so that the bits line up:

Immediate	`imm[31:12]`	`imm[11]`	`imm[10:5]`	`imm[4:1]`	`imm[0]`
I-immediate	`inst[31]`	`inst[31]`	`inst[30:25]`	`inst[24:21]`	`inst[20]`
S-immediate	`inst[31]`	`inst[31]`	`inst[30:25]`	`inst[11:8]`	`inst[7]`
B-immediate	`inst[31]`	`inst[7]`	`inst[30:25]`	`inst[11:8]`	0

The upper bits are always sign-extended from `inst[31]`, and only bit 7 of the instruction changes role between the S- and B-immediates. The branches all use opcode 1100011, with `funct3` selecting `beq` (000), `bne` (001), `blt` (100), `bge` (101), `bltu` (110), and `bgeu` (111).

Example: Consider the loop

```
Loop: beq  x19, x10, End
      add  x18, x18, x10
      addi x19, x19, -1
      j    Loop
End:                        # target instruction
```

The target of the `beq` is 4 instructions after it, so the offset is $4 \times 4 = 16$ bytes (an offset of 0 would branch to itself). The 13-bit immediate $16 = 0\,0000\,0001\,0000_2$ has `imm[12]` = 0, `imm[11]` = 0, `imm[10:5]` = 000000, `imm[4:1]` = 1000, and `imm[0]` discarded, so the instruction is `0 000000 01010 10011 000 1000 0 1100011`: `rs2` = 10, `rs1` = 19, `funct3` = 000 (BEQ), opcode BRANCH.

Moving individual lines of code changes the branch immediates, but moving all of the code does not: such code is POSITION-INDEPENDENT CODE. If a destination is more than 2^{10} instructions away, other instructions help: `beq x10,x0,far` can be replaced by `bne x10,x0,next`, then `j far`, then the label `next:`.

12.3 U-format and long immediates

The U-FORMAT has a 20-bit immediate in the upper 20 bits of the instruction word, one destination register `rd`, and an opcode. It is used by two instructions: LOAD UPPER IMMEDIATE (`lui`) and ADD UPPER IMMEDIATE TO PC (`auipc`).

`lui` writes the immediate into the upper 20 bits of the destination and clears the lower 12 bits. Together with an `addi` that sets the low 12 bits, it creates any 32-bit value in a register with two instructions: `lui x10, 0x87654` sets `x10` to `0x87654000`, and `addi x10, x10, 0x321` then makes it `0x87654321`. There is one corner case. Setting `0xDEADBEEF` with `lui x10, 0xDEADB` and `addi x10, x10, 0xEEF` gives `0xDEADAEEF`, because the 12-bit `addi` immediate is always sign-extended: if its top bit is set, it subtracts 1 from the upper 20 bits. The solution is to place the pre-incremented value in the upper 20 bits when the immediate's sign bit will be set: `lui x10, 0xDEADC` followed by `addi x10, x10, 0xEEF` gives `0xDEADBEEF`. The assembler pseudo-instruction `li x10, 0xDEADBEEF` handles all of this, creating the two instructions.

`auipc` adds the upper immediate to the PC and places the result in the destination register; it is used for PC-relative addressing. For example, `Label: auipc x10, 0` puts the address of `Label` in `x10`.

12.4 J-format and JALR

The J-FORMAT holds `jal`: bit 31 is `imm[20]`, bits 30–21 are `imm[10:1]`, bit 20 is `imm[11]`, bits 19–12 are `imm[19:12]`, followed by `rd` and the opcode. `jal` saves PC + 4 in `rd` (the return address) and sets PC = PC + offset, a PC-relative jump. The target can be anywhere within $\pm 2^{19}$ locations, 2 bytes apart, that is $\pm 2^{18}$ 32-bit instructions. As for branches, the immediate encoding is arranged to reduce hardware cost. The assembler's `j` is a pseudo-instruction that uses `jal` with `rd` = `x0` to discard the return address: `j Label` is `jal x0, Label`, while `jal ra, FuncName` calls a function within 2^{18} instructions of the PC.

JALR is I-format: `jalr rd, rs, immediate` writes PC + 4 to `rd` (the return address) and sets PC = `rs` + immediate. It uses the same immediates as arithmetic and loads, with no multiplication by 2 bytes, in contrast to branches and `jal`. Its uses:

```
# ret and jr pseudo-instructions
ret = jr ra = jalr x0, ra, 0
# Call function at any 32-bit absolute address
lui  x1, <hi20bits>
jalr ra, x1, <lo12bits>
# Jump PC-relative with 32-bit offset
auipc x1, <hi20bits>
jalr  x0, x1, <lo12bits>
```

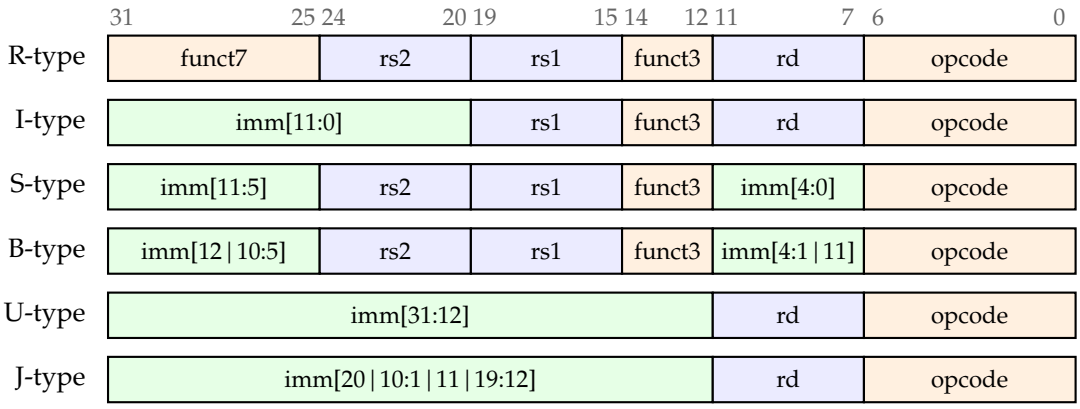


Figure 14: The six RV32 instruction formats. The register specifiers, `funct3`, and `opcode` keep the same positions in every format that has them.

These formats cover the complete base integer instruction set RV32I (shifts, arithmetic including `lui` and `auipc`, logical, compares, branches, jump and link, loads, and stores), apart from `fence` (synchronization) and `ecall` and `ebreak` (environment), which are not used here.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 12 (Fall 2020).

Lecture 13: Compilation, Assembly, Linking, Loading

13.1 Interpretation and translation

Languages range from those that are easy to program but inefficient to interpret (Python, Java, C++) down to those that are difficult to program but efficient to interpret (C, assembly, machine code, with Java bytecode in between). An interpreter is a program that directly executes programs in a source language; a translator converts a program from the source language to an equivalent program in another language. The Python interpreter, for example, is just a program that reads a Python program and performs its functions. In general we interpret a high-level language when efficiency is not critical and translate to a lower-level language to increase performance.

Machine language is sometimes interpreted in software too: the Venus RISC-V simulator is useful for learning and debugging, and when Apple switched the Macintosh from the Motorola 680x0 ISA to PowerPC, rather than require every program to be re-translated from its high-level source, executables could contain old and/or new machine code, with the old code interpreted in software when necessary (emulation).

An interpreter is generally easier to write, and being closer to the high-level language it can give better error messages (translators respond by adding debugging information such as line numbers and names). Interpreted code is slower (perhaps 10×) but smaller (perhaps 2×), and an interpreter provides instruction-set independence: the program runs on any machine. Translated code is almost always more efficient, which matters for many applications, particularly operating systems. Translation also helps hide the program source from users, one model for creating value in the marketplace; the alternative "open source" model creates value by publishing the source and fostering a community of developers.

13.2 Compiler and assembler

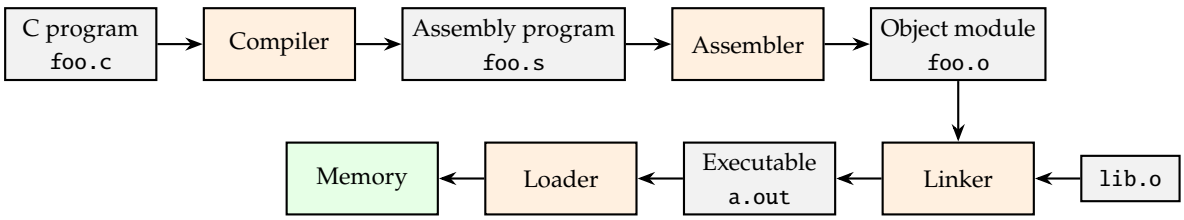


Figure 15: Steps in compiling and running a C program (`gcc -O2 -S -c foo.c` stops after the compiler and keeps `foo.s`).

The COMPILER takes high-level language code (`foo.c`) and outputs assembly language code (`foo.s` for RISC-V); the output may contain PSEUDO-INSTRUCTIONS, instructions the assembler understands but that are not in the machine, such as `mv t1,t2` for `addi t1,t2,0` (Figure 15).

The ASSEMBLER takes assembly language code, including pseudo operations, and outputs object code plus information tables (`foo.o`). It reads and uses directives, replaces pseudo-instructions, produces machine language, and creates the object file. DIRECTIVES give directions to the assembler but produce no machine instructions:

`.text`	subsequent items go in the user text segment (machine code)
`.data`	subsequent items go in the user data segment (source file data in binary)
`.globl sym`	declares `sym` global, so other files can reference it
`.string str`	stores the string `str` in memory and null-terminates it
`.word w1...wn`	stores the n 32-bit quantities in successive memory words

The assembler treats convenient variations of machine instructions as if they were real instructions:

Pseudo-instruction	Real instructions
`mv t0, t1`	`addi t0, t1, 0`
`neg t0, t1`	`sub t0, zero, t1`
`li t0, imm`	`addi t0, zero, imm`
`not t0, t1`	`xori t0, t1, -1`
`beqz t0, loop`	`beq t0, zero, loop`
`la t0, str`	`lui t0, str[31:12]`; `addi t0, t0, str[11:0]` (static addressing) or `auipc t0, str[31:12]`; `addi t0, t0, str[11:0]` (PC-relative)

(Remember that immediates are sign-extended and that branch immediates count halfwords.)

Producing machine language. For arithmetic, logical, shift, and similar instructions, all the necessary information is already in the instruction. PC-relative branches and jumps (`beq`, `bne`, `jal`) can also be handled: once pseudo-instructions are replaced by real ones, the assembler knows how many instructions to branch or jump over. Branches can, however, refer *forward* to labels:

```
1      addi t2, zero, 9      # t2 = 9
2  L1: slt  t1, zero, t2     # 0 < t2? Set t1
3      beq  t1, zero, L2     # NO! t2 <= 0; Go to L2
4      addi t2, t2, -1       # YES! t2 > 0; t2--
5      j    L1               # Goto L1
6  L2:
```

Here `j L1` jumps 3 words (6 halfwords) back and `beq` branches 3 words (6 halfwords) forward. This FORWARD REFERENCE problem is solved by taking two passes over the program: the first pass

remembers the positions of labels, and the second uses them to generate code. The pseudo-instruction `j offset` expands to `jal zero, offset`, and counting the instruction halfwords between target and jump gives the offset; such code is position-independent (PIC).

References to static data are harder. `la` becomes `lui` and `addi` (or `auipc` and `addi` for PIC), which need the full 32-bit address of the data, and that cannot be determined yet. So the assembler creates two tables. The SYMBOL TABLE lists the items in this file that other files may use: labels (for function calls) and data (anything in the `.data` section, variables that may be accessed across files). The RELOCATION TABLE lists the items whose address this file needs: any absolute label jumped to (`jal`, `jalr`), internal or external (including library files), as in the `la` instruction or a `jalr` base register; and any piece of data in the static section, as in `la` or a `lw/sw` base register.

An OBJECT FILE contains a header (the size and position of the other pieces), the text segment (the machine code), the data segment (the binary representation of the static data in the source file), relocation information (the lines of code that must be fixed up later), the symbol table (this file's labels and static data that can be referenced), and debugging information. A standard format is ELF (Microsoft uses its own).

13.3 Linker

The LINKER takes object code files and their information tables (for example `foo.o` and `libc.o`) and outputs executable code (`a.out`). Combining several `.o` files into a single executable enables separate compilation: changing one file does not require recompiling the whole program (the Linux source has more than 20 million lines of code). The old name "link editor" comes from editing the "links" in jump and link instructions. The linker works in three steps (Figure 16): it puts together the text segments of all the `.o` files; it puts together their data segments and concatenates them onto the end of the text; and it resolves references, going through the relocation table and handling each entry, that is, filling in all absolute addresses.

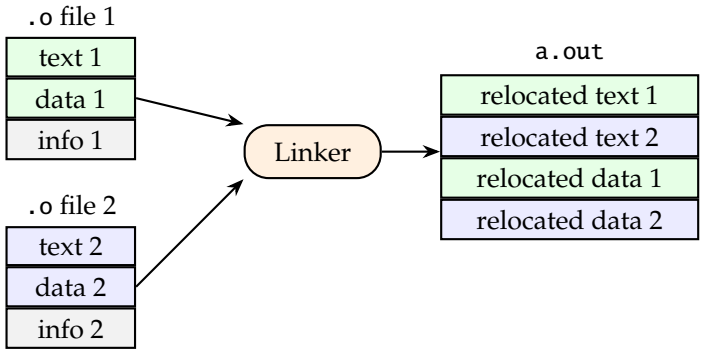


Figure 16: The linker places the text segments of all object files first, then their data segments, and then resolves references using each file's information (symbol and relocation tables).

There are four types of addresses:

- PC-relative addressing (`beq`, `bne`, `jal`; `auipc/addi`): never needs relocation (PIC);
- absolute function address (`auipc/jalr`): always relocate;
- external function reference (`auipc/jalr`): always relocate;
- static data reference (often `lui/addi`): always relocate.

So the instructions that may need relocation editing are J-format jumps and jump and links, and I- and S-format loads and stores of variables in the static area, relative to the global pointer. Conditional branches do not: PC-relative addressing is preserved even if the code moves.

To resolve references, the linker assumes that the first word of the first text segment is at address `0x10000` for RV32. It knows the length and ordering of every text and data segment, so it can calculate the absolute address of each label to be jumped to (internal or external) and of each piece of data being referenced. For each reference it searches all the "user" symbol tables; if the symbol is not found, it searches library files (for `printf`, say). Once the absolute address is determined, it fills in the machine code appropriately. The output is an executable file containing text and data (plus a header).

Static and dynamic linking. The traditional approach just described links libraries *statically*: the library becomes part of the executable, so if the library is updated the program does not get the fix (it must be recompiled, if we have the source); the entire library is included even if not all of it is used; and the executable is self-contained. The alternative is DYNAMICALLY LINKED LIBRARIES (DLLs), common on Windows and UNIX platforms. They save space and time: storing a program requires less disk space, sending it takes less time, and two programs sharing a library need less memory when executing; but there is time overhead at run time to do the link. Replacing one file (`libXYZ.so`) upgrades every program that uses library XYZ, but having the executable is no longer enough. Overall, dynamic linking adds quite a bit of complexity to the compiler, linker, and operating system, but its benefits often outweigh this. The prevailing approach links at the machine code level, using machine code as the "lowest common denominator": the linker does not use information about how the program or library was compiled (which compiler or language). This is not the only way to do it.

13.4 Loader

The LOADER takes an executable (`a.out`) and runs it. Executable files are stored on disk, and when one is run the loader's job is to load it into memory and start it running. In reality the loader is the operating system: loading is one of its tasks. The loader

1. reads the executable file's header to determine the sizes of the text and data segments;
2. creates a new address space for the program, large enough to hold the text and data segments along with a stack segment;

3. copies the instructions and data from the executable file into the new address space;
4. copies the arguments passed to the program onto the stack;
5. initializes the machine registers: most are cleared, but the stack pointer is assigned the address of the first free stack location;
6. jumps to a start-up routine that copies the program's arguments from the stack to registers and sets the PC. If the main routine returns, the start-up routine terminates the program with the `exit` system call.

13.5 Example: Hello world

The program

```c
#include <stdio.h>
int main() {
  printf("Hello, %s\n", "world");
  return 0;
}
```

compiles to

```
    .text                   # Directive: enter text section
    .align 2                # Directive: align code to 2^2 bytes
    .globl main             # Directive: declare global symbol main
main:                       # label for start of main
    addi sp,sp,-16          # allocate stack frame
    sw   ra,12(sp)          # save return address
    lui  a0,%hi(string1)    # compute address of
    addi a0,a0,%lo(string1) #   string1
    lui  a1,%hi(string2)    # compute address of
    addi a1,a1,%lo(string2) #   string2
    call printf             # call function printf
    lw   ra,12(sp)          # restore return address
    addi sp,sp,16           # deallocate stack frame
    li   a0,0               # load return value 0
    ret                     # return
    .section .rodata        # Directive: enter read-only data section
    .balign 4               # Directive: align data section to 4 bytes
string1:                    # label for first string
    .string "Hello, %s\n"   # Directive: null-terminated string
string2:                    # label for second string
    .string "world"         # Directive: null-terminated string
```

Assembling gives the linkable `Hello.o`, in which the addresses of the strings and of `printf` are still placeholders:

```
00000000 <main>:
 0: ff010113 addi sp,sp,-16
 4: 00112623 sw   ra,12(sp)
 8: 00000537 lui  a0,0x0      # addr placeholder
 c: 00050513 addi a0,a0,0     # addr placeholder
10: 000005b7 lui  a1,0x0      # addr placeholder
14: 00058593 addi a1,a1,0     # addr placeholder
18: 00000097 auipc ra,0x0     # addr placeholder
1c: 000080e7 jalr ra          # addr placeholder
20: 00c12083 lw   ra,12(sp)
24: 01010113 addi sp,sp,16
28: 00000513 addi a0,zero,0
2c: 00008067 jalr ra
```

Linking places `main` at `0x101b0` and fills in the addresses; the call becomes a single `jal` to `printf`:

```
000101b0 <main>:
101b0: ff010113 addi sp,sp,-16
101b4: 00112623 sw   ra,12(sp)
101b8: 00021537 lui  a0,0x21
101bc: a1050513 addi a0,a0,-1520   # 20a10 <string1>
101c0: 000215b7 lui  a1,0x21
101c4: a1c58593 addi a1,a1,-1508   # 20a1c <string2>
101c8: 288000ef jal  ra,10450      # <printf>
101cc: 00c12083 lw   ra,12(sp)
101d0: 01010113 addi sp,sp,16
101d4: 00000513 addi a0,zero,0
101d8: 00008067 jalr ra
```

Example: The target address of `string1` is `0x00020A10`. The sequence `lui 0x00020`, `addi 0xA10` does not quite work, because RISC-V immediates are sign-extended and `0xA10` has a 1 in its high-order bit: `0x00020000` + `0xFFFFFA10` = `0x0001FA10`, off by `0x00001000`. Adding that back, (`0x00020000` + `0x00001000`) + `0xFFFFFA10` = `0x00020A10`. The two's complement of `0xFFFFFA10` is `0x000005EF` + 1 = `0x000005F0` = 1520, so `0xFFFFFA10` = −1520, and the sequence `lui 0x00021`, `addi -1520` computes `0x00020A10`.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 13 (Fall 2020).

Lecture 14: Intro to Synchronous Digital Systems

Below the instruction set architecture, a machine is built from layers of hardware: the processor, memory, and I/O system; their datapath and control; digital design; circuit design; transistors; and fabrication. In the "new-school" view, parallelism appears at every level: parallel requests assigned to computers (a search for "cats"), parallel threads assigned to cores (lookups, ads), parallel instructions (more than one instruction at a time, such as five pipelined instructions), parallel data (more than one data item at a time, such as adding four pairs of words), and hardware descriptions, in which all gates work in parallel at the same time.

Abstraction (great idea #1) continues below machine language:

Level of representation	Example
High-level language program (C)	`temp = v[k]; v[k] = v[k+1]; v[k+1] = temp;`
Assembly language program (RISC-V)	`lw x3,0(x10); lw x4,4(x10); sw x4,0(x10); sw x3,4(x10)`
Machine language program (RISC-V)	32-bit binary instruction words
Hardware architecture description	block diagrams of the datapath (PC, IMEM, registers, ALU, DMEM)
Logic circuit description	circuit schematics, e.g. gates computing $Out = AB + CD$

The compiler and assembler connect the first three levels; the architecture implementation connects the block diagrams to logic circuits.

Definition: The hardware of a processor such as RISC-V is a SYNCHRONOUS DIGITAL SYSTEM. *Synchronous*: all operations are coordinated by a central clock, the "heartbeat" of the system. *Digital*: all values are represented by discrete values; electrical signals are treated as 1s and 0s and grouped together to form words.

Studying hardware design shows the capabilities and limitations of hardware in general and processors in particular: what processors do fast and what they do slowly (avoid the slow things to make code fast). It is also the background for designing custom hardware when standard processors cannot deliver enough performance.

14.1 Switches

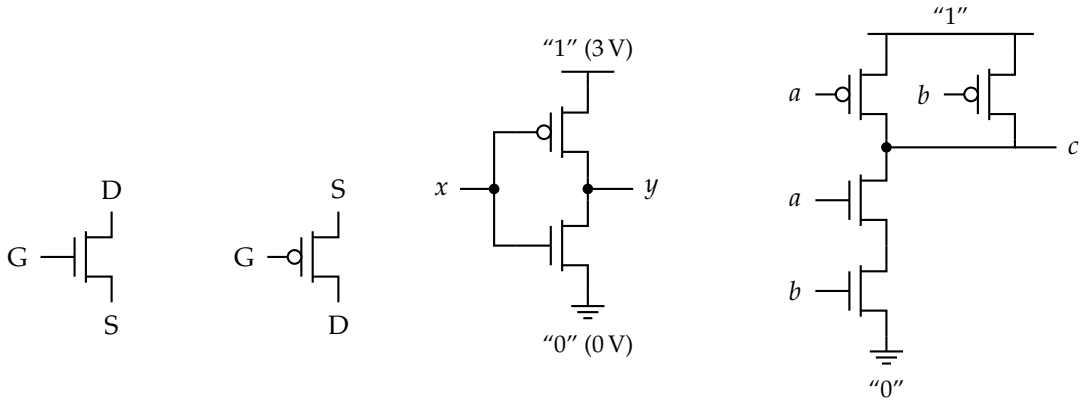
The basic element of a physical circuit is a switch. A switch controlled by A closes when A is 1 (asserted), completing a circuit that lights a bulb Z, and opens when A is 0 (unasserted), so $Z \equiv A$. Composing switches gives Boolean functions: two switches in series conduct only when both are closed, so $Z \equiv A$ and B; two in parallel conduct when either is closed, so $Z \equiv A$ or B.

Early designers built ad hoc circuits from switches and noticed recurring patterns (ANDs, ORs, . . .). Claude Shannon's master's thesis linked such switching circuits to the algebra of the 19th-century mathematician George Boole (hence "Boolean"), so mathematics could give a theory of hardware design, minimization, and so on.

14.2 Transistors

The transistor ("born" December 23, 1947; John Bardeen, William Shockley, Walter Brattain) is a semiconductor device that amplifies or switches signals and is the key component of all modern electronics. It replaced vacuum tubes and led to the integrated circuit and the microprocessor.

Modern digital systems are designed in CMOS: MOS stands for metal-oxide on semiconductor, and C for complementary, since CMOS uses both normally-open and normally-closed switches. MOS transistors act as voltage-controlled switches. A MOS transistor has three terminals, drain, gate, and source: if the voltage on the gate is (some amount) higher or lower than that on the source, a conducting path is established between drain and source. An N-CHANNEL transistor is open when the gate voltage is low and closes when $V(G) > V(S) + \varepsilon$; a P-CHANNEL transistor (drawn with a circle on its gate) behaves complementarily, closed when the gate voltage is low and open when it is high (Figure 17).



(a) n-channel closed when G high (b) p-channel closed when G low (c) inverter (d) NAND

Figure 17: (a), (b) MOS transistor symbols with gate G, source S, and drain D. (c) The CMOS inverter: $x = 0$ V closes the p-channel transistor and gives $y = 3$ V; $x = 3$ V closes the n-channel transistor and gives $y = 0$ V. (d) The CMOS NAND gate: parallel p-channel transistors to "1", series n-channel transistors to "0".

In the inverter, the p-channel transistor connects the output y to the voltage source ("1", 3 V) and the n-channel transistor connects it to ground ("0", 0 V); exactly one of them conducts, so $x = 0$ V gives $y = 3$ V and $x = 3$ V gives $y = 0$ V.

Chips are composed of nothing but transistors and wires, and small groups of transistors form useful building blocks. The four-transistor circuit of Figure 17(d) pulls c to ground only when both

series n-channel transistors conduct ($a = b = 1$); otherwise a p-channel transistor connects c to "1". It is the NAND block:

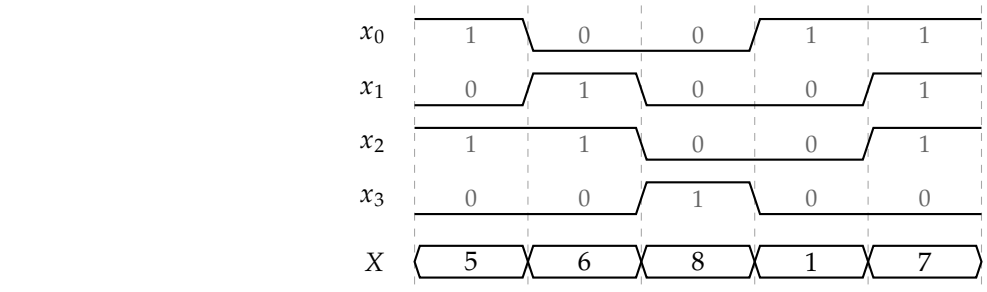
a	0	0	1	1
b	0	1	0	1
c	1	1	1	0

Blocks are organized in a hierarchy to build higher-level blocks such as adders, and AND, OR, and NOT can all be built out of NAND.

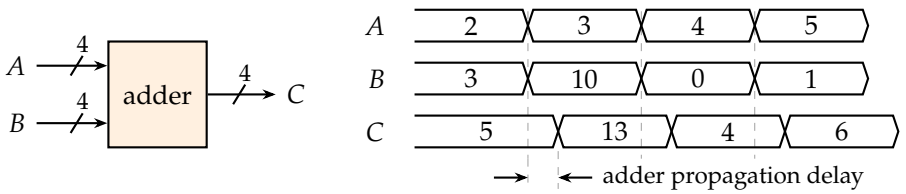
14.3 Signals and waveforms

The clock is a periodic signal alternating between a high voltage (3 V) and a low voltage (0 V); its period is $T = 1/\text{frequency}$, roughly 1 ns. A SIGNAL, when digital, is treated only as 1 or 0. It is transmitted over a wire continuously, transmission is effectively instant, and so a wire contains one value at a time.

Real waveforms are analog voltages: they are noisy, and a circuit's outputs change only some delay after its inputs. Signals are also grouped: the bits x_3, x_2, x_1, x_0 form a 4-bit bus X, drawn as one waveform labelled with its value in each interval (Figure 18(a)). Every circuit has a delay: when the inputs $A = [a_3, a_2, a_1, a_0]$ and $B = [b_3, b_2, b_1, b_0]$ of a 4-bit adder change, its output C changes one PROPAGATION DELAY later (Figure 18(b)). Logic simulators display a circuit's behavior as such waveforms for debugging.



(a) grouping $x_3x_2x_1x_0$ into the bus X



(b) a 4-bit adder: $C = A + B$ follows its inputs after the propagation delay

Figure 18: (a) Four one-bit signals grouped into the bus $X = x_3x_2x_1x_0$, whose value in successive intervals is $5, 6, 8, 1, 7$. (b) A 4-bit adder (a slash with a number marks a bus of that width): the sums $5, 13, 4, 6$ appear on C only after the adder propagation delay.

14.4 Types of circuits

Synchronous digital systems are made of two basic types of circuits.

Definition: In a COMBINATIONAL LOGIC (CL) circuit, such as the adder above, the output is a function of the inputs only. Like a pure function $y = f(x)$ in mathematics, it has no way to store information from one invocation to the next and no side effects. STATE ELEMENTS are circuits that store information.

A register is a state element: it holds an n-bit value on its output and loads the n-bit value on its input when its LOAD (or clock, CLK) input tells it to.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 14 (Fall 2020).

Lecture 15: State, State Machines

State elements have two uses. They store values for some indeterminate amount of time: register files (like `x0`–`x31` in RISC-V) and memory (caches and main memory). They also control the flow of information between combinational logic blocks: they hold up the movement of information at the inputs to combinational logic blocks and allow for its orderly passage.

15.1 The accumulator

Why control the flow of information? Suppose a circuit should compute

```
S = 0;
for (i = 0; i < n; i++)
  S = S + X[i];
```

where the values X_i are applied in succession, one per cycle, and after n cycles the sum is present on S. A first try feeds the output of an adder straight back to one of its inputs. This does not work: nothing controls the next iteration of the loop (the feedback loop simply keeps adding as fast as signals propagate), and there is no way to say $S = 0$.

The fix is a register in the feedback path that holds up the transfer of data to the adder. It loads the adder's output once per cycle, on LOAD or the clock CLK, so S takes the values X_0, $X_0 + X_1$, $X_0 + X_1 + X_2, \ldots$ in successive cycles as X takes the values X_0, X_1, $X_2, \ldots$.

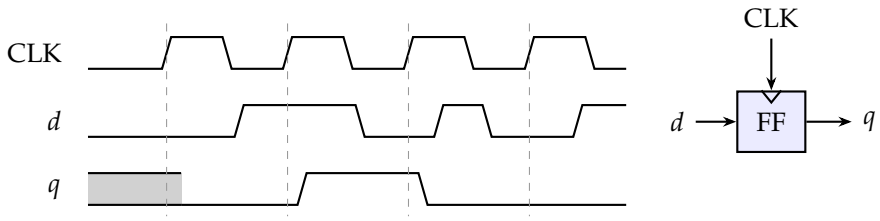
15.2 Flip-flops

An n-bit register is n instances of a FLIP-FLOP, all sharing the clock; flip-flop i takes input bit d_i and produces output bit q_i. The name comes from the output flipping and flopping between 0 and 1. The input D is "data" and the output Q is the "output"; the circuit is also called a D-TYPE FLIP-FLOP (there used to be other types).

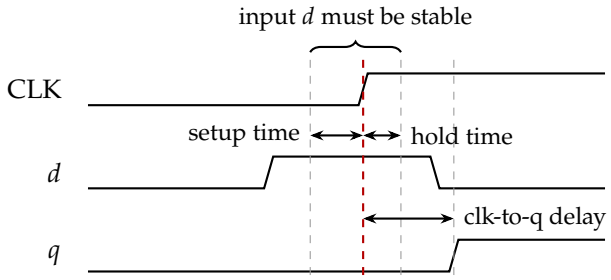
The flip-flops here are edge-triggered and, specifically, rising-edge (positive-edge) triggered; falling-edge flip-flops also exist. On the rising edge of the clock, the input d is sampled and transferred to the output. At all other times, the input d is ignored (Figure 19(a)).

Definition: The CLOCK (CLK) is a steady square wave that synchronizes the system. The SETUP TIME is how long the input must be stable before the rising edge of the clock, and the HOLD TIME how long it must stay stable after the rising edge. The CLK-TO-Q DELAY is how long the output takes to change, measured from the rising edge. A flip-flop is one bit of state that samples its input at every rising

edge of the clock; a register is several bits of state that sample on the rising edge of the clock or on LOAD.



(a) q takes the value of d sampled at each rising edge



(b) setup time, hold time, and clk-to-q delay around a rising edge

Figure 19: A rising-edge-triggered D flip-flop. (a) At each rising edge (dashed) q takes the value d has at that instant, here $0, 1, 0, 0$; changes of d between edges are ignored, and q is unknown (gray) until the first edge. (b) The input must be stable from one setup time before the edge until one hold time after it; the output changes one clk-to-q delay after the edge.

15.3 Accumulator timing

A RESET input forces the register to all zeros and takes priority over its D input; asserting it at the first rising edge implements $S = 0$. The register output S_{i-1} holds the result of the $(i-1)$th iteration, and the adder computes $S_i = S_{i-1} + X_i$. Timing is analyzed starting at the output of the register (Figure 20): one clk-to-q delay after each rising edge the register output changes, and one adder delay later S_i is valid. In practice X_i may not arrive at the adder at the same time as S_{i-1}, so S_i is temporarily wrong, but the register always captures the correct value as long as S_i has settled before the next rising edge. In good circuits, instability never happens around the rising edge of the clock.

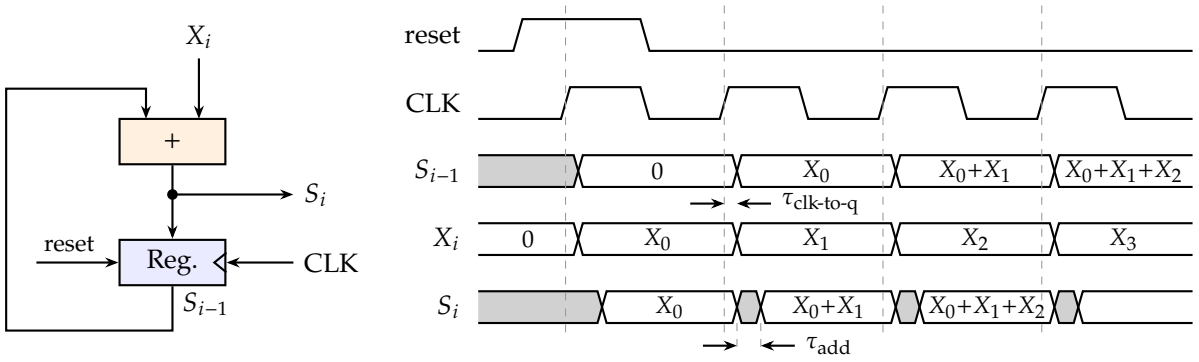


Figure 20: The accumulator with a resettable register and its timing. Reset is asserted at the first rising edge, so S_{i-1} becomes 0; after each later edge S_{i-1} changes after $\tau_{\text{clk-to-q}}$ and S_i settles (gray: unknown or changing) after a further τ_{add}.

15.4 Maximum clock frequency

In a circuit whose registers feed combinational logic that feeds registers (Figure 21), each clock period must leave time for a new value to leave a register, propagate through the logic, and be stable at the next register's input before the next rising edge:

$$\text{max delay} = \text{clk-to-q delay} + \text{CL delay} + \text{setup time}, \qquad f_{\max} = \frac{1}{\text{max delay}},$$

since frequency = 1/period.

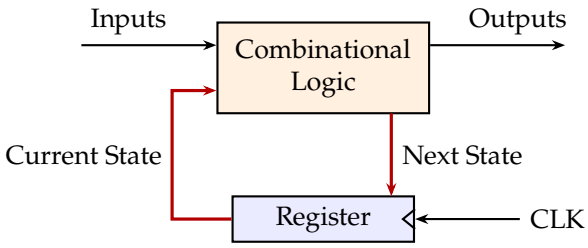
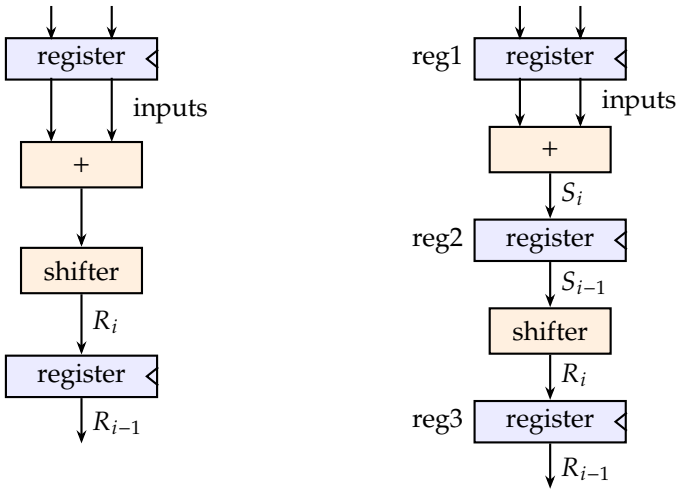


Figure 21: Combinational logic whose current state comes from a register and whose next state is stored back into it. The red path from register output to register input sets the minimum clock period.

15.5 Pipelining for performance

Extra registers are often added to speed up the clock rate. In Figure 22(a) an adder and a shifter sit between two registers. A result appears at the output one clock cycle after its inputs are applied, and the clock period is limited by the propagation delay of the adder and the shifter together. Inserting a register between them (Figure 22(b)) means each cycle has to cover only the adder delay or only the shifter delay, which allows a higher clock frequency: a result now takes two (shorter) cycles to emerge, but a new result emerges every cycle, so there are more outputs per second. Long-delay

combinational logic is pipelined for a faster clock.



(a) one register stage (b) pipelined with an extra register

Figure 22: (a) Adder and shifter between two registers: the period must cover both delays. (b) With a register between adder and shifter, inputs i produce S_i in one cycle and R_{i-1} two cycles later, while new inputs enter every cycle.

15.6 Finite state machines

A FINITE STATE MACHINE (FSM) can be represented by a STATE TRANSITION DIAGRAM, and with combinational logic and registers any FSM can be implemented in hardware. State transitions are controlled by the clock: on each clock cycle the machine checks its input, moves to a new state, and produces a new output.

Example: An FSM that detects three consecutive 1s in its input has states S0, S1, S2 (the number of consecutive 1s seen) and outputs 1 exactly on the third 1 (Figure 23). On the input below the output is 1 at the third 1 of each run, so a run of six 1s produces two 1s:

```
input   01011011101111011111110
output  00000000100001000010010
```

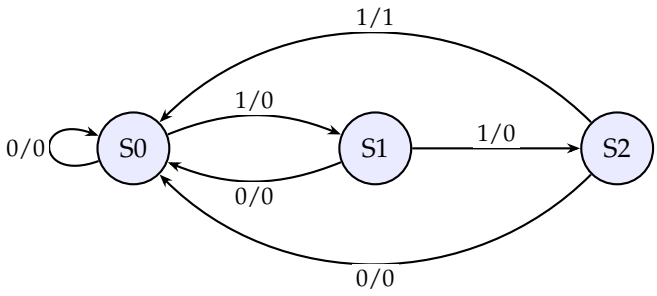


Figure 23: State transition diagram of the three-consecutive-1s detector; each arc is labelled input/output.

The hardware has the structure of Figure 21. A register holds the representation of the state the machine is in, using a unique bit pattern for each state (S0 = 00, S1 = 01, S2 = 10). A combinational logic circuit implements the function mapping the input and the present state (PS) to the next state (NS) and the output. Its functional specification is the truth table

PS	Input	NS	Output
00	0	00	0
00	1	01	0
01	0	00	0
01	1	10	0
10	0	00	0
10	1	00	1

15.7 The general model

A synchronous system is a collection of combinational logic blocks separated by registers. Registers may be back-to-back and CL blocks may be back-to-back; feedback is optional; and the clock signal connects only to the clock inputs of registers. Designs are organized hierarchically: a system consists of a datapath (code registers, multiplexers, comparators) and control (state registers and combinational logic); these are built from registers and logic, which are built from switching networks.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 15 (Fall 2020).

Lecture 16: Combinational Logic

16.1 Truth tables

A combinational block with inputs a, b, c, d and output $y = F(a, b, c, d)$ is specified completely by its truth table, which lists F for each of the $2^4 = 16$ input combinations. Since each of the 16 rows can be 0 or 1, there are 2^{16} different 4-input functions.

Example: *Exactly one of a, b.* The function that is 1 iff one (not both) of a, b is 1 has $y = 0, 1, 1, 0$ for $ab = 00, 01, 10, 11$. Grouped by a, the table compresses to: $y = b$ when $a = 0$ and $y = \overline{b}$ when $a = 1$.

Example: *Adders.* A 2-bit adder takes $A = a_1a_0$ and $B = b_1b_0$ and produces the 3-bit sum $C = c_2c_1c_0$; its truth table has $2^4 = 16$ rows:

$A \backslash B$	00	01	10	11
00	000	001	010	011
01	001	010	011	100
10	010	011	100	101
11	011	100	101	110

A 32-bit unsigned adder has 64 input bits and a 33-bit output (from $000\ldots0 + 000\ldots0 = 000\ldots00$ to $111\ldots1 + 111\ldots1 = 111\ldots10$), so its truth table would have 2^{64} rows. Truth tables describe large blocks only in principle.

Example: *3-input majority.* y is 1 when at least two of a, b, c are 1; the table is in Figure 25.

16.2 Logic gates

The basic gates are drawn as in Figure 24 (the AND symbol is shaped like the "D" in "AND"); their truth tables are

a	b	AND	OR	XOR	NAND	NOR
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

and NOT maps $a = 0, 1$ to $b = 1, 0$.

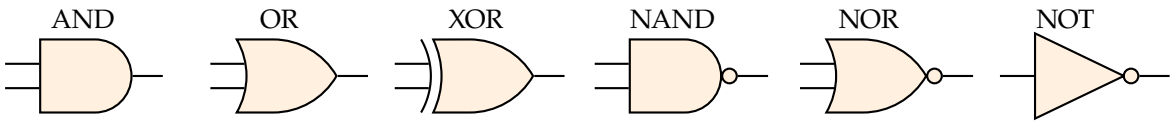


Figure 24: Logic gate symbols, inputs on the left and output on the right. A bubble on an output (or input) inverts it.

Two-input gates extend to n inputs: an n-input AND is 1 iff all inputs are 1, and an n-input OR iff any input is 1. The n-input XOR is the only one that is not obvious, but it is simple: XOR is 1 iff the number of 1s at its inputs is odd. The tables of the 3-input XOR and the majority function, and of the example used below, are in Figure 25.

a	b	c	XOR
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

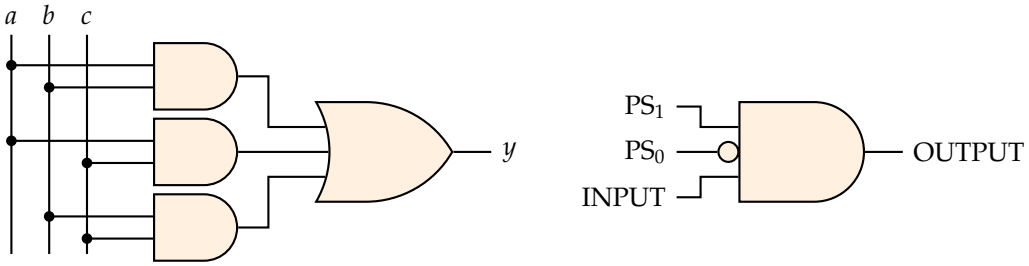
a	b	c	majority
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

a	b	c	y	product term
0	0	0	1	$\overline{a} \cdot \overline{b} \cdot \overline{c}$
0	0	1	1	$\overline{a} \cdot \overline{b} \cdot c$
0	1	0	0	
0	1	1	0	
1	0	0	1	$a \cdot \overline{b} \cdot \overline{c}$
1	0	1	0	
1	1	0	1	$a \cdot b \cdot \overline{c}$
1	1	1	0	

Figure 25: Three-input truth tables: XOR (1 iff an odd number of inputs are 1), majority, and the function used for the sum-of-products example, with the product term for each row where $y = 1$.

16.3 From truth tables to gates

A truth table can be implemented directly with gates. The majority function is 1 when a and b are 1, or a and c, or b and c, so three AND gates feed an OR gate (Figure 26(a)). In the truth table of the three-consecutive-1s FSM (states S0 = 00, S1 = 01, S2 = 10), the output is 1 only in the row with present state 10 and input 1, that is, only when $PS_1 = 1$, $PS_0 = 0$, and the input is 1: one 3-input AND gate with PS_0 inverted, by a NOT gate or equivalently by a bubble on that input (Figure 26(b)).



(a) $y = ab + ac + bc$ (b) OUTPUT = $PS_1 \cdot \overline{PS_0} \cdot$ INPUT

Figure 26: (a) The majority circuit. (b) The FSM output logic; the bubble inverts PS_0.

16.4 Boolean algebra

George Boole, a 19th-century mathematician, developed a mathematical system (an algebra) involving logic, later known as Boolean algebra. Its primitive functions are AND, OR, and NOT, written + for OR, · for AND, and $\overline{x}$ for NOT. Its power is a one-to-one correspondence between circuits made of AND, OR, and NOT gates and equations in Boolean algebra. The majority circuit is

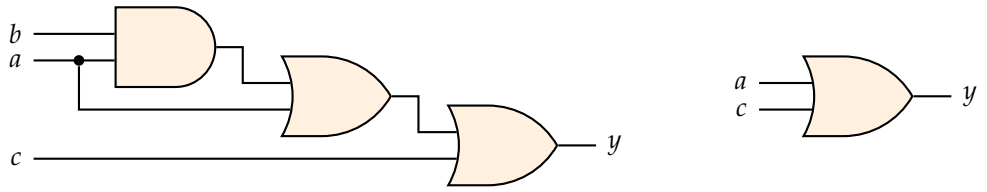
$$y = a \cdot b + a \cdot c + b \cdot c = ab + ac + bc,$$

and the FSM output is OUTPUT = $\text{PS}_1 \cdot \overline{\text{PS}_0} \cdot \text{INPUT}$.

Algebra simplifies circuits. The circuit of Figure 27(a) computes $y = ab + a + c$, and

$$\begin{aligned} y &= ab + a + c \\ &= a(b + 1) + c && \text{distribution, identity} \\ &= a(1) + c && \text{law of 1's} \\ &= a + c && \text{identity,} \end{aligned}$$

so a single OR gate suffices (Figure 27(b)). Boolean algebra is also good for circuit verification: to show that circuit X equals circuit Y, prove their equations equal.



(a) $y = ab + a + c$ (b) $y = a + c$

Figure 27: (a) A circuit computing $ab + a + c$. (b) The simplified equivalent circuit $a + c$.

The laws of Boolean algebra are:

$x \cdot \overline{x} = 0$	$x + \overline{x} = 1$	complementarity
$x \cdot 0 = 0$	$x + 1 = 1$	laws of 0's and 1's
$x \cdot 1 = x$	$x + 0 = x$	identities
$x \cdot x = x$	$x + x = x$	idempotent law
$x \cdot y = y \cdot x$	$x + y = y + x$	commutative law
$(xy)z = x(yz)$	$(x + y) + z = x + (y + z)$	associativity
$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	distribution
$xy + x = x$	$(x + y)x = x$	uniting theorem
$\overline{x \cdot y} = \overline{x} + \overline{y}$	$\overline{x + y} = \overline{x} \cdot \overline{y}$	DeMorgan's law

16.5 Canonical forms

Every truth table has a canonical SUM-OF-PRODUCTS form (an OR of ANDs): for each row where the output is 1, take the product (AND) of all inputs, each complemented if it is 0 in that row, and OR these products together. For the third table of Figure 25,

$$\begin{aligned} y &= \overline{a}\,\overline{b}\,\overline{c} + \overline{a}\,\overline{b}c + a\overline{b}\,\overline{c} + ab\overline{c} \\ &= \overline{a}\,\overline{b}(\overline{c} + c) + a\overline{c}(\overline{b} + b) && \text{distribution} \\ &= \overline{a}\,\overline{b}(1) + a\overline{c}(1) && \text{complementarity} \\ &= \overline{a}\,\overline{b} + a\overline{c} && \text{identity,} \end{aligned}$$

which needs two AND gates and one OR gate (Figure 28). Truth tables, Boolean expressions, and gate diagrams are three descriptions of the same function, and these techniques translate any one into the others.

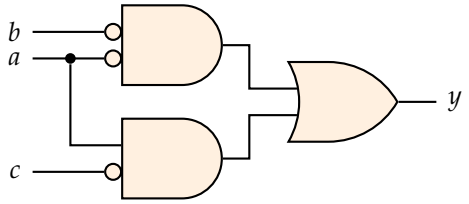


Figure 28: The circuit for $y = \overline{a}\,\overline{b} + a\overline{c}$; the bubbles invert b, a, and c at the AND inputs.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 16 (Fall 2020).

Lecture 17: Combinational Logic Blocks

17.1 Data multiplexors

A data multiplexor ("mux") selects one of its inputs. The 2-to-1, n-bit-wide mux of Figure 29(a) outputs $C = A$ when $S = 0$ and $C = B$ when $S = 1$. It is n instances of a 1-bit-wide mux, all sharing S. The 1-bit mux has inputs s, a, b, so its truth table has 8 rows:

s	0	0	0	0	1	1	1	1
ab	00	01	10	11	00	01	10	11
c	0	0	1	1	0	1	0	1

Its sum of products simplifies to two AND gates and an OR gate (Figure 29(b)):

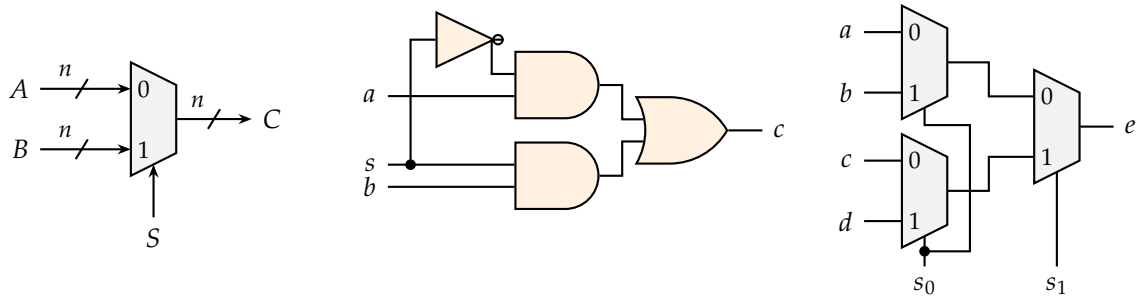
$$c = \overline{s}a\overline{b} + \overline{s}ab + s\overline{a}b + sab = \overline{s}(a\overline{b} + ab) + s(\overline{a}b + ab)$$
$$= \overline{s}(a(\overline{b} + b)) + s((\overline{a} + a)b) = \overline{s}(a(1)) + s((1)b) = \overline{s}a + sb,$$

which is the compressed table "$c = a$ when $s = 0$, $c = b$ when $s = 1$".

A 4-to-1 mux selects among a, b, c, d with a 2-bit select $S = s_1 s_0$: $e = a, b, c, d$ when $S = 00, 01, 10, 11$, so

$$e = \overline{s_1}\,\overline{s_0}a + \overline{s_1}s_0b + s_1\overline{s_0}c + s_1s_0d.$$

Its truth table has $2^6 = 64$ rows (six inputs). It can also be built hierarchically, like a tournament bracket: two 2-to-1 muxes controlled by s_0 choose between a, b and between c, d, and a third controlled by s_1 chooses between the winners (Figure 29(c)). In general, S select bits choose among 2^S inputs, and each input can be n bits wide independently of S.



(a) 2-to-1 mux (b) $c = \overline{s}a + sb$ (c) hierarchical 4-to-1 mux

Figure 29: (a) A 2-to-1 mux; a slash marks an n-bit bus. (b) A 1-bit mux from gates. (c) A 4-to-1 mux built from three 2-to-1 muxes.

17.2 Arithmetic and logic unit

Most processors contain a special logic block called the ARITHMETIC AND LOGIC UNIT (ALU). A simple ALU on 32-bit inputs A, B with a 2-bit control S computes

$$R = \begin{cases} A + B & S = 00, \\ A - B & S = 01, \\ A \,\&\, B & S = 10, \\ A \mid B & S = 11. \end{cases}$$

It is built from an adder/subtractor (controlled by S_0, with an overflow output), a bitwise AND, and a bitwise OR, all fed by A and B, and two muxes: one picks AND or OR by S_0, and the other picks the arithmetic or the logical result by S_1 (Figure 30).

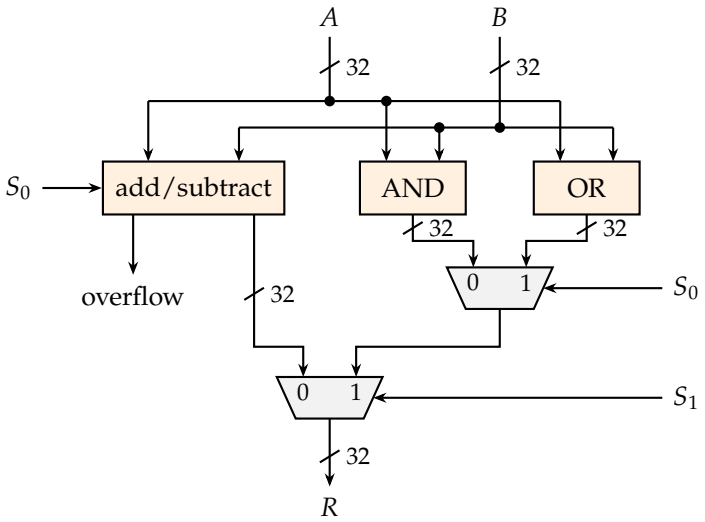


Figure 30: The simple ALU. S_1 chooses arithmetic (0) or logic (1); S_0 chooses add or subtract, and AND or OR.

17.3 Adder/subtractor

One could design an adder from its truth table, find the canonical form, minimize, and implement it. The better approach breaks the problem into smaller pieces that can be cascaded or layered hierarchically: one adder per bit position.

The least significant bit has only two inputs: $s_0 = a_0 \,\text{XOR}\, b_0$ and $c_1 = a_0 \,\text{AND}\, b_0$ ($a_0 b_0 = 00, 01, 10, 11$ give $s_0 = 0, 1, 1, 0$ and $c_1 = 0, 0, 0, 1$).

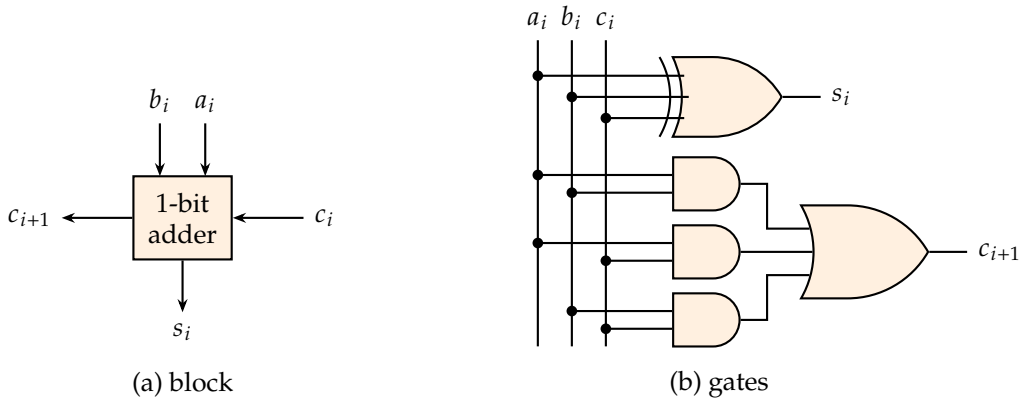
Every other position also adds the carry c_i from the position to its right:

a_i	0	0	0	0	1	1	1	1
b_i	0	0	1	1	0	0	1	1
c_i	0	1	0	1	0	1	0	1
s_i	0	1	1	0	1	0	0	1
c_{i+1}	0	0	0	1	0	1	1	1

so

$$s_i = \mathrm{XOR}(a_i, b_i, c_i), \qquad c_{i+1} = \mathrm{MAJ}(a_i, b_i, c_i) = a_i b_i + a_i c_i + b_i c_i$$

(Figure 31). Chaining n such 1-bit adders, each passing its carry out to the next, gives an n-bit adder with carry out c_n.



(a) block (b) gates

Figure 31: The 1-bit adder: (a) as a block, (b) as gates (a 3-input XOR and the majority circuit).

17.4 Overflow

When is the n-bit sum wrong? Consider all sums of two 2-bit numbers, with c_1 the carry into the highest adder (C_{in}) and c_2 its carry out (C_{out}); as signed (two's complement) numbers, $00, 01, 10, 11$ are $0, 1, -2, -1$:

$A + B$	$c_2 s_1 s_0$	c_1	signed sum	unsigned wrong	signed overflow
00 + 00	000	0	$0 + 0 = 0$		
00 + 01	001	0	$0 + 1 = 1$		
00 + 10	010	0	$0 + (-2) = -2$		
00 + 11	011	0	$0 + (-1) = -1$		
01 + 01	010	1	$1 + 1 = 2$		yes
01 + 10	011	0	$1 + (-2) = -1$		
01 + 11	100	1	$1 + (-1) = 0$	yes	
10 + 10	100	0	$(-2) + (-2) = -4$	yes	yes
10 + 11	101	0	$(-2) + (-1) = -3$	yes	yes
11 + 11	110	1	$(-1) + (-1) = -2$	yes	

For unsigned numbers the low two bits are wrong exactly when there is a carry out, so unsigned overflow is c_n. For signed numbers, look at the highest adder: with neither C_{in} nor C_{out}, or with both, there is no overflow; with C_{in} but no C_{out}, A and B are both positive and the sum overflows; with C_{out} but no C_{in}, A or B is -2 and the sum overflows. Overflow occurs exactly when the two carries differ:

$$\text{overflow} = c_n \text{ XOR } c_{n-1}.$$

17.5 Subtractor

Subtraction reuses the adder: $A - B = A + (-B)$. Since $\text{XOR}(x, 0) = x$ and $\text{XOR}(x, 1) = \overline{x}$, an XOR gate serves as a conditional inverter. In the adder/subtractor, each b_i passes through an XOR with the control signal SUB, which is also the carry in c_0 (Figure 32). With SUB = 0 the circuit adds $A + B$; with SUB = 1 it adds $A + \overline{B} + 1$, which is $A + (-B)$ in two's complement.

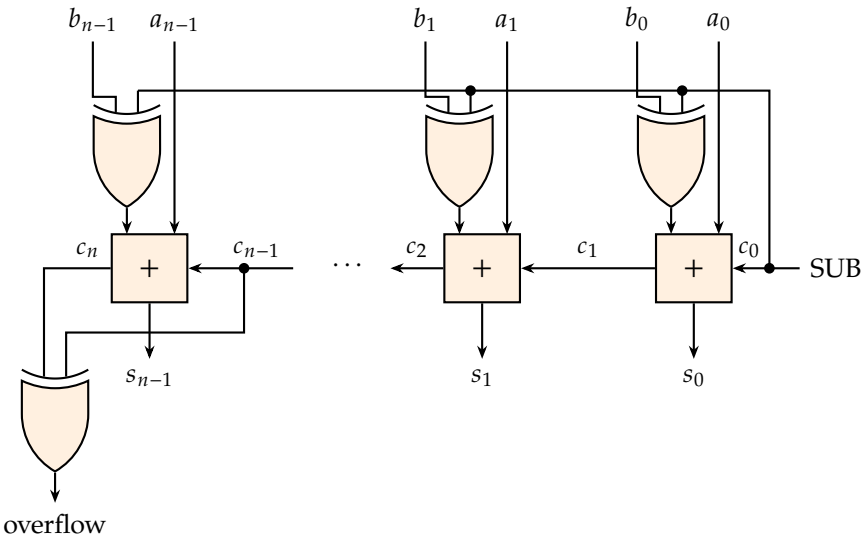


Figure 32: The n-bit adder/subtractor: n 1-bit adders with XOR gates on the b inputs, c_0 = SUB, and signed overflow c_n XOR c_{n-1}.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 17 (Fall 2020).

Lecture 18: Single-Cycle CPU Datapath I

Definition: The processor (CPU) is the active part of the computer that does all the work: data manipulation and decision making. Its datapath is the hardware that performs the operations the processor requires (the brawn), and its control, also hardware, tells the datapath what needs to be done (the brain).

The processor holds the program counter (PC), the registers, and the ALU; memory holds the program and its data as bytes, and the processor sends it an address, a read or write enable, and write data, and receives read data. The goal is a processor that implements every RV32I instruction except the environment instructions `ecall` and `ebreak`.

18.1 One instruction per cycle

In the simplest design, the computer executes one instruction on every tick of the clock. The outputs of the current state elements (PC, IMEM, register file, DMEM) drive the inputs of a block of combinational logic, whose outputs settle at the values of the next state before the next clock edge. At the rising edge, all state elements are updated with the combinational outputs and execution moves on to the next cycle.

A single monolithic block that performs everything needed to execute an instruction, starting with fetching it, would be bulky and inefficient. Instead, executing an instruction is broken into stages that are connected to form the whole datapath: smaller stages are easier to design, and one stage can be optimized without touching the others (modularity). The five stages are

1. Instruction fetch (IF),
2. Instruction decode (ID), including reading registers,
3. Execute (EX), using the arithmetic-logic unit (ALU),
4. Memory access (MEM),
5. Write back to a register (WB).

In the single-cycle machine, all five happen one after another within one clock cycle (Figure 33).

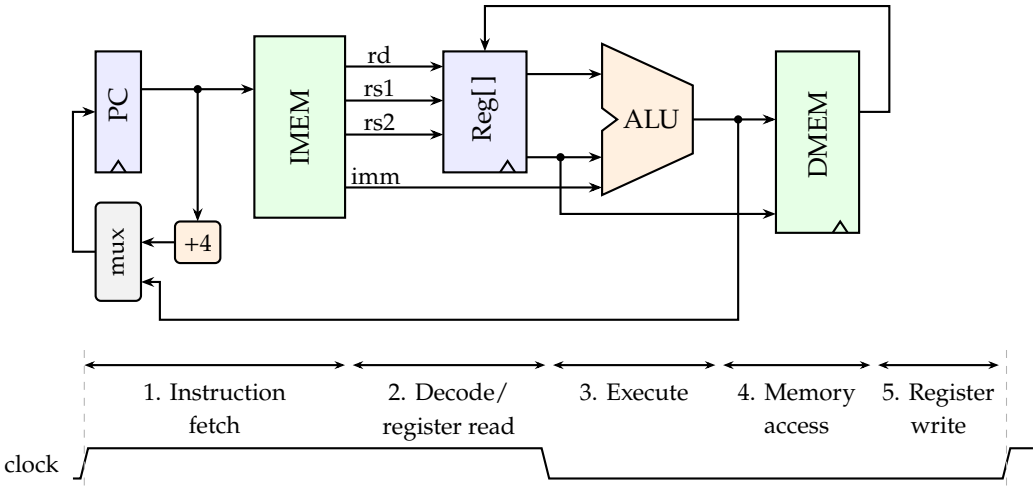


Figure 33: The phases of instruction execution in a single-cycle datapath. The PC addresses IMEM; the instruction's register fields select registers in `Reg[]`; the ALU combines register values or an immediate; the ALU result addresses DMEM; and a result is written back to `Reg[]`. The mux selects the next PC: PC + 4 or an ALU result. All five phases fit between two rising clock edges.

18.2 Datapath elements

The COMBINATIONAL ELEMENTS are 32-bit adders (inputs A, B, CarryIn; outputs Sum, CarryOut), multiplexers (inputs A, B, Select; output Y), and ALUs (inputs A, B, OP; output Result). The storage elements are clocked:

- A REGISTER of N bits has Data In, Data Out, Write Enable, and a clock. When Write Enable is low (deasserted, 0), Data Out does not change; when it is asserted (1), Data Out becomes Data In on the positive clock edge.
- The REGISTER FILE (regfile, RF) has 32 registers of 32 bits, two 32-bit output buses busA and busB, and one 32-bit input bus busW. The 5-bit register number RA selects the register put on busA, RB the one put on busB, and RW the register written from busW when Write Enable is 1. The clock matters only for writes; a read behaves as combinational logic: once RA or RB is valid, busA or busB is valid after an access time.
- The idealized ("magic") MEMORY has one input bus Data In, one output bus Data Out, an Address, and Write Enable. For a read, the address selects the word put on Data Out; for a write, with Write Enable set to 1, the address selects the word written from Data In. Again the clock matters only for writes, and a read is combinational: once the address is valid, Data Out is valid after the access time.

Every instruction reads and updates three kinds of RV32I state.

- *Registers* `x0`–`x31`: the register file `Reg` holds 32 registers of 32 bits, `Reg[0]` to `Reg[31]`. The first register read is specified by the instruction's `rs1` field, the second by `rs2`, and the destination

by `rd`. Register `x0` is always 0; writes to `Reg[0]` are ignored.

- The *program counter* (PC) holds the address of the current instruction.
- *Memory* holds both instructions and data in one 32-bit byte-addressed space. The datapath uses separate memories for instructions (`IMEM`) and data (`DMEM`), placeholders for the instruction and data caches. Instructions are fetched from `IMEM`, assumed read-only; loads and stores access `DMEM`.

18.3 R-type instructions: add and sub

R-format instructions have fields `funct7` (bits 31–25), `rs2` (24–20), `rs1` (19–15), `funct3` (14–12), `rd` (11–7), and `opcode` (6–0), of widths 7, 5, 5, 3, 5, 7. All of them have opcode `0110011` (OP-R) and differ only in `funct7` and `funct3`:

Instr.	`funct7`	`funct3`	Instr.	`funct7`	`funct3`
`add`	`0000000`	`000`	`xor`	`0000000`	`100`
`sub`	`0100000`	`000`	`srl`	`0000000`	`101`
`sll`	`0000000`	`001`	`sra`	`0100000`	`101`
`slt`	`0000000`	`010`	`or`	`0000000`	`110`
`sltu`	`0000000`	`011`	`and`	`0000000`	`111`

The instruction `add rd, rs1, rs2` makes two changes to the machine's state:

```
Reg[rd] = Reg[rs1] + Reg[rs2],    PC = PC + 4.
```

Its datapath is Figure 35 without the immediate generator and the mux in front of the ALU, so that DataB feeds the ALU directly. The PC addresses `IMEM`, and a dedicated adder computes PC + 4, which is loaded into the PC at the next clock edge. The instruction's fields drive the register file directly: `inst[11:7]` (`rd`) goes to AddrD, `inst[19:15]` (`rs1`) to AddrA, and `inst[24:20]` (`rs2`) to AddrB. The two register values DataA and DataB feed the ALU, whose output `alu` returns to DataD. The control logic decodes `inst[31:0]` and sets the register write enable RegWEn = 1.

Figure 34 follows the signals through two cycles. Shortly after a rising edge the PC holds 1000. After the adder delay PC + 4 is 1004, and after the `IMEM` access time `inst[31:0]` is `add x1,x2,x3`. The register file then outputs `Reg[2]` and `Reg[3]`, and after the ALU delay `alu` holds their sum, which waits at DataD with RegWEn = 1. Until the next rising edge, `Reg[1]` still holds its old value; at that edge it becomes `Reg[2]`+`Reg[3]` and the PC becomes 1004, and the same sequence repeats for `add x6,x7,x9`.

The instruction `sub rd, rs1, rs2` is almost the same as `add`, except that the ALU must subtract instead of add. The two encodings differ only in `inst[30]`, which selects between add and subtract; the control logic turns it into the ALU select signal ALUSel (add = 0, sub = 1). All other R-format

instructions are implemented the same way, by decoding `funct3` and `funct7` and selecting the appropriate ALU function.

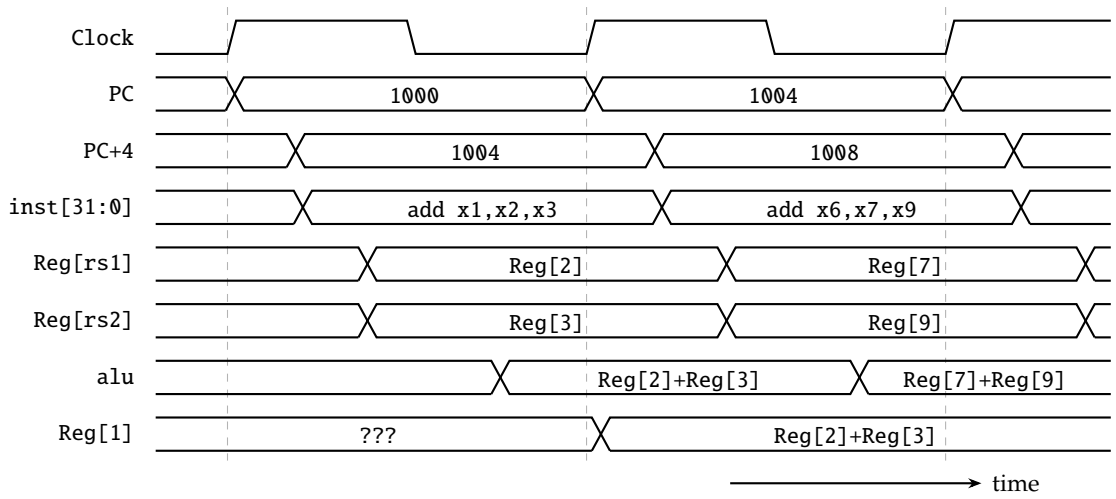


Figure 34: Timing of two consecutive instructions, `add x1,x2,x3` at address 1000 and `add x6,x7,x9` at 1004. Each signal settles some delay after the previous one in the chain; `Reg[1]` changes only at the next rising edge.

18.4 I-type arithmetic: addi

The I-format replaces `funct7` and `rs2` with a 12-bit immediate `imm[11:0]` in bits 31–20. For example, `addi x15,x1,-50` is encoded as

`imm[11:0]`	`rs1`	`funct3`	`rd`	`opcode`
`111111001110`	`00001`	`000`	`01111`	`0010011`
imm = −50	*rs1* = 1	add	*rd* = 15	OP-Imm

and performs `Reg[rd] = Reg[rs1] + Imm` and `PC = PC + 4`. The immediate must reach the ALU's second input, where the add/sub datapath has `Reg[rs2]`. Two pieces of hardware are added (Figure 35):

- an IMMEDIATE GENERATOR (Imm. Gen) that turns `inst[31:20]` into a 32-bit `Imm[31:0]`, under control of ImmSel = I;
- a mux in front of the ALU's second input, controlled by BSel, which selects `Reg[rs2]` (BSel = 0) or the immediate (BSel = 1).

For `addi`, the control sets ImmSel = I, RegWEn = 1, BSel = 1, and ALUSel to add.

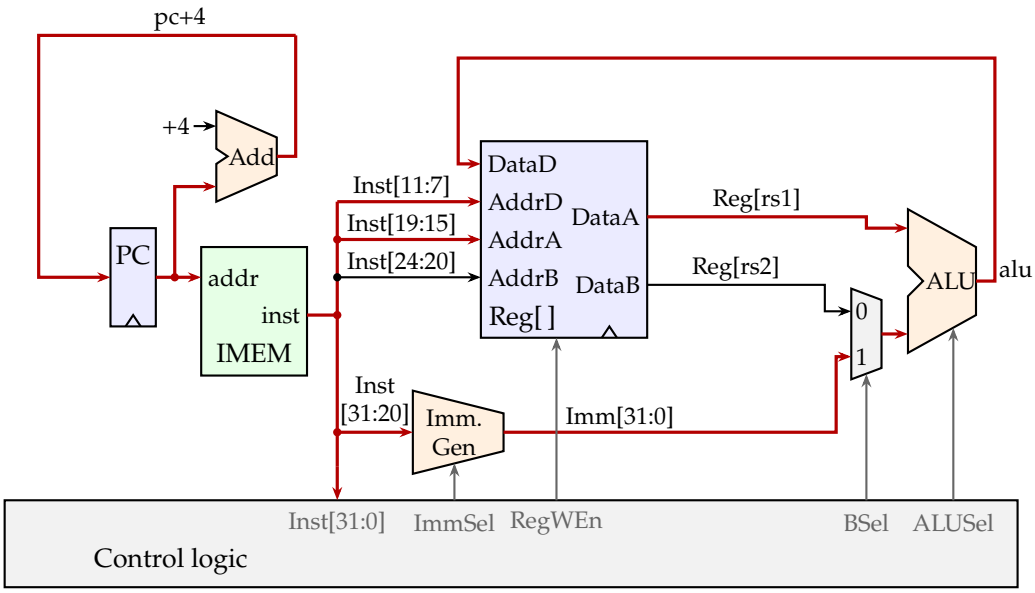


Figure 35: Datapath for R-type and I-type arithmetic instructions. The immediate generator and the BSel mux let the ALU's second operand be `Reg[rs2]` (input 0) or the immediate (input 1). Highlighted: the wires used by `addi`, which reads `rs1`, adds the immediate, and writes `rd`, while the PC advances by 4.

The I-format immediate is formed as follows. The high 12 bits of the instruction, `inst[31:20]`, are copied to the low 12 bits of the immediate, `imm[11:0]`, and the immediate is sign-extended by copying `inst[31]` into the upper 20 bits `imm[31:12]`. Equivalently, `imm[31:11]` are all copies of `inst[31]` and `imm[10:0] = inst[30:20]`.

The same datapath executes every other I-format arithmetic instruction (`slti`, `sltiu`, `andi`, `ori`, `xori`, `slli`, `srli`, `srai`) just by changing ALUSel.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 18 (Fall 2020).

Lecture 19: Single-Cycle CPU Datapath II

The datapath for R-type and I-type arithmetic instructions consists of the PC with its +4 adder, `IMEM`, the register file, the immediate generator, the BSel mux, and the ALU, with control signals ImmSel, RegWEn, BSel, and ALUSel. This unit extends it, one instruction class at a time, to all of RV32I.

19.1 Loads

The load `lw x14, 8(x2)` is an I-type instruction:

`imm[11:0]`	`rs1`	`funct3`	`rd`	`opcode`
offset	base	width	destination	LOAD
`000000001000`	`00010`	`010`	`01110`	`0000011`
imm = +8	*rs1* = 2	`lw`	*rd* = 14	LOAD

The 12-bit signed immediate is added to the base address in register `rs1` to form the memory address. This is very similar to `addi`, except that the sum is an address rather than the final result; the value loaded from memory is stored in register `rd`.

The datapath therefore gains a data memory `DMEM`, addressed by the ALU output, and a write-back mux selected by WBSel that chooses what reaches DataD: the ALU result `alu` (input 1) or the memory read data `mem` (input 0); `DMEM` is controlled by MemRW (Figure 36). For `lw` the control sets ImmSel = I, RegWEn = 1, BSel = 1, ALUSel = Add, MemRW = Read, and WBSel = 0.

All RV32I loads have opcode `0000011`; `funct3` encodes the size and signedness of the loaded data: `lb` (`000`), `lh` (`001`), `lw` (`010`), `lbu` (`100`), `lhu` (`101`). The narrower loads need additional logic to extract the correct byte or halfword from the value read from memory and to sign- or zero-extend it to 32 bits before writing it to the register file; this is just a mux and a few gates.

19.2 Stores

A store reads two registers, `rs1` for the base address and `rs2` for the data to be stored, as well as an immediate offset, which the S-format splits into two fields. For `sw x14, 8(x2)`:

`imm[11:5]`	`rs2`	`rs1`	`funct3`	`imm[4:0]`	`opcode`
offset[11:5]	source	base	width	offset[4:0]	STORE
`0000000`	`01110`	`00010`	`010`	`01000`	`0100011`
0	*rs2* = 14	*rs1* = 2	`sw`	8	STORE

The two pieces combine into the 12-bit offset `0000000 01000` = 8.

In the datapath, `Reg[rs2]` is wired to `DMEM`'s write-data input DataW. For `sw`: ImmSel = S, RegWEn = 0, BSel = 1, ALUSel = Add, MemRW = Write, and WBSel = ∗ (don't care, since nothing is written to a register).

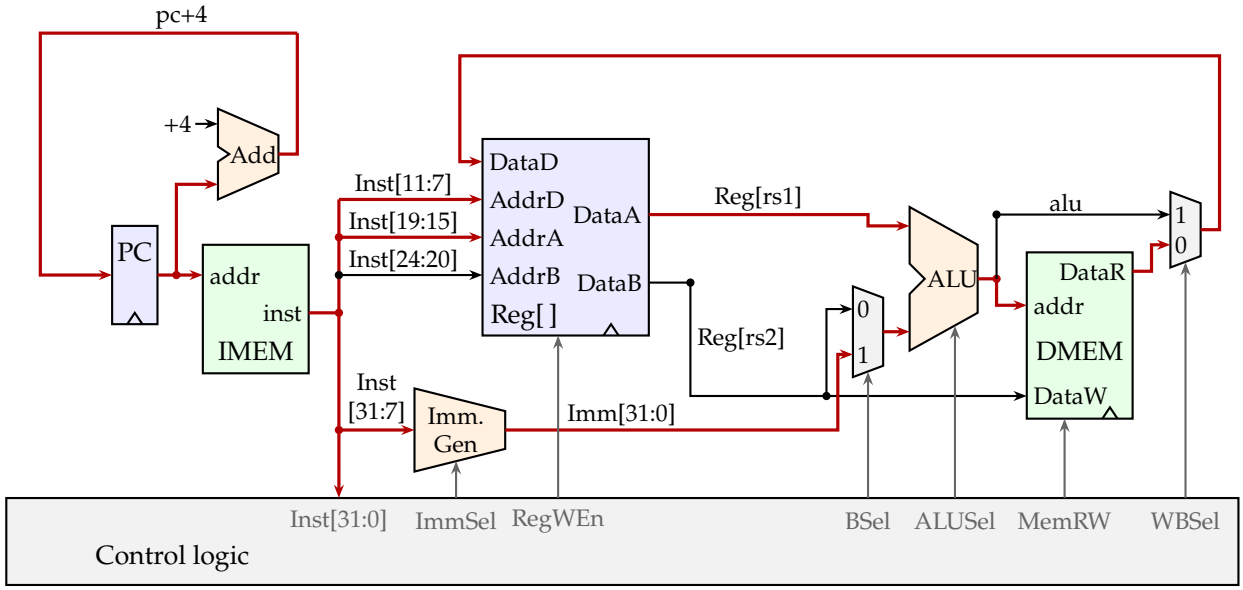


Figure 36: Datapath with loads and stores. `DMEM` is addressed by the ALU result and writes `Reg[rs2]` on a store; the WBSel mux writes back the ALU result (input 1) or the loaded data (input 0). Highlighted: the wires used by `lw`.

I and S immediates. Both formats keep the immediate's sign bit in `inst[31]` and bits 10–5 in `inst[30:25]`; only the low five bits move. So `imm[31:11]` are copies of `inst[31]` (sign extension), `imm[10:5]` = `inst[30:25]`, and `imm[4:0]` is `inst[24:20]` for the I-format or `inst[11:7]` for the S-format. The immediate generator needs only a 5-bit mux to choose between the two positions of the low five bits; all other immediate bits are wired to fixed positions in the instruction.

All stores have opcode `0100011`: `sb` (`funct3` `000`) writes the low byte to memory, `sh` (`001`) the lower two bytes, and `sw` (`010`) the whole word.

19.3 Branches

The B-format is mostly the same as the S-format, with two source registers and a 12-bit immediate `imm[12:1]`: `inst[31]` holds `imm[12]`, `inst[30:25]` hold `imm[10:5]`, `inst[11:8]` hold `imm[4:1]`, and `inst[7]` holds `imm[11]`. The immediate now represents even 13-bit signed byte offsets from −4096 to +4094 in 2-byte increments; the lowest bit of the offset is always zero, so it is not stored.

A branch changes the state differently from the instructions so far:

$$\texttt{PC} = \begin{cases} \texttt{PC + 4}, & \text{branch not taken,} \\ \texttt{PC + immediate}, & \text{branch taken.} \end{cases}$$

The six branches `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu` must both compute `PC + immediate` and compare the values of `rs1` and `rs2`, but there is only one ALU, so more hardware is needed:

- a BRANCH COMPARATOR that compares the two register values;
- an ASel mux in front of the ALU's first input, choosing `Reg[rs1]` (ASel = 0) or the PC (ASel = 1), so that the ALU can compute `PC + immediate`;
- a PCSel mux in front of the PC, choosing PC + 4 (input 0) or the ALU result (input 1) as the next PC.

Definition: The branch comparator has inputs A and B and outputs BrEq = 1 if $A = B$ and BrLT = 1 if $A < B$. The control input BrUn = 1 selects an unsigned comparison for BrLT, and BrUn = 0 a signed one.

A `bge` branch is taken when $A \geq B$, that is when $\overline{A < B} = \neg(A < B)$, so it uses the complement of BrLT. For a branch the control sets ImmSel = B, RegWEn = 0, ASel = 1, BSel = 1, ALUSel = Add, MemRW = Read, and WBSel = $*$, and sets PCSel to "taken" or "not taken" from BrEq and BrLT.

Branch immediates. The 12-bit immediate encodes a PC-relative offset from −4096 to +4094 bytes in multiples of 2 bytes. The standard approach in other ISAs treats the immediate as an S-type value in the range −2048 to +2047 and shifts it left by one bit to multiply by 2. Then each instruction bit can appear in one of two places of the output immediate, which needs one 2-way mux per bit. RISC-V instead keeps 11 immediate bits in a fixed position in the output and moves only the S-format's lowest bit, `inst[7]`, which becomes bit 11 of the B-immediate:

Immediate	`imm[31:12]`	`imm[11]`	`imm[10:5]`	`imm[4:1]`	`imm[0]`
I	`inst[31]`	`inst[31]`	`inst[30:25]`	`inst[24:21]`	`inst[20]`
S	`inst[31]`	`inst[31]`	`inst[30:25]`	`inst[11:8]`	`inst[7]`
B	`inst[31]`	`inst[7]`	`inst[30:25]`	`inst[11:8]`	`0`

The upper bits are always sign-extended from `inst[31]`, and only `inst[7]` changes role between S and B, so a single-bit 2-way mux suffices.

19.4 Jumps

jalr. `jalr rd, rs1, immediate` is I-format (`funct3` = 0, opcode JALR) and makes two changes to the state: it writes PC + 4 to `rd` (the return address) and sets `PC = rs1 + immediate`. It uses the

same immediates as arithmetic and loads, with no multiplication by 2 bytes; the least significant bit of the target is ignored. Writing PC + 4 to a register requires a third input, PC + 4 (input 2), on the write-back mux. For `jalr`: PCSel = taken (the ALU result), ImmSel = I, RegWEn = 1, BrUn, BrEq, BrLT = ∗, ASel = 0, BSel = 1, ALUSel = Add, MemRW = Read, WBSel = 2.

jal. The J-format has a 20-bit offset `imm[20:1]`, stored as `imm[20]` in `inst[31]`, `imm[10:1]` in `inst[30:21]`, `imm[11]` in `inst[20]`, and `imm[19:12]` in `inst[19:12]`, followed by `rd` and the opcode. `jal` saves PC + 4 in `rd` and sets `PC = PC + offset` (a PC-relative jump), so the target lies within $\pm 2^{19}$ locations, 2 bytes apart ($\pm 2^{18}$ 32-bit instructions). The immediate encoding is optimized like the branch encoding to reduce hardware cost: `imm[10:1]` sit in the same instruction bits as in the I-format, `imm[19:12]` keep their own positions, and the sign is again `inst[31]`. The datapath needs no new hardware: ImmSel = J, ASel = 1 (the PC), BSel = 1, ALUSel = Add, PCSel = taken, RegWEn = 1, MemRW = Read, WBSel = 2.

19.5 Upper immediates

The U-format has a 20-bit immediate in the upper 20 bits of the instruction (`inst[31:12]`), a destination register `rd`, and the opcode. It is used by two instructions: `lui` (load upper immediate) and `auipc` (add upper immediate to PC). The immediate generator places `inst[31:12]` in `imm[31:12]` and zeros in `imm[11:0]` (ImmSel = U). Both instructions use PCSel = PC + 4, RegWEn = 1, BSel = 1, MemRW = Read, and WBSel = 1 (the ALU result):

- `lui` passes the immediate through the ALU unchanged (ALUSel = B, ASel = ∗);
- `auipc` adds it to the PC (ASel = 1, ALUSel = Add).

19.6 The complete datapath

Figure 37 shows the complete RV32I datapath. It executes every RISC-V instruction in one cycle each, but not all units are used by every instruction. The five phases of execution (IF, ID, EX, MEM, WB) remain, and not all instructions are active in all phases. The controller, still to be designed, sets the control signals that specify how each instruction executes.

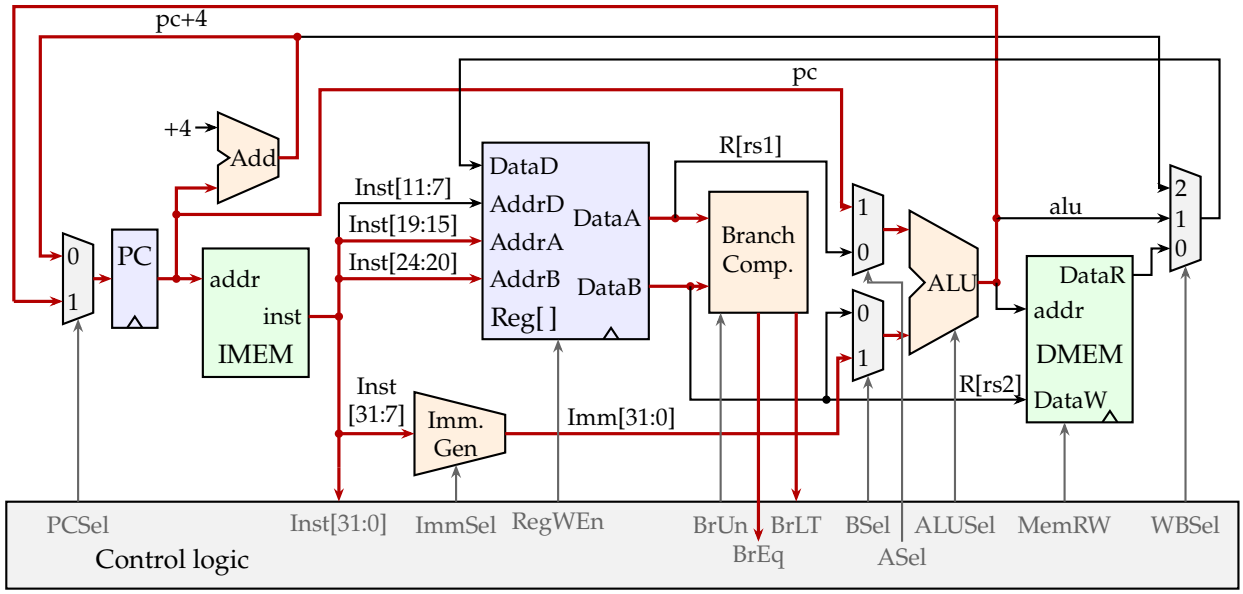


Figure 37: The complete single-cycle RV32I datapath and its control signals. Mux inputs: PCSel chooses PC + 4 (0) or the ALU result (1); ASel chooses `R[rs1]` (0) or the PC (1); BSel chooses `R[rs2]` (0) or the immediate (1); WBSel writes back memory data (0), the ALU result (1), or PC + 4 (2). Triangles mark clocked inputs. Highlighted: the wires used by a branch, which compares `R[rs1]` with `R[rs2]` and computes PC + immediate in the ALU, while PCSel picks the next PC.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 19 (Fall 2020).

Lecture 20: Single-Cycle CPU Control

The complete single-cycle RV32I datapath (Figure 38) executes any RV32I instruction in one clock cycle once the control logic sets its control signals: PCSel, ImmSel, RegWEn, BrUn, ASel, BSel, ALUSel, MemRW, and WBSel, with the branch comparator's outputs BrEq and BrLT and the instruction `inst[31:0]` as inputs.

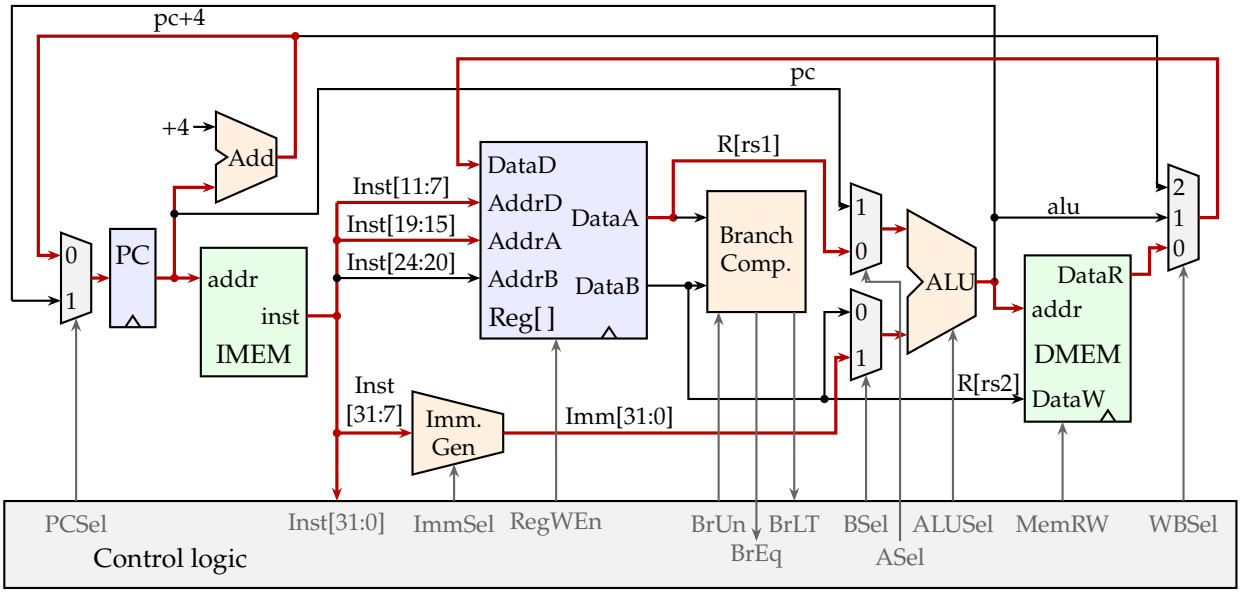


Figure 38: The single-cycle RV32I datapath with its control logic. Highlighted: the wires used by `lw`, from the PC through `IMEM`, the register file and the immediate generator, the ALU, and `DMEM` back to DataD, while PC + 4 becomes the next PC.

20.1 Control and status registers

Definition: Control and status registers (CSRs) are separate from the register file (`x0`–`x31`). They are used for monitoring status and performance, and there can be up to 4096 of them.

CSRs are not in the base ISA, since the ISA is modular, but they are almost mandatory in every implementation: they are necessary for counters and timers and for communication with peripherals.

CSR instructions have opcode SYSTEM (`1110011`): the 12-bit field `csr` in `inst[31:20]` names the CSR (source and destination), `funct3` selects the instruction, `rd` is the integer destination, and `inst[19:15]` holds either the source register `rs1` or a 5-bit immediate `uimm[4:0]`, zero-extended to 32 bits. Whether the CSR is read or written depends on the operands:

Instruction	`rd`	`rs1` or `uimm`	Read CSR?	Write CSR?
`csrrw`	`x0`	–	no	yes
`csrrw`	not `x0`	–	yes	yes
`csrrs`, `csrrc`	–	`x0`	yes	no
`csrrs`, `csrrc`	–	not `x0`	yes	yes
`csrrwi`	`x0`	–	no	yes
`csrrwi`	not `x0`	–	yes	yes
`csrrsi`, `csrrci`	–	0	yes	no
`csrrsi`, `csrrci`	–	not 0	yes	yes

Example: `csrrw` (atomic read/write CSR) atomically swaps the values in a CSR and an integer register: it reads the previous value of the CSR and writes it to `rd`, then writes `rs1` to the CSR. The pseudoinstruction `csrw csr, rs1` is `csrrw x0, csr, rs1`: with `rd = x0` it just writes `rs1` to the CSR. Likewise `csrwi csr, uimm` is `csrrwi x0, csr, uimm`, which just writes `uimm` to the CSR.

The remaining system instructions are `ecall` (I-format), which makes requests such as system calls to the supporting execution environment (the operating system), and `ebreak` (I-format), used for example by debuggers to transfer control to a debugging environment. Both have opcode SYSTEM, `rs1` = 0, `rd` = 0, and `funct3` = PRIV, and are distinguished by the 12-bit `funct12` field (ECALL or EBREAK). Finally, `fence` sequences memory (and I/O) accesses as viewed by other threads or co-processors.

20.2 Datapath control examples

Each instruction lights up a different subset of the datapath. For `sw`, the control sets PCSel = PC + 4, ImmSel = S, RegWEn = 0, BrUn, BrEq, BrLT = $*$, ASel = 0, BSel = 1, ALUSel = Add, MemRW = Write, WBSel = $*$. For `beq`: ImmSel = B, RegWEn = 0, ASel = 1, BSel = 1, ALUSel = Add, MemRW = Read, WBSel = $*$, and PCSel follows BrEq. For `add`: PCSel = PC + 4, ImmSel = $*$, RegWEn = 1, ASel = 0, BSel = 0, ALUSel = Add, MemRW = Read, WBSel = 1.

20.3 Instruction timing

During an `add`, the clock edge loads the PC; PC + 4 and the instruction then settle, followed by the control word for `add` (decoded from the instruction) and, in parallel, the register values `Reg[rs1]` and `Reg[rs2]`; then the ALU result, then the write-back value at DataD. The destination register changes only at the next rising edge. The longest of the paths from the PC to a state element input sets the minimum clock period. For `add`,

$$\begin{aligned}\text{critical path} &= t_{\text{clk-q}} + \max\{t_{\text{Add}} + t_{\text{mux}},\ t_{\text{IMEM}} + t_{\text{Reg}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{mux}}\} + t_{\text{setup}} \\ &= t_{\text{clk-q}} + t_{\text{IMEM}} + t_{\text{Reg}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{mux}} + t_{\text{setup}},\end{aligned}$$

where the first term inside the maximum is the PC + 4 path and the second goes through the register file, the BSel mux, the ALU, and the write-back mux. For `lw` the memory read adds to the paths, and the immediate path also competes:

$$t_{\text{clk-q}} + \max\left\{\begin{array}{l} t_{\text{Add}} + t_{\text{mux}}, \\ t_{\text{IMEM}} + t_{\text{Imm}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{DMEM}} + t_{\text{mux}}, \\ t_{\text{IMEM}} + t_{\text{Reg}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{DMEM}} + t_{\text{mux}} \end{array}\right\} + t_{\text{setup}}.$$

Within one cycle the phases follow each other (Figure 39). With the stage delays below, the slowest instruction, `lw`, takes 800 ps.

	IF	ID	EX	MEM	WB	Total
Unit	I-MEM	Reg read	ALU	D-MEM	Reg write	
Delay	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps

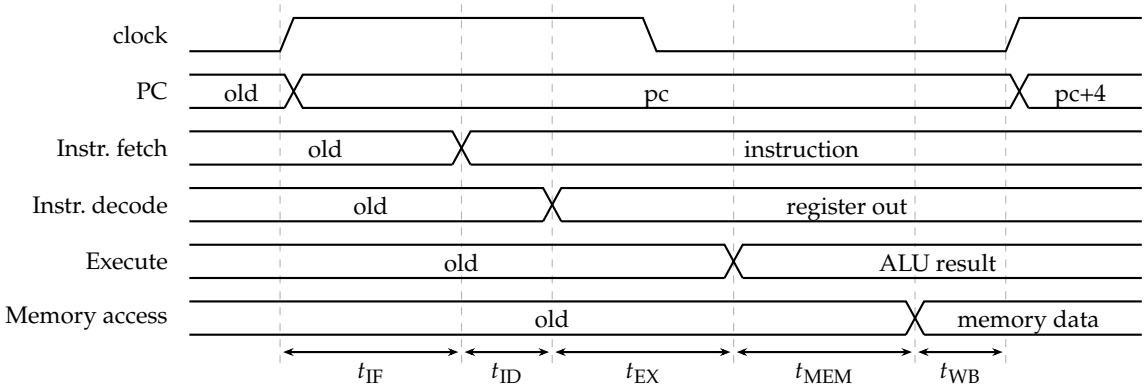


Figure 39: Instruction timing within one clock cycle. After the clock edge the PC changes; instruction fetch, decode (register read), execute, and memory access complete one after another, and the result is written back at the next rising edge. Stage widths follow the delays in the table.

Not every instruction uses every phase:

Instr.	IF (200 ps)	ID (100 ps)	ALU (200 ps)	MEM (200 ps)	WB (100 ps)	Total
`add`	X	X	X		X	600 ps
`beq`	X	X	X			500 ps
`jal`	X	X	X		X	600 ps
`lw`	X	X	X	X	X	800 ps
`sw`	X	X	X	X		700 ps

Since the clock period must accommodate the slowest instruction, the maximum clock frequency is $f_{\text{max}} = 1/800\,\text{ps} = 1.25\,\text{GHz}$. Most blocks are idle most of the time: the ALU alone could run at $f_{\text{max,ALU}} = 1/200\,\text{ps} = 5\,\text{GHz}$.

20.4 Control logic design

The control logic is specified by a truth table from the instruction and the branch comparison to the control signals (∗ means don't care):

`inst[31:0]`	BrEq	BrLT	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWEn	WBSel
`add`	∗	∗	+4	∗	∗	Reg	Reg	Add	Read	1	ALU
`sub`	∗	∗	+4	∗	∗	Reg	Reg	Sub	Read	1	ALU
(R-R op)	∗	∗	+4	∗	∗	Reg	Reg	(op)	Read	1	ALU
`addi`	∗	∗	+4	I	∗	Reg	Imm	Add	Read	1	ALU
`lw`	∗	∗	+4	I	∗	Reg	Imm	Add	Read	1	Mem
`sw`	∗	∗	+4	S	∗	Reg	Imm	Add	Write	0	∗
`beq`	0	∗	+4	B	∗	PC	Imm	Add	Read	0	∗
`beq`	1	∗	ALU	B	∗	PC	Imm	Add	Read	0	∗
`bne`	0	∗	ALU	B	∗	PC	Imm	Add	Read	0	∗
`bne`	1	∗	+4	B	∗	PC	Imm	Add	Read	0	∗
`blt`	∗	1	ALU	B	0	PC	Imm	Add	Read	0	∗
`bltu`	∗	1	ALU	B	1	PC	Imm	Add	Read	0	∗
`jalr`	∗	∗	ALU	I	∗	Reg	Imm	Add	Read	1	PC + 4
`jal`	∗	∗	ALU	J	∗	PC	Imm	Add	Read	1	PC + 4
`auipc`	∗	∗	+4	U	∗	PC	Imm	Add	Read	1	ALU

The control can be realized in two ways.

- A ROM (read-only memory) has a regular structure and can be easily reprogrammed to fix errors or add instructions; it is popular when control logic is designed manually.
- Combinational logic: today chip designers use logic synthesis tools to convert truth tables into networks of gates.

RV32I is effectively a nine-bit ISA for control purposes: the instruction type is encoded by only nine bits, `inst[30]`, `inst[14:12]`, and `inst[6:2]`. A ROM-based controller therefore has an 11-bit address, `inst[30,14:12,6:2]` together with BrEq and BrLT, and 15 data bits: PCSel, ImmSel[2:0], BrUn, ASel, BSel, ALUSel[3:0], MemRW, RegWEn, and WBSel[1:0]. Internally the ROM is an address decoder (an AND plane) that asserts one line per instruction (`add`, `sub`, `or`, . . . , `jal`), and each line selects that instruction's stored control word (an OR plane), which drives the controller outputs.

In combinational control each signal is a logic function of those bits. The simplest example is BrUn: among the branches, `inst[13]` (the middle bit of `funct3`) is 1 exactly for the unsigned ones, so

$$\text{BrUn} = \texttt{inst[13]} \cdot \text{Branch}.$$

Decoding `add` (opcode `0110011`, `funct3` `000`, `inst[30]` = 0), with i standing for `inst`:

$$\begin{aligned}
\text{add} &= \overline{i[30]} \cdot \overline{i[14]} \cdot \overline{i[13]} \cdot \overline{i[12]} \cdot \text{Rtype}, \\
\text{Rtype} &= \overline{i[6]} \cdot i[5] \cdot i[4] \cdot \overline{i[3]} \cdot \overline{i[2]} \cdot \text{RV32I}, \\
\text{RV32I} &= i[1] \cdot i[0].
\end{aligned}$$

With control in place the processor is complete: it executes every RISC-V instruction in one cycle each, not all units are used by all instructions, and the critical path changes from instruction to instruction. This closes the gap between the levels of abstraction, from a C program through assembly and machine code down to the datapath block diagram and its logic circuits.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 20 (Fall 2020).

Lecture 21: Pipelining I

21.1 Instruction timing in the single-cycle datapath

In the single-cycle RISC-V processor every instruction completes in one clock cycle, so the cycle must be long enough for the slowest instruction. An instruction passes through five phases, each implemented by different hardware: instruction fetch (IF, reading IMEM), instruction decode and register read (ID), execute (EX, the ALU), memory access (MEM, reading or writing DMEM), and write back (WB, writing the register file). After the rising clock edge loads a new PC, the signals settle one phase after another (Figure 40): the instruction is valid after t_{IF}, the register outputs after t_{ID}, the ALU result after t_{EX}, and the memory data after t_{MEM}; the result must then be written back before the next edge, which loads $pc + 4$.

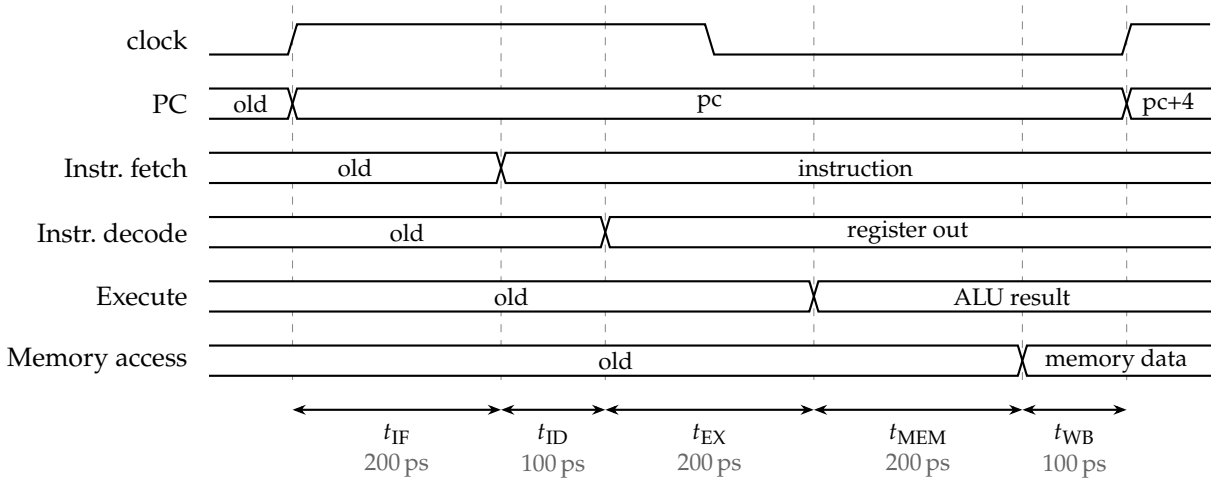


Figure 40: Signal timing of a load in the single-cycle datapath. Each signal holds its old value until the preceding phases have completed; the next rising clock edge, 800 ps later, loads $pc + 4$.

Not every instruction uses every phase. With phase delays IF 200 ps, ID 100 ps, EX 200 ps, MEM 200 ps, and WB 100 ps:

Instr.	IF	ID	EX	MEM	WB	Total
`add`	✓	✓	✓		✓	600 ps
`beq`	✓	✓	✓			500 ps
`jal`	✓	✓	✓		✓	600 ps
`lw`	✓	✓	✓	✓	✓	800 ps
`sw`	✓	✓	✓	✓		700 ps

The load is the slowest instruction, so the maximum clock frequency is $f_{\text{max}} = 1/800\,\text{ps} = 1.25\,\text{GHz}$:

one instruction every 800 ps.

21.2 Performance measures

"Improving performance" can mean different things: a quicker response time (one job finishes faster), more jobs per unit time (a web server returning pages, spoken words recognized), or longer battery life. A transportation analogy separates them. For a 50-mile trip (assuming an instantaneous return):

	Sports car	Bus
Passenger capacity	2	50
Travel speed	200 mph	50 mph
Gas mileage	5 mpg	2 mpg
Travel time	15 min	60 min
Time for 100 passengers	750 min (50 two-person trips)	120 min (two 50-person trips)
Gallons per passenger	5 gallons	0.5 gallons

Trip time corresponds to PROGRAM EXECUTION TIME (for example, the time to update a display); time for 100 passengers to THROUGHPUT (for example, server requests handled per hour); and gallons per passenger to ENERGY PER TASK (how many movies one battery charge plays, or a datacenter's energy bill). Power is not a good measure: a low-power CPU may run for a long time to complete a task and consume more energy than a faster computer running at higher power for a shorter time.

21.3 The iron law of processor performance

Definition: The IRON LAW of processor performance factors execution time as

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}},$$

where the middle factor is the average number of CYCLES PER INSTRUCTION (CPI).

- *Instructions per program* are determined by the task, the algorithm ($O(N^2)$ versus $O(N)$), the programming language, the compiler, and the ISA.
- *CPI* is determined by the ISA and the processor implementation (MICROARCHITECTURE): CPI = 1 for the single-cycle RISC-V design, CPI $\gg$ 1 for complex instructions (such as `strcpy`), and CPI $<$ 1 for superscalar processors.
- *Time per cycle* (1/frequency) is determined by the microarchitecture (which sets the critical path through the logic gates), the technology (5 nm versus 28 nm), and the power budget (lower voltages reduce transistor speed).

Example: For one task (say, image compression):

	Processor A	Processor B
Instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate f	2.5 GHz	2 GHz
Execution time	1 ms	0.75 ms

($10^6 \times 2.5/2.5\,\text{GHz} = 1\,\text{ms}$ and $1.5 \times 10^6 \times 1/2\,\text{GHz} = 0.75\,\text{ms}$.) Processor B is faster for this task despite executing more instructions and having a slower clock.

21.4 Energy efficiency

In CMOS logic, such as an inverter driving a capacitive load, about 70% of the energy goes into charging capacitors (proportional to CV^2) and about 30% is leakage. The energy analogue of the iron law is

$$\frac{\text{Energy}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Energy}}{\text{Instruction}}, \qquad \frac{\text{Energy}}{\text{Program}} \propto \frac{\text{Instructions}}{\text{Program}} \times C\,V^2,$$

where the "capacitance" C depends on the technology and on processor features (such as the number of cores) and V is the supply voltage (for example 1 V). Reducing capacitance and voltage reduces energy per task.

Example: A next-generation processor whose capacitance (thanks to Moore's law) and supply voltage are each 15% lower uses energy $0.85 \times 0.85^2 = 0.85^3 \approx 0.61$ times the old value, a change of $0 - (1 - 0.85^3) = -39\%$. The improvement comes from both Moore's law and the reduced supply voltage.

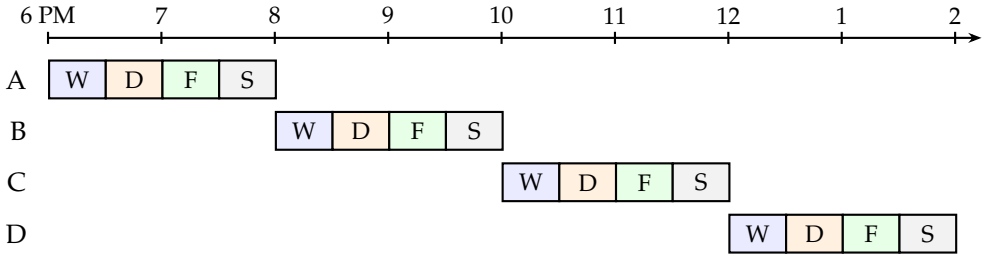
In 48 years of microprocessor trend data (1970–2020), transistor counts have kept growing, single-thread performance has grown ever more slowly, clock frequency and typical power have leveled off since the mid-2000s, and the number of logical cores has grown. The scaling that drove the improvement is ending. Industry has not been able to reduce the supply voltage much further, because that would increase LEAKAGE POWER (transistor switches that do not fully turn off, more like a dimmer than an on-off switch), and transistor size, and hence capacitance, is shrinking less between generations than before (the next step is to go to 3D). Power is a growing concern: the POWER WALL. The energy version of the iron law is

$$\text{Performance (tasks/second)} = \text{Power (joules/second)} \times \text{Energy efficiency (tasks/joule)}.$$

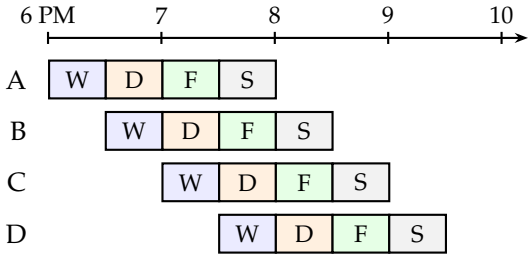
Energy efficiency (for example, instructions per joule) is the key metric in all computing devices. Power-constrained systems (a 20 MW datacenter) need better energy efficiency to get more performance at the same power; energy-constrained systems (a 1 W phone) need it to prolong battery life.

21.5 Pipelining: the laundry analogy

Four people, A, B, C, and D, each have one load of clothes to wash, dry, fold, and put away; the washer, the dryer, the "folder," and the "stasher" each take 30 minutes. Done sequentially, each load finishes before the next starts, and the four loads take 8 hours (Figure 41a). Pipelined, a load enters the washer as soon as the previous one moves to the dryer, and the four loads take 3.5 hours (Figure 41b): each load still passes through its steps sequentially, but different loads use different resources simultaneously.



(a) sequential: 8 hours for 4 loads



(b) pipelined: 3.5 hours for 4 loads

Figure 41: Four loads of laundry, each with four 30-minute steps (W wash, D dry, F fold, S stash), done sequentially and pipelined.

- Pipelining does not help the *latency* of a single task; it helps the *throughput* of the entire workload.
- Multiple tasks operate simultaneously using different resources.
- The potential speedup equals the number of pipeline stages, but the time to "fill" the pipeline and the time to "drain" it reduce the speedup: here it is $8/3.5 \approx 2.3\times$ instead of $4\times$.
- The pipeline rate is limited by the slowest stage. If a new washer and a new stasher took only 20 minutes, the pipeline would still finish one load every 30 minutes, as the dryer and folder still take that long: unbalanced stage lengths reduce the speedup.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 21 (Fall 2020).

Lecture 22: Pipelining II

22.1 Pipelining the RISC-V datapath

The single-cycle datapath executes each instruction's five phases one after another, so an instruction takes the sum of the phase delays: 200 ps for instruction fetch, 100 ps for register read, 200 ps for the ALU, 200 ps for memory, and 100 ps for register write, 800 ps in all. A pipelined datapath gives every phase the same length, one clock cycle of 200 ps (the register read and write phases are stretched from 100 ps), and starts a new instruction every cycle.

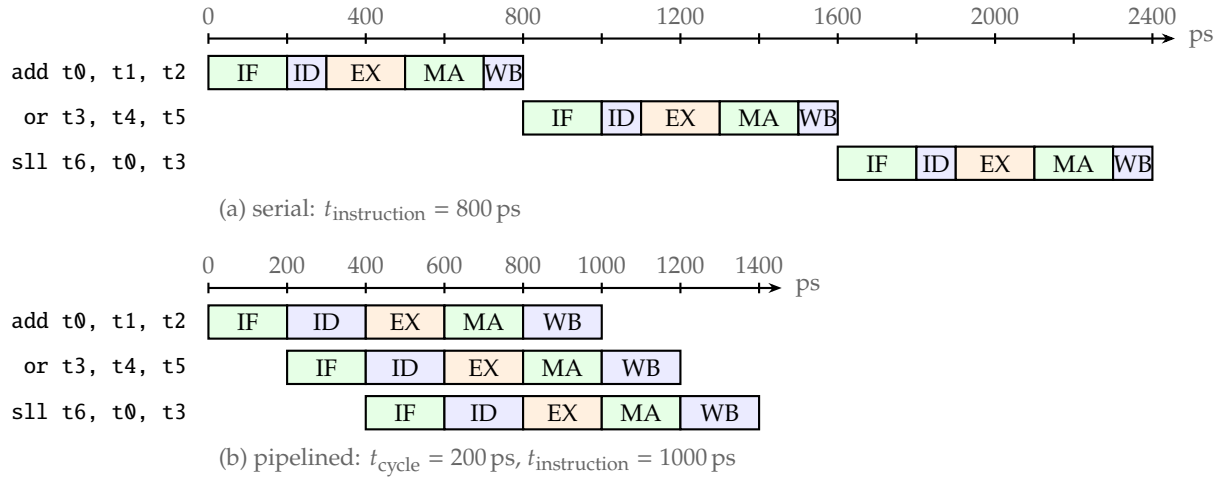


Figure 42: Three instructions, to scale. (a) Serial execution: each instruction takes 800 ps and the next starts when it finishes. (b) Pipelined execution: every stage takes $t_{\text{cycle}} = 200$ ps and a new instruction starts every cycle.

Each instruction now takes longer (1000 ps instead of 800 ps), but one instruction completes every cycle (Figure 42):

	Single cycle	Pipelined
Timing	$t_{\text{step}} = 100 \ldots 200$ ps (register access only 100 ps)	$t_{\text{cycle}} = 200$ ps (all cycles the same length)
Instruction time $t_{\text{instruction}}$	$t_{\text{cycle}} = 800$ ps	1000 ps
CPI	≈1 (ideal)	≈1 (ideal)
Clock rate f_s	1/800 ps = 1.25 GHz	1/200 ps = 5 GHz
Relative speed	1×	4×

Pipelining thus raises the clock rate, not the per-instruction latency. Each instruction still uses the resources sequentially (a row of a pipeline diagram shows one instruction's resource use over time),

while in any one cycle all five stages work simultaneously on five different instructions (a column shows the resource use in one time slot).

22.2 The pipelined datapath

The single-cycle RV32I datapath divides naturally into the five stages: INSTRUCTION FETCH (IF), INSTRUCTION DECODE and register read (ID), ALU EXECUTE (EX), MEMORY ACCESS (MA), and WRITE BACK (WB). PIPELINE REGISTERS between the stages, named IF/ID, ID/EX, EX/MA, and MA/WB after the stages they separate, hold the data of each instruction in flight (Figure 43). For example, while `or t3,t4,t5` is in MA, the next three instructions `slt t6,t0,t3`, `sw t0,4(t3)`, and `lw t0,8(t3)` can be in EX, ID, and IF.

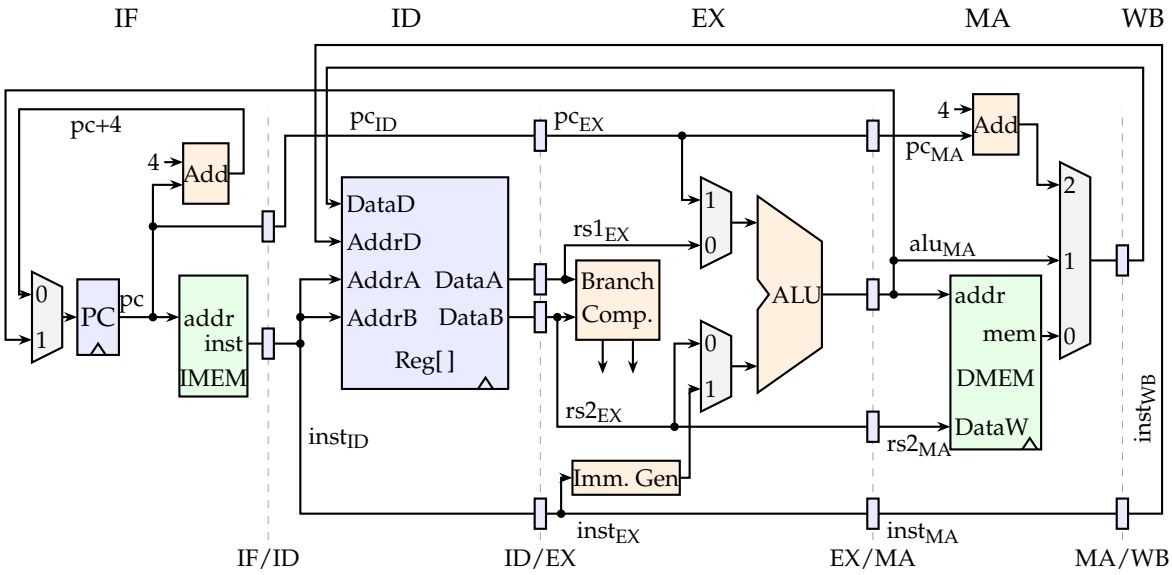


Figure 43: Pipelined RV32I datapath (control signals not shown). Small boxes on the dashed lines are the pipeline registers; a subscript names the stage in which a value is used. The outputs of Branch Comp. go to the control logic, and inst_{WB} supplies the destination register number to AddrD.

Two details of the design:

- *PC+4 is recalculated in the MA stage* (from pc_{MA}), which avoids sending both PC and PC+4 down the pipeline.
- *The instruction is pipelined along with the data* (inst_{ID}, inst_{EX}, inst_{MA}, inst_{WB}), so that control operates correctly in each stage; for example, the register written in WB is the destination of the instruction in WB, not of the one being decoded.

Definition: In PIPELINED CONTROL, control signals are derived from the instruction as in the single-cycle implementation, and stored in the pipeline registers for use by later stages: the ID/EX register holds the signals for EX, MA, and WB; the EX/MA register those for MA and WB; and the MA/WB

register those for WB.

22.3 Hazards

Definition: A HAZARD is a situation that prevents starting the next instruction in the next clock cycle. A STRUCTURAL HAZARD occurs when a required resource is busy (for example, needed in multiple stages); a DATA HAZARD when there is a data dependency between instructions, so one must wait for a previous instruction to complete its data read or write; and a CONTROL HAZARD when the flow of execution depends on a previous instruction.

22.4 Structural hazards

In a structural hazard two or more instructions in the pipeline compete for access to a single physical resource. One solution is for instructions to take turns using the resource, so some of them have to stall; the other is to add more hardware, which can always resolve a structural hazard.

Register file. Each instruction can read up to two operands in the decode stage and write one value in the write-back stage. Two independent read ports and one independent write port avoid a structural hazard: the three accesses can happen simultaneously in one cycle.

Memory. A load or store accesses data memory in its MA stage while the third instruction after it is being fetched (Figure 44).

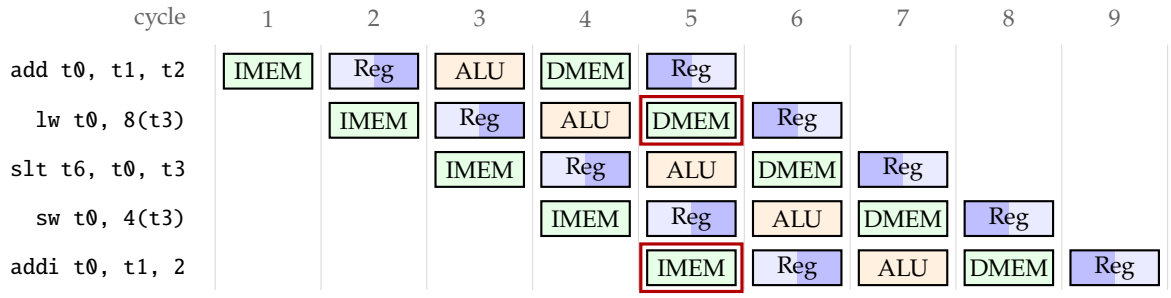


Figure 44: Pipeline diagram. In cycle 5 the load accesses data memory while the fifth instruction is fetched from instruction memory. In a Reg box, the shaded half marks a write (first half of the cycle, in WB) or a read (second half, in ID).

With a single memory, the instruction fetch would have to stall for that cycle, and all other operations in the pipeline would have to wait. Pipelined datapaths therefore require separate instruction and data memories, or separate instruction and data caches: fast on-chip memories, one for instructions and one for data, between the processor and the single main memory that holds both program and data. RISC ISAs, including RISC-V, are designed to avoid structural hazards, for example by allowing at most one memory access per instruction.

22.5 Data hazards

Register access. The register file has separate ports, but an instruction may write a register in WB in the same cycle in which a later instruction reads it in ID. In Figure 45, `add` writes `t0` in cycle 5 while `sw` reads `t0` in cycle 5.

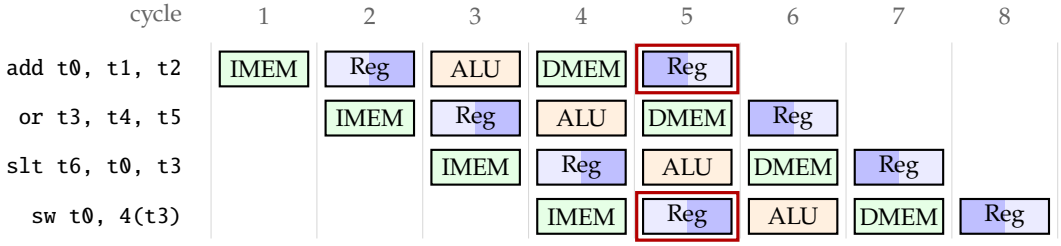


Figure 45: `add` writes `t0` in the first half of cycle 5 and `sw` reads it in the second half.

The register file is fast (100 ps), so the cycle can be split: WB updates the value in the first half of the cycle and ID reads the new value in the second half. Thus `sw` gets the new value of `t0`. Writing and then reading in the same cycle may not always be possible, especially in high-frequency designs, so this assumption must be checked in any given design.

ALU result. In the sequence of Figure 46, `add` produces `s0` in EX (cycle 3) but writes it to the register file only in WB (cycle 5), so the register holds the old value 5 through cycle 4 and the new value 9 from the middle of cycle 5. `sub` and `or` read `s0` in cycles 3 and 4, so without some fix they calculate the wrong result; `xor` (cycle 5) and `sw` (cycle 6) read the new value.

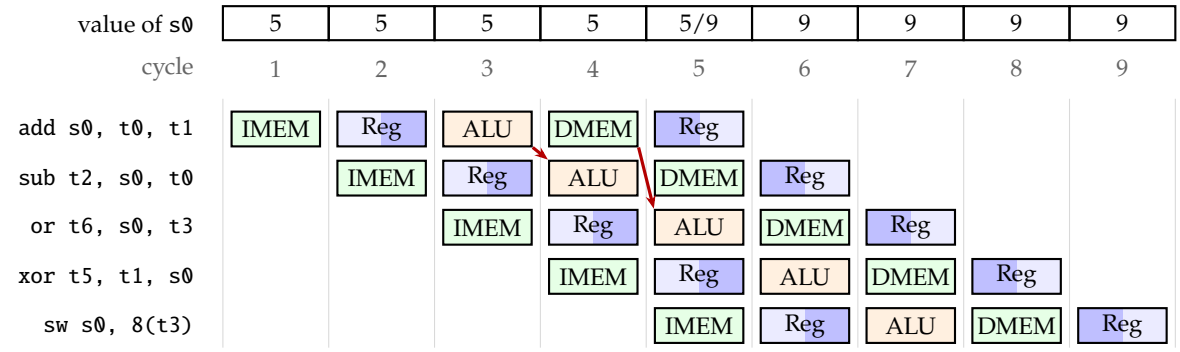


Figure 46: Data hazard on `s0`. The top row is the value of `s0` in the register file. The red arrows are the forwarding paths: from the EX/MA register (end of `add`'s EX) to `sub`'s ALU input, and from the MA/WB register (end of `add`'s MA) to `or`'s ALU input.

Solution 1: stalling. If an instruction depends on the result of the previous one, as in `add s0,t0,t1` followed by `sub t2,s0,t3`, the pipeline can delay `sub` by two cycles, so that it reads `s0` in ID in the same cycle in which `add` writes it in WB. The empty slots are BUBBLES: effectively `nop`s, during which the affected pipeline stages do nothing. Stalls reduce performance, but they are required to get correct results. The compiler can also arrange the code, or insert `nop`s (`addi x0, x0, 0`), to

avoid hazards and stalls; this requires knowledge of the pipeline structure.

Solution 2: forwarding. FORWARDING (also called BYPASSING) grabs an operand from a pipeline stage rather than from the register file: a result is used as soon as it is computed, without waiting for it to be stored in a register. This requires extra connections in the datapath. In Figure 46, `sub` takes `s0` from the EX/MA register and `or` from the MA/WB register.

To decide when to forward, the hardware compares the destination registers of the older instructions in the pipeline with the source registers of the new instruction in the decode stage, and must ignore writes to `x0` (which always reads as zero). For `add t0,t0,t1` followed by `sub t3,t0,t5`, while `add` is in EX and `sub` is in ID, the *rd* field of the instruction in EX (inst_X.*rd*) matches the *rs1* field of the instruction in decode (inst_D.*rs1*), so `sub`'s first operand is forwarded in the next cycle. In the datapath, forwarding control logic drives the operand multiplexers, which gain inputs from alu_{MA} and the write-back value (Figure 47); operand B must be forwarded as well.

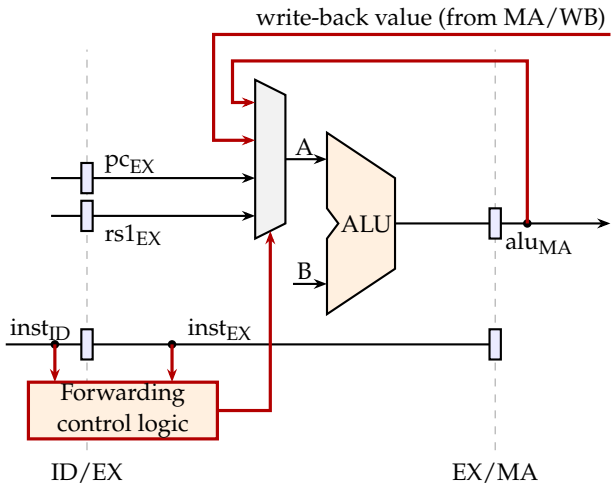


Figure 47: EX stage with forwarding for operand A (additions in red). Operand B, whose multiplexer is not shown, receives the same forwarded inputs.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 22 (Fall 2020).

Lecture 23: Pipelining III

23.1 Load data hazards

Forwarding removes the stalls after ALU instructions, but not after a load. A load's data comes from memory only at the end of its MA stage, while an instruction that immediately follows and uses it needs the value at the start of its EX stage, in the same cycle. In

```
lw s2,20(s1);  and s4,s2,s5;  or s8,s2,s6;  add s9,s4,s2;  slt s1,s6,s7
```

the loaded `s2` would have to travel backward in time to reach `and`, so a one-cycle stall is unavoidable. The `or` after it can receive `s2` by forwarding from the MA/WB register, and the later instructions are unaffected: `add` reads `s2` from the register file in the cycle in which `lw` writes it.

The pipeline is stalled by converting `and` into a `nop` and repeating it one cycle later, when the loaded value can be forwarded to it (Figure 48).

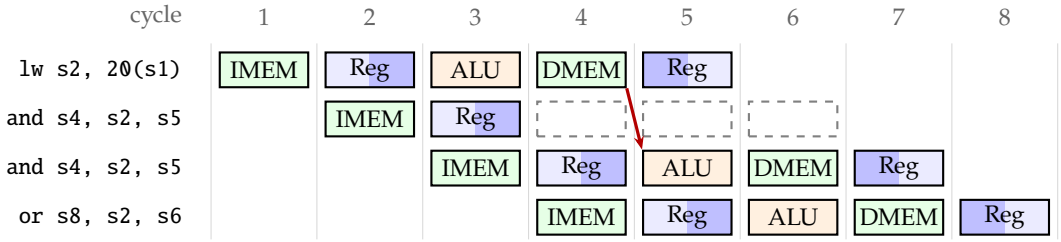


Figure 48: A load requires a one-cycle pipeline stall. The first `and` becomes a `nop` (dashed boxes are bubbles); the repeated `and` receives `s2` forwarded from the MA/WB register (red), and `or` reads `s2` in cycle 5, when `lw` writes it.

Definition: The slot after a load is called the LOAD DELAY SLOT. If the instruction in it uses the result of the load, the hardware stalls for one cycle. This is equivalent to inserting an explicit `nop` in the slot, except that the latter uses more code space; either way performance is lost.

The remedy is to put an unrelated instruction into the load delay slot, which costs no performance: the code is reordered so that a load result is not used by the next instruction.

Example: RISC-V code for `A[3]=A[0]+A[1]; A[4]=A[0]+A[2]` (with the address of `A` in `t0`). In the original order each `add` uses the register loaded by the instruction just before it, so it stalls twice and takes 9 cycles (counting one cycle per instruction and per stall). Moving the third load up removes both stalls, and the code takes 7 cycles.

Original order (9 cycles)

```
lw   t1, 0(t0)
lw   t2, 4(t0)
add  t3, t1, t2
sw   t3, 12(t0)
lw   t4, 8(t0)
add  t5, t1, t4
sw   t5, 16(t0)
```

Reordered (7 cycles)

```
lw   t1, 0(t0)
lw   t2, 4(t0)
lw   t4, 8(t0)
add  t3, t1, t2
sw   t3, 12(t0)
add  t5, t1, t4
sw   t5, 16(t0)
```

23.2 Control hazards

A branch such as `beq t0,t1,Label` computes its outcome in the EX stage, so the PC is updated only at the end of the branch's EX stage. By then the two instructions after the branch have already been fetched, and they would be executed regardless of the branch outcome: two stall cycles after a branch.

If the branch is not taken, the instructions fetched sequentially after it are the correct ones. If a branch or jump is taken, the incorrect instructions must be FLUSHED from the pipeline by converting them to nops (Figure 49).

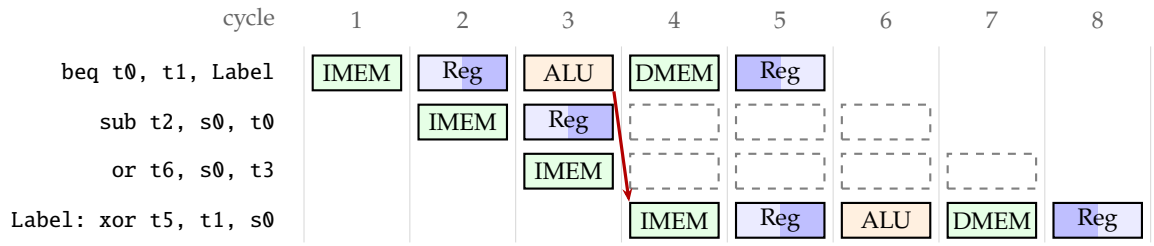


Figure 49: Killing instructions after a taken branch. `beq` resolves in EX (cycle 3); `sub` and `or` are converted to nops (dashed bubbles), and the updated PC fetches the target `xor` in cycle 4 (red).

Every taken branch in this simple pipeline therefore costs two dead cycles. To improve performance, BRANCH PREDICTION guesses which way the branch will go earlier in the pipeline: the guessed next PC is fetched in the cycle right after the branch, the guess is checked when the branch resolves in EX, and the pipeline is flushed only if the prediction was incorrect.

23.3 Superscalar processors

Processor performance can be increased in three ways.

1. *Clock rate*, which is limited by technology and power dissipation.
2. *Pipelining*, which overlaps instruction execution. A deeper pipeline (5 → 10 → 15 stages) has

less work per stage and so a shorter clock cycle, but more potential for hazards (CPI > 1).

3. *Multi-issue "superscalar" processors.*

Definition: A SUPERSCALAR (multiple-issue) processor replicates pipeline stages to form multiple pipelines and starts multiple instructions per clock cycle. Since then CPI < 1, performance is quoted in INSTRUCTIONS PER CYCLE (IPC). In OUT-OF-ORDER EXECUTION the hardware reorders instructions dynamically to reduce the impact of hazards.

A 4 GHz, 4-way multiple-issue processor executes at most 16 billion instructions per second: peak CPI = 0.25, peak IPC = 4. Dependencies reduce this in practice. In a typical organization, an instruction fetch and decode unit issues instructions in order to reservation stations, which feed the functional units (integer, floating point, load-store) that execute out of order, and a commit unit commits the results in order.

Measured CPI on an Intel i7 shows the gap between peak and practice. On each of twelve benchmarks the ideal CPI is 0.25, and stalls and misspeculation add the rest:

Benchmark	libquantum	h264ref	hmmer	perlbench	bzip2	xalancbmk
CPI	0.44	0.59	0.61	0.65	0.74	0.77
Benchmark	sjeng	gobmk	astar	gcc	omnetpp	mcf
CPI	0.82	1.02	1.06	1.23	2.12	2.67

The iron law also shows how to obtain CPI: the time per program can be timed, the instructions per program can be counted, and the time per cycle can be looked up, so

$$\text{CPI} = \frac{\text{Cycles}}{\text{Instruction}} = \frac{\text{Time}}{\text{Program}} \div \left(\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Time}}{\text{Cycle}}\right).$$

23.4 Pipelining and ISA design

The RISC-V ISA is designed for pipelining. All instructions are 32 bits, so they are easy to fetch and decode in one cycle (x86 instructions are 1 to 15 bytes). There are few and regular instruction formats, so the processor can decode and read registers in one step. Load/store addressing lets the address be calculated in the third stage and memory be accessed in the fourth. Memory operands are aligned, so a memory access takes only one cycle.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 23 (Fall 2020).

Lecture 24: Caches I

24.1 Binary prefixes

The common prefixes kilo, mega, giga, tera, peta, exa, zetta, and yotta are SI prefixes (except that SI writes kilo as lowercase k) and mean powers of 10. Common usage is confusing: a "kilobyte" usually means 1024 bytes, while the correct SI value is 1000 bytes. Hard disk manufacturers and telecommunications are the only computing groups that use the SI factors, so a drive advertised as 1 TB holds about 90% of what one expects ($10^{12}/2^{40} \approx 0.91$), and a 1 Mbit/s connection transfers 10^6 bits per second.

To remove the ambiguity, the International Electrotechnical Commission (IEC) introduced binary prefixes in 1999 (officially only up to exbi). Their names shorten the SI names and add "bi" for binary (pronounced "bee"). The SI prefixes now have only their base-10 meaning.

SI name	Abbr.	SI size	IEC name	Abbr.	Binary factor
kilo	K	10^3	kibi	Ki	$2^{10} = 1{,}024$
mega	M	10^6	mebi	Mi	$2^{20} = 1{,}048{,}576$
giga	G	10^9	gibi	Gi	$2^{30} = 1{,}073{,}741{,}824$
tera	T	10^{12}	tebi	Ti	$2^{40} = 1{,}099{,}511{,}627{,}776$
peta	P	10^{15}	pebi	Pi	$2^{50} = 1{,}125{,}899{,}906{,}842{,}624$
exa	E	10^{18}	exbi	Ei	$2^{60} = 1{,}152{,}921{,}504{,}606{,}846{,}976$
zetta	Z	10^{21}	zebi	Zi	$2^{70} = 1{,}180{,}591{,}620{,}717{,}411{,}303{,}424$
yotta	Y	10^{24}	yobi	Yi	$2^{80} = 1{,}208{,}925{,}819{,}614{,}629{,}174{,}706{,}176$

To convert quickly, write a power of two as 2^{XY} with tens digit X and units digit Y. The digit X selects the prefix ($X = 0$: none, 1: kibi $\approx 10^3$, 2: mebi $\approx 10^6$, 3: gibi $\approx 10^9$, 4: tebi, 5: pebi, 6: exbi, 7: zebi, 8: yobi $\approx 10^{24}$), and 2^Y is the multiplier ($1, 2, 4, \ldots, 512$ for $Y = 0, \ldots, 9$).

Example: $2^{34} = 2^4 \cdot 2^{30} = 16\,\text{Gi}$. Addressing $2.5\,\text{TiB} = 2.5 \cdot 2^{40}$ bytes needs $\lceil \lg(2.5 \cdot 2^{40}) \rceil = 42$ bits, since $2^{41} = 2\,\text{TiB} < 2.5\,\text{TiB} \leq 2^{42} = 4\,\text{TiB}$.

24.2 Why memory is slow

A computer has a processor (control plus a datapath containing the PC, the registers, and the ALU), memory, and input and output devices. The processor-memory interface carries an enable signal, a read/write signal, an address, write data, and read data; separate I/O-memory interfaces connect the devices.

Large memories are slow, as in a large library: finding a book requires searching a large card

catalog (mapping title or author to an index number) and a round trip to the stacks, and a larger library makes both delays longer. Electronic memories have the same problem, and in addition the technologies that store bits more densely are slower (SRAM, then DRAM, then disk). What we want is a memory that is both large and fast.

The gap keeps growing. CPU performance improved about 55% per year from 1980 (slowing after 2004) while DRAM improved about 7% per year. A 1980 microprocessor executed about one instruction in the time of one DRAM access; a 2020 microprocessor executes about 1000 instructions in that time, so slow DRAM access has a disastrous impact on CPU performance.

The library suggests the fix. To write a report on the works of J. D. Salinger, fetch the relevant books from the stacks and keep them on a desk, checking out more as needed without returning the earlier ones, since they may be needed again. About ten books, only 0.00001% of the books in the UC Berkeley libraries, are hopefully enough for the report.

24.3 Memory caching and the hierarchy

Definition: A MEMORY CACHE is a level added between the processor and main memory to bridge their speed mismatch. It is built with the same integrated-circuit technology as the CPU (usually on the same chip), so it is faster but more expensive per bit than DRAM. A cache is a copy of a subset of main memory. Most processors have separate caches for instructions and data.

The MEMORY HIERARCHY (Figure 50) extends this idea. A level closer to the processor is smaller, faster, more expensive per byte, and holds a subset of the lower levels (the most recently used data). The lowest level, usually a disk (HDD or SSD), contains all available data. The hierarchy is INCLUSIVE: what is in the L1 cache is a subset of what is in main memory, which is a subset of what is in secondary memory. Data moves between adjacent levels in units that grow with distance from the processor.

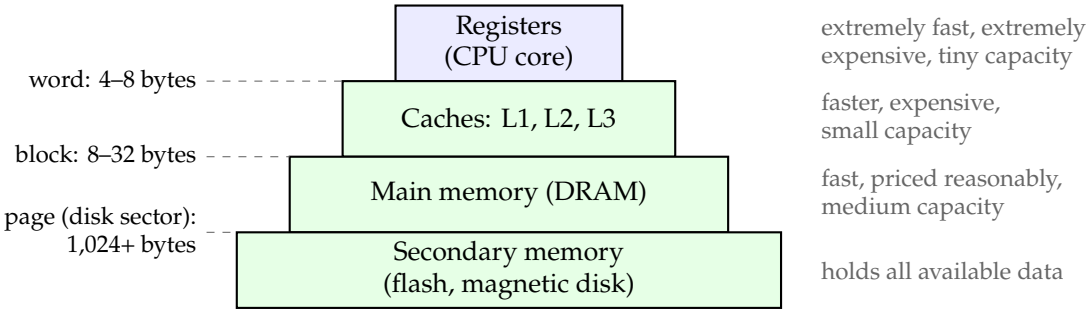


Figure 50: The memory hierarchy. Levels lower in the figure are farther from the processor in access time and larger; on the left is the unit moved between adjacent levels.

The trick is to present the processor with as much memory as is available in the cheapest technology, at the speed offered by the fastest technology. Typical orders of magnitude:

	Registers	L1 caches	Lower caches	DRAM	Disk or flash
Speed (cycles)	fractions	ones	tens	hundreds	ten thousands
Size (bytes)	hundreds	tens of K	M	G	T
Cost	highest				lowest

Jim Gray's analogy rescales access latency to human terms: a register (1 ns) is like data in your head (1 minute away), the on-chip cache (2 ns) like data in this room (2 minutes), and memory (100 ns) like data in Sacramento (1.5 hours).

The hierarchy thus presents the processor with the illusion of a very large and very fast memory.

24.4 Locality

The cache contains copies of the data in memory that are being used, and memory contains copies of the data on disk that are being used. Caches work because of two kinds of locality:

- Temporal locality (locality in time): a memory location that is referenced tends to be referenced again soon. So keep the most recently accessed data items closer to the processor.
- Spatial locality (locality in space): when a location is referenced, locations with nearby addresses tend to be referenced soon. So move blocks of contiguous words closer to the processor.

Designing a cache means answering: how is it organized; where does each memory address map (since the cache is a subset of memory, many addresses map to the same cache location); how do we know which elements are in the cache; and how do we locate them quickly.

Each boundary of the hierarchy is managed by a different agent: registers and memory by the compiler (or assembly programmer); cache and main memory by the cache controller hardware; main memory and disk by the operating system (virtual memory, with virtual-to-physical address translation assisted by hardware in the translation lookaside buffer, or TLB, itself a kind of cache) and by the programmer (files).

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 24 (Fall 2020).

Lecture 25: Caches II

25.1 Direct-mapped caches

Definition: In a direct-mapped cache each memory address is associated with exactly one possible block within the cache, so only a single cache location needs to be checked for the data. The block is the unit of transfer between the cache and memory.

In a 4-byte direct-mapped cache with 1-byte blocks, memory location a maps to cache location $a \bmod 4$: cache location 0 can be occupied by data from memory locations $0, 4, 8, \ldots$, any multiple of 4.

With larger blocks, a request for one byte makes the cache controller find the right block and load all of it. Take an 8-byte direct-mapped cache with 2-byte blocks (four blocks), and draw memory with the same width as a block (Figure 51). Consecutive blocks map to consecutive cache rows, wrapping around every four blocks. Three questions arise: which row a byte maps to, which byte of the block it is, and, since many memory blocks share a row, which of them is currently there. As at baggage claim, each row carries a tag that identifies its occupant. The tag need not be the whole address: cutting memory into cache-sized pieces and numbering the pieces $0, 1, 2, \ldots$ (the "cache number"), the tag is the piece number. As with branch immediates, which do not store a low bit that is always zero, the tag omits address bits that are the same for every block that can occupy the row.

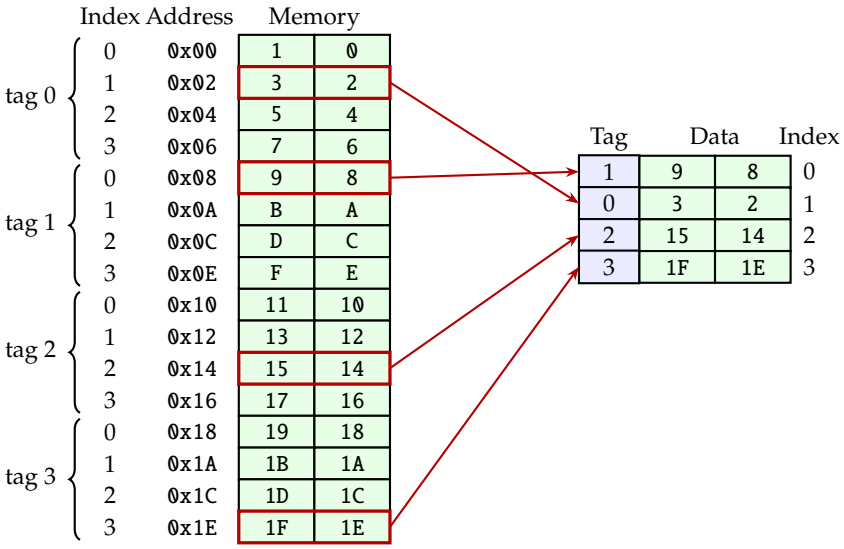


Figure 51: An 8-byte direct-mapped cache with 2-byte blocks. Memory is drawn one block wide, each cell labeled with its byte address in hex. Each block's index is its row number modulo 4, and its tag is the number of the cache-sized piece of memory containing it. The cache holds the blocks at `0x08`, `0x02`, `0x14`, and `0x1E`, with tags 1, 0, 2, and 3.

Example: Byte address 11101_2 = `0x1D` splits as tag $11_2 = 3$, index $10_2 = 2$, offset 1: it is the upper byte of the block at `0x1C`, which maps to row 2 with tag 3.

25.2 Tag, index, and offset

The answer to all three questions is to divide the address into three fields, read as unsigned integers:

- the OFFSET, which, once the correct block is found, specifies which byte within the block we want;
- the INDEX, which specifies the cache index (the row, or block, of the cache to look in);
- the TAG, the remaining bits after the offset and index are determined, which distinguish among all the memory addresses that map to the same row.

Picture the cache as a rectangle (Figure 52): its area (cache size in bytes) is its height (number of blocks) times its width (block size in bytes), so $2^{I+O} = 2^I \cdot 2^O$. Hence

$$O = \lg(\text{block size}), \qquad I = \lg \frac{\text{cache size}}{\text{block size}}, \qquad T = (\text{address bits}) - I - O.$$

To solve cache problems, draw memory one block wide, cut into pieces the size of the cache. The tag says which piece an address lies in, the index which row of that piece, and the offset which byte of the row. Address $0\cdots0\ 0\cdots0\ 0\cdots0$ (tag, index, offset) is the first byte of piece 0, $0\cdots01\ 0\cdots0\ 0\cdots0$ the first byte of piece 1, and $1\cdots1\ 1\cdots1\ 1\cdots1$ the last byte of the last piece.

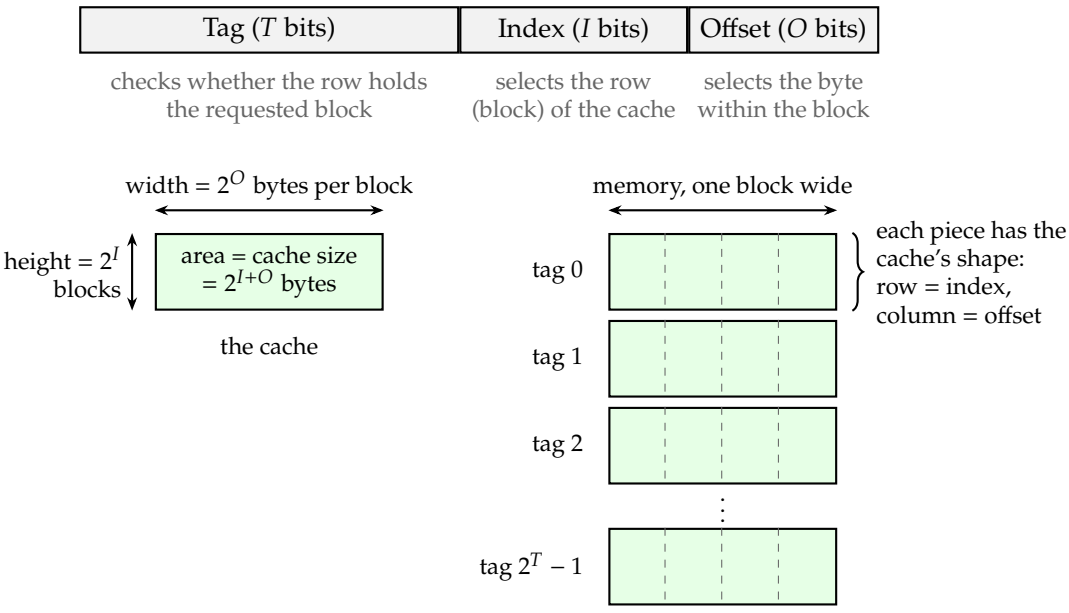


Figure 52: The tag, index, and offset fields; the cache as a rectangle of 2^I blocks of 2^O bytes; and memory drawn one block wide as a stack of cache-sized pieces, numbered by tag.

Example: For 8 B of data in a direct-mapped cache with 2-byte blocks on a 32-bit architecture (RV32): a block holds $2 = 2^1$ bytes, so the offset needs 1 bit; the cache holds $2^3/2^1 = 2^2$ blocks, so the index needs 2 bits; and the tag is the remaining $32 - 1 - 2 = 29$ bits, the leftmost 29 bits of the address. The tag can be thought of as the cache number. A full 32-bit tag is unnecessary: all bytes of a block share the same address bits above the offset, and the index is the same for every address that maps to the row, so it is redundant in the tag check and can be left off to save memory.

25.3 Memory access with a cache

Consider `lw t0, 0(t1)` with `t1` = 1022 and `Memory[1022]` = 99. Without a cache:

1. the processor issues address 1022 to memory;
2. memory reads the word at address 1022 (99);
3. memory sends 99 to the processor;
4. the processor loads 99 into register `t0`.

With a cache, which behaves much like a hash table:

1. the processor issues address 1022 to the cache;
2. the cache checks whether it has a copy of the data at address 1022:
   (a) if it finds a match (a hit), it reads 99 and sends it to the processor;
   (b) if not (a miss), it sends address 1022 to memory; memory reads 99 and sends it to the cache; the cache replaces a word with the new 99 and sends 99 to the processor;
3. the processor loads 99 into register `t0`.

25.4 Terminology and the valid bit

When reading memory, three things can happen:

- a CACHE HIT: the cache block is valid and contains the proper address, so the desired word is read;
- a CACHE MISS: nothing is in the appropriate cache block, so the data is fetched from memory;
- a CACHE MISS WITH BLOCK REPLACEMENT: the wrong data is in the appropriate block, so it is discarded and the desired data is fetched from memory (the cache always holds copies).

A cache is *cold* when empty, *warming* while it fills with values that will hopefully be accessed again soon, *warm* when it is doing its job with a fair percentage of hits, and *hot* when it is doing very well with a high percentage of hits.

Definition: The HIT RATE is the fraction of accesses that hit in the cache, and the MISS RATE is 1 − hit rate. The MISS PENALTY is the time to replace a block from a lower level of the memory hierarchy into the cache. The HIT TIME is the time to access the cache memory, including the tag comparison. "$" abbreviates "cache" (a Berkeley innovation).

When a new program starts, the cache does not yet hold valid information for it, so each tag entry needs an indicator of whether it is valid. The VALID BIT is added to each tag entry: if it is 0, the access is a miss even if by chance the address matches the tag; if it is 1, the access is a hit when the processor's address matches the tag. When the computer is powered up, all entries are invalid.

Example: A 16 KB direct-mapped cache with 16 B blocks has rows indexed 0 to 1023. Each row holds a valid bit, a tag, and four 4-byte words, which hold the bytes at offsets `0xc`–`f`, `0x8`–`b`, `0x4`–`7`, and `0x0`–`3` of the block (written in that order, highest first). Initially every valid bit is 0.

A direct-mapped cache is thus a mechanism for transparently moving data among levels of the memory hierarchy: it keeps a set of address/value bindings, uses the address to index a set of candidates, compares the desired address with the stored tag, and services a hit or a miss, loading the new block and binding on a miss.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 25 (Fall 2020).

Lecture 26: Caches III

26.1 Reading from a direct-mapped cache

A read from a direct-mapped cache proceeds as follows:

1. Divide the address into tag, index, and offset bits; go to the row given by the index and check its valid bit.
2. If the valid bit is 0, the access misses: load the block from memory, set the valid bit and the tag, and use the offset to return the right chunk (1, 2, or 4 bytes).
3. If the valid bit is 1, compare the stored tag with the address tag. On a match the access hits and the offset selects the chunk to return. Otherwise the access misses: load the block (replacing the old one), set the tag, and use the offset to return the chunk.

Example: Take 16 KB of data in a direct-mapped cache with 4-word (16-byte) blocks and 32-bit addresses. The width is $16 = 2^4$ bytes, so the offset has 4 bits; the height is $2^{14}/2^4 = 1024$ blocks, so the index has 10 bits; the tag has $32 - 10 - 4 = 18$ bits. Memory holds the words a, b, c, d at `0x00000010`, `0x14`, `0x18`, `0x1C`; e, f, g, h at `0x00000030`–`0x3C`; and i, j, k, l at `0x00008010`–`0x801C`. Starting from an empty cache (all valid bits 0), read four addresses, then two more:

Address	Tag	Index	Offset	Outcome	Value
`0x00000014`	`00000000000000000`	`0000000001`	`0100`	miss, row 1 invalid	b
`0x0000001C`	`00000000000000000`	`0000000001`	`1100`	hit	d
`0x00000034`	`00000000000000000`	`0000000011`	`0100`	miss, row 3 invalid	f
`0x00008014`	`00000000000000010`	`0000000001`	`0100`	miss, tag $0 \neq 2$	j
`0x00000030`	`00000000000000000`	`0000000011`	`0000`	hit	e
`0x0000001C`	`00000000000000000`	`0000000001`	`1100`	miss, tag $2 \neq 0$	d

After the first four reads the cache holds:

Index	Valid	Tag	`0xc`–`f`	`0x8`–`b`	`0x4`–`7`	`0x0`–`3`
0	0					
1	1	2	l	k	j	i
2	0					
3	1	0	h	g	f	e
4–1023	0					

The first read finds row 1 invalid, so it loads the block `0x10`–`0x1F` (d, c, b, a from offset `0xc` down to `0x0`), sets the tag to 0 and the valid bit to 1, and returns the word at offset `0x4`, b. The second

read finds row 1 valid with a matching tag and returns the word at offset `0xc`, d. The third read loads h, g, f, e into row 3 and returns f. The fourth also indexes row 1, which is valid, but its tag 0 differs from 2, so the block is replaced by l, k, j, i with tag 2 and j is returned. Now `0x00000030` hits (index 3, tag matches, offset 0) and returns e, while `0x0000001C` misses (index 1, tag mismatch), so row 1 is refilled from memory and the word at offset `0xc`, d, is returned. Since these are reads, the values returned must equal the memory values whether or not they are cached.

26.2 Multiword blocks in hardware

Figure 53 shows the hardware for a direct-mapped cache with four words per block and a cache size of 4K words. The address splits into an 18-bit tag (bits 31–14), a 10-bit index (bits 13–4), a 2-bit block offset that selects the word (bits 3–2), and a byte offset (bits 1–0). The index selects one of the 1024 rows; the stored 18-bit tag is compared with the address tag, and *Hit* is the AND of the comparison and the valid bit. In parallel, a 4-to-1 multiplexer controlled by the block offset picks one of the row's four words as the 32-bit data output. Fetching four words per miss takes advantage of spatial locality.

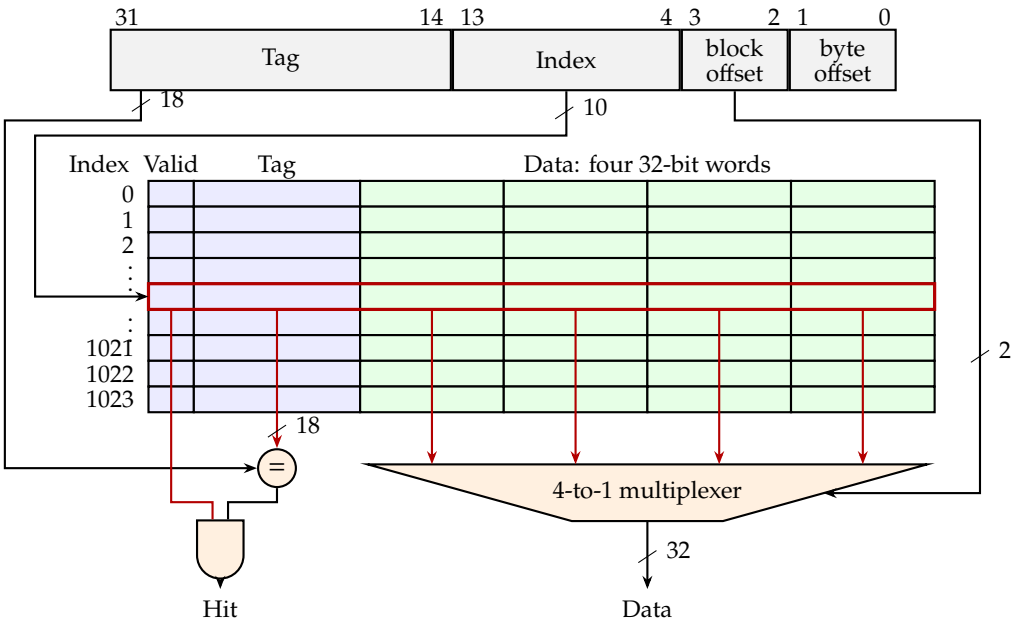


Figure 53: A direct-mapped cache with four-word blocks and 4K words of data; the row selected by the index is outlined.

26.3 Write hits

On a write hit, a WRITE-THROUGH cache updates both the cache and memory. A WRITE-BACK cache updates only the word in the cache block and allows the memory word to be "stale"; a DIRTY BIT added to each block records that memory and cache are inconsistent, so the block must be written back to memory when it is replaced. (The operating system flushes the cache before I/O.)

26.4 Block size

A larger block size exploits spatial locality: having accessed a word, we are likely to access nearby words soon. This applies strongly to instructions under the stored-program concept and to sequential array accesses. But a larger block size also means a larger miss penalty, since a miss takes longer to load the block from the next level, and if the block is too large relative to the cache there are too few blocks, so the miss rate goes up.

Example: In the extreme, a 4-byte cache with 4-byte blocks has only one entry (a valid bit, a tag, and bytes B3–B0). An item that was accessed is likely to be accessed again soon, but not immediately, so the next access is likely another miss: the cache keeps loading data and discarding it before it is used again. This ping-pong effect is a nightmare for the cache designer.

Figure 54 summarizes the trade-off: the miss penalty grows with block size, the miss rate first falls and then rises, and so the average access time is smallest at an intermediate block size.

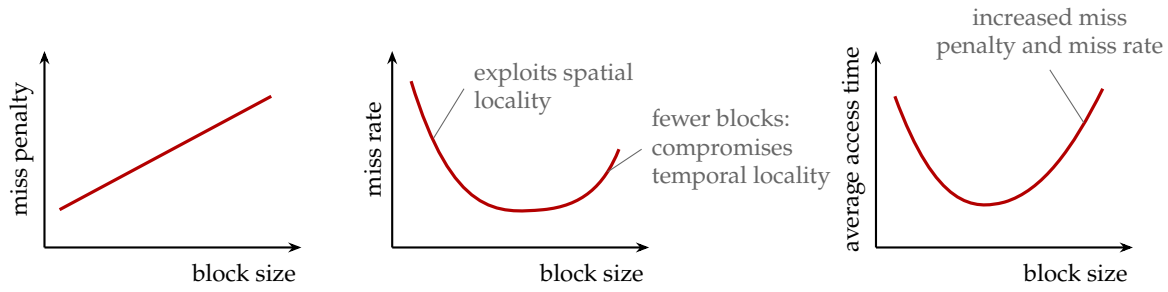


Figure 54: Qualitative effect of block size on the miss penalty, the miss rate, and the average access time (schematic).

26.5 The three Cs and full associativity

Definition: The "three Cs" model classifies misses. COMPULSORY MISSES occur when a program first starts: the cache does not yet contain any of the program's data, so misses are bound to occur. They cannot be avoided easily. Every block of memory (not only every block of the cache) has one compulsory miss.

Definition: A CONFLICT MISS occurs because two distinct memory addresses map to the same cache location: two blocks that happen to map to the same location can keep overwriting each other.

Conflict misses are a big problem in direct-mapped caches. One remedy is to make the cache bigger, which fails at some point; another is to let multiple distinct blocks fit at the same cache index.

The extreme is a FULLY ASSOCIATIVE CACHE. The address has a tag and an offset (as before) but no index: there are no rows, any block can go anywhere in the cache, and the address tag must be compared with all tags in the cache, in parallel, to see whether the data is there. With 32 B blocks the offset is 5 bits and the tag is 27 bits (Figure 55). A fully associative cache has no conflict misses,

since data can go anywhere, but it needs a hardware comparator for every entry: 64 KB of data in 4 B entries would need 16K comparators, which is infeasible.

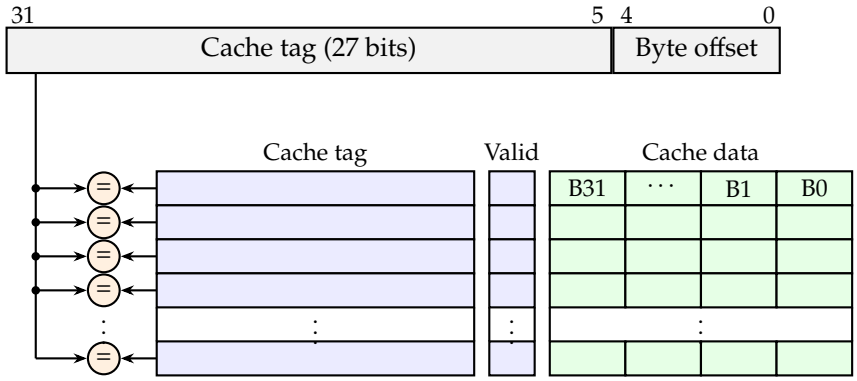


Figure 55: A fully associative cache with 32 B blocks: the 27-bit address tag goes to one comparator per entry, and all comparisons happen in parallel.

Definition: A CAPACITY MISS occurs because the cache has a limited size: it would not occur if the cache were larger (a loose definition). Capacity misses are the primary type of miss for fully associative caches.

To categorize the misses of an address trace, run it against a set of caches:

1. an infinite-size, fully associative cache: every miss here is a compulsory miss;
2. a fully associative cache of the size being examined: every miss not counted in step 1 is a capacity miss;
3. a cache of that size with the actual (finite) associativity: all remaining misses are conflict misses.

Example: In the six reads of the direct-mapped example, the misses at `0x14`, `0x34`, and `0x8014` are compulsory: each is the first access to its block, although the last one also evicts a valid block. Only three blocks are touched, so a fully associative cache of the same size has no capacity misses, and the final miss at `0x1C`, caused by the block of `0x8014` taking its row, is a conflict miss. A tag mismatch is therefore not always a conflict miss: it is compulsory when the block is accessed for the first time.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 26 (Fall 2020).

Lecture 27: Caches IV

27.1 N-way set-associative caches

An N-way set-associative cache sits between direct-mapped and fully associative caches. The address still has a tag and an offset, used as before, and an index that now points to a "row" called a set. Each set contains N blocks, so once the correct set is found, the address tag must be compared with all the tags in that set. The size of the cache is

$$\text{cache size} = \#\text{sets} \times N \text{ blocks/set} \times \text{block size}.$$

The cache is direct-mapped with respect to sets, and each set is fully associative with N blocks in it. Given an address:

1. find the correct set using the index;
2. compare the tag with all tag values in that set;
3. if one matches the access hits, otherwise it misses;
4. on a hit, use the offset as usual to find the desired data within the block.

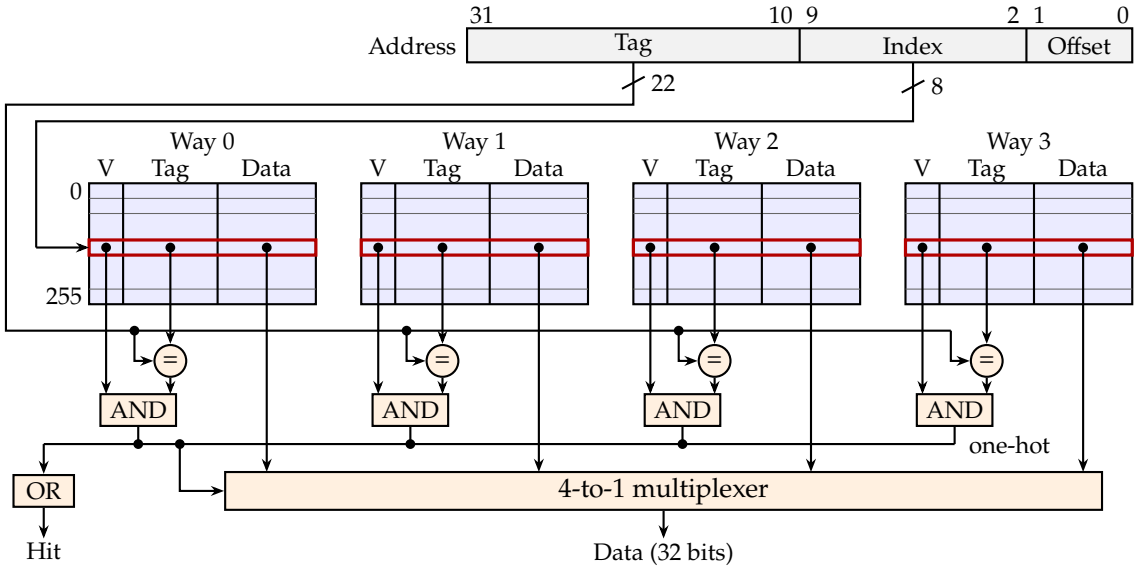


Figure 56: A 4-way set-associative cache with 256 sets of one-word blocks. The index selects the outlined set in every way; the one-hot hit signals select the data and their OR signals a hit.

For example, a 2-way set-associative cache with one-byte blocks and four blocks in total has 2 sets

of 2 blocks; even addresses map to set 0 and odd addresses to set 1.

Even a 2-way set-associative cache avoids a lot of conflict misses, and the hardware cost is modest: only N comparators. For a cache with M blocks, 1-way set associativity is direct mapping and M-way set associativity is full associativity, so both are special cases of the general design.

Figure 56 shows a 4-way set-associative cache with 256 sets and one-word blocks: a 22-bit tag, an 8-bit index, and a 2-bit byte offset. The index selects one row in each of the four ways; four comparators check the stored tags against the address tag in parallel, and each comparison is ANDed with that block's valid bit. At most one of the four results is 1 (a ONE-HOT encoding), so they can drive the select inputs of a 4-to-1 multiplexer that picks the data, while their OR is the hit signal.

27.2 Block replacement

In a direct-mapped cache the index completely specifies which position a block can go in on a miss. In an N-way set-associative cache the index specifies a set, but the block can occupy any position within the set; in a fully associative cache the block can be written into any position. When there is a choice, a new block goes into the first position whose valid bit is off. If all are valid, a REPLACEMENT POLICY, a rule for which block gets "cached out" on a miss, decides.

- LRU (least recently used): evict the block that has been accessed (read or written) least recently. Temporal locality means recent past use implies likely future use, so this is a very effective policy. With 2-way associativity it is easy to track (one LRU bit); with 4-way or greater it requires complicated hardware and much time.
- FIFO: ignore accesses and track only the order in which blocks were brought in.
- RANDOM: works acceptably if the workload has low temporal locality.

Example: Take the 2-way set-associative cache above (4 bytes in total, one-byte blocks, set = address mod 2) with LRU replacement and the byte accesses 0, 2, 0, 1, 4, 0, 2, 3, 5, 4:

Access	Set	Outcome	Set 0 (loc 0, loc 1)	Set 1 (loc 0, loc 1)
0	0	miss	0, –	–, –
2	0	miss	0, 2	–, –
0	0	hit	0, 2	–, –
1	1	miss	0, 2	1, –
4	0	miss, replace 2	0, 4	1, –
0	0	hit	0, 4	1, –
2	0	miss, replace 4	0, 2	1, –
3	1	miss	0, 2	1, 3
5	1	miss, replace 1	0, 2	5, 3
4	0	miss, replace 0	4, 2	5, 3

There are 2 hits and 8 misses. Each replacement evicts the block of its set that was used least recently: 4 evicts 2 because 0 was just reused, and the final 4 evicts 0 because 2 was used after it.

27.3 Average memory access time

Associativity, block size, replacement policy, and write policy are chosen by designing against a performance model, influenced by technology and program behavior.

Definition: The AVERAGE MEMORY ACCESS TIME (AMAT) is

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty},$$

and the design goal is to minimize it.

The goal is the illusion of a memory that is large, cheap, and fast, on average. With a hit time of 1 cycle, a miss rate of 5%, and a miss penalty of 20 cycles, AMAT $= 1 + 0.05 \times 20 = 1 + 1 = 2$ cycles.

There are two ways to reduce the miss rate. A larger cache is limited by cost and technology, and bigger caches are slower, while the hit time of a first-level cache must stay below the cycle time. More places to put each block of memory means more associativity: a fully associative cache puts any block in any line, and an N-way set-associative cache offers N places for each block (direct mapping is $N = 1$).

The miss penalty has grown. When caches first became popular it was about 10 processor cycles; a 3 GHz processor today (1/3 ns per cycle) needs 80 ns to reach DRAM, which is 240 cycles, of the order of 200 cycles. The solution is another cache between the processor cache and memory: a SECOND-LEVEL (L2) CACHE. This is the memory hierarchy again: registers, then L1, L2, and L3 caches on the processor chip, then DRAM, then flash and magnetic disks, each level larger, cheaper per byte, and slower than the one above.

With two levels of cache,

$$\text{AMAT} = \text{L1 hit time} + \text{L1 miss rate} \times \text{L1 miss penalty},$$
$$\text{L1 miss penalty} = \text{L2 hit time} + \text{L2 miss rate} \times \text{L2 miss penalty},$$

so AMAT = L1 hit time + L1 miss rate × (L2 hit time + L2 miss rate × L2 miss penalty). Here the L2 miss rate is the fraction of L1 misses that also miss in L2.

Typical scales: an L1 cache holds tens of KB, completes a hit in one clock cycle, and has miss rates of 1–5%; an L2 cache holds hundreds of KB, hits in a few clock cycles, and has miss rates of 10–20%. The L2 miss rate is high because L2 sees only the accesses that already missed in L1.

Example: Assume an L1 hit time of 1 cycle, an L1 miss rate of 5%, an L2 hit time of 5 cycles, an L2 miss rate of 15% (of L1 misses), and an L2 miss penalty of 200 cycles. Then the L1 miss penalty is $5 + 0.15 \times 200 = 35$ cycles and AMAT $= 1 + 0.05 \times 35 = 2.75$ cycles. Without the L2 cache the L1

miss penalty is the full 200 cycles and AMAT = $1 + 0.05 \times 200 = 11$ cycles, so the L2 cache makes the average access 4× faster (11/2.75 = 4).

27.4 Actual CPUs and caching in general

An early PowerPC had separate 32 KiB instruction and 32 KiB data L1 caches, an external L2 cache interface with an integrated controller and cache tags supporting up to 1 MiB of external L2, and dual memory management units with translation lookaside buffers; it was superscalar (3 instructions per cycle) with 6 execution units (2 integer and 1 double-precision IEEE floating point among them). The Pentium M also has 32 KiB L1 instruction and data caches, plus a 1 MB L2 cache, and an Intel Core i7 die shows six cores sharing an L3 cache, with an on-chip memory controller for the off-chip DRAM.

Caching shows up over and over in computer systems: filesystem caches, web page caches, game databases and tablebases, and software memoization. The big idea is that if something is expensive but we want to do it repeatedly, we do it once and cache the result. The cache design choices are the size (speed versus capacity), the block size (the cache's aspect ratio), the write policy (write-through or write-back), the associativity (direct-mapped, set associative, or fully associative), the block replacement policy, and whether to add second- or third-level caches; a performance model picks among them depending on programs, technology, and budget.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 27 (Fall 2020).

Lecture 28: OS and Virtual Memory Intro

28.1 From a processor to a computer

A processor with memory, like the datapath that runs RISC-V programs in a simulator, executes one program and then stops. A laptop adds input and output: a screen, a keyboard, storage, and networking. Even a $35 Raspberry Pi has a CPU with caches, memory, and storage I/O (a micro SD card), wireless I/O (WiFi), network I/O (Ethernet), screen I/O (HDMI), and serial I/O (USB). What makes such a machine start up into a desktop instead of a single program is software: the operating system (OS), the biggest piece of software on the machine (the Linux kernel alone has grown to millions of lines of code).

The OS is the first thing that runs when the computer starts. It

- finds and controls all the devices in the machine in a general way, relying on hardware-specific device drivers;
- starts services (more than 100): the file system, the network stack (Ethernet, WiFi, Bluetooth, . . .), the TTY (keyboard), and so on;
- loads, runs, and manages programs: it runs multiple programs at the same time (time-sharing), isolates programs from each other (isolation), and multiplexes resources such as devices between applications.

The core of the OS provides isolation, so each program runs in its own little world, and interaction with the outside world through devices: disk, display, network, and so on.

28.2 What the OS needs from hardware

- *Memory translation.* Each running process has a mapping from "virtual" to "physical" addresses, different for each process. A load or store issues a virtual address, but the memory actually accessed is at a physical address.
- *Protection and privilege.* The processor has at least two running modes, "user" and "supervisor" (RISC-V also has a "machine" mode below supervisor). A lesser privilege cannot change its memory mapping, but the supervisor can change the mapping of any program, and has its own mapping from virtual to physical addresses.
- *Traps and interrupts*: a way of going into supervisor mode on demand.

28.3 Booting and launching applications

When the computer switches on, the CPU does what the simulator does: it executes instructions from some default start address (for example `PC = 0x2000`) in a memory-mapped flash ROM. The code there copies the firmware into regular memory and jumps into it. Booting then proceeds in four steps:

1. the BIOS (Basic Input Output System) finds a storage device and loads its first sector (block of data);
2. the BOOTLOADER, stored for example on disk, loads the OS KERNEL from disk into a location in memory and jumps into it;
3. the OS boots, initializing services, drivers, and so on;
4. INIT launches an application that waits for input in a loop (a terminal, a desktop, . . .).

Applications are called PROCESSES in most operating systems. A process has separate memory, while THREADS share memory; both run (pseudo) simultaneously. An application is started by another process, such as the shell, calling an OS routine through a SYSCALL; how depends on the OS, and Linux uses `fork` to create a new process and `execve` (execute file) to load the application. The OS loads the executable file from disk using the file system service, puts the instructions and data into memory (the `.text` and `.data` sections), prepares the stack and heap, sets `argc` and `argv`, and jumps to the start of `main`. The shell waits for `main` to return (a JOIN).

28.4 Supervisor mode and syscalls

If something goes wrong in an application it could crash the entire machine, and malware could do worse. The OS therefore enforces resource constraints on applications (access to memory and devices), and to protect the OS from applications, CPUs have a SUPERVISOR MODE, set for example by a status bit in a special register. In user mode (not supervisor mode) a process can access only a subset of the instructions and of (physical) memory. A process can leave supervisor mode with a special instruction, but it cannot enter it directly, only through an interrupt. Supervisor mode is a bit like "superuser" but is used much more sparingly: most of the OS does not run in it, and errors in supervisor mode are often catastrophic (a blue "screen of death", or a corrupted disk).

To call an OS routine, for example to read a file, launch a new process, ask for more memory (`malloc`), or send data, a program performs a syscall: it sets up the function arguments in registers and raises a software interrupt with a special instruction. The OS performs the operation and returns to user mode. In this way the OS mediates access to all resources and devices.

28.5 Interrupts, exceptions, and traps

The processor must switch into supervisor mode when "something" happens.

Definition: An INTERRUPT is caused by an event external to the running program, such as a key press or disk I/O. It is asynchronous to the program, so it can be handled on any convenient instruction ("whenever it is convenient, just don't wait too long").

Definition: An EXCEPTION is caused by an event during the execution of one instruction of the running program, such as a memory error, a bus error, an illegal instruction, accessing memory it is not supposed to, reading a CSR it may not read at that privilege, or a raised exception. It is synchronous, so it must be handled precisely on the instruction that caused it ("drop whatever you are doing and act now").

Definition: A TRAP is the action of servicing an interrupt or exception: a hardware jump to the "interrupt" or "trap handler" code.

RISC-V has two instructions that raise exceptions on purpose: `ECALL` triggers an exception to the higher privilege level and is how a program communicates with the operating system (it implements syscalls), while `EBREAK` triggers an exception within the current privilege level.

A trap alters the regular execution flow: while the program executes instructions i_{i-1}, i_i, i_{i+1}, an external or internal event, often unexpected from the program's point of view, transfers control at i_i to a handler (another program, often the OS), which runs its own instructions and then returns to the program.

Definition: A trap is PRECISE if the handler's view of the machine state is that every instruction before the trapped one has completed and no instruction after it has executed.

Precise traps let the handler return from an interrupt by restoring the user registers and jumping back to the interrupted instruction; the handler software does not need to understand the pipeline or what the program was doing. Handling a trap caused by an exception is more complex than handling an interrupt, and providing precise traps is tricky in a pipelined, superscalar, out-of-order processor, but it is a requirement for things to work right. In the five-stage pipeline, exceptions are detected in several stages (Figure 57) and handled like pipeline hazards:

1. complete the execution of the instructions before the one that caused the exception;
2. flush the instructions currently in the pipeline (convert them to `nop`s, or "bubbles");
3. optionally store the exception cause in a status register, indicating its type;
4. transfer execution to the trap handler;
5. optionally, return to the original program and re-execute the instruction.

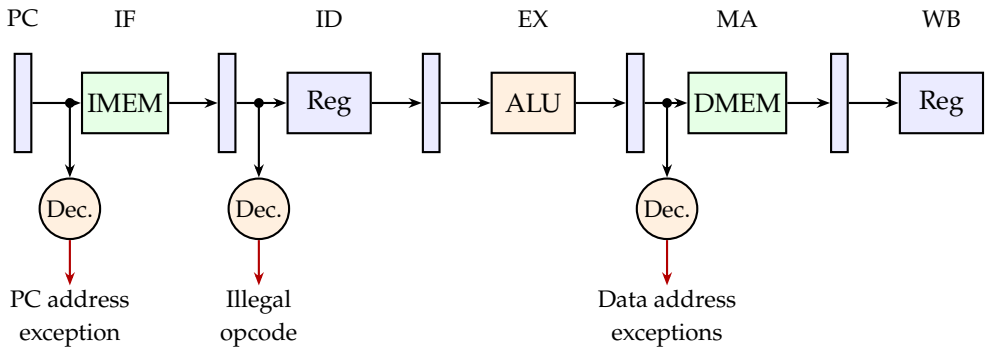


Figure 57: Exceptions in a five-stage pipeline: instruction fetch can raise a PC address exception, decode an illegal opcode exception, and the memory stage data address exceptions.

28.6 Multiprogramming and the need for virtual memory

The OS runs multiple applications at the same time, but not really (unless there is a core per process): it switches between processes very quickly on a human time scale, which is a CONTEXT SWITCH. When jumping into a process, the OS sets a timer (an interrupt). When the timer expires, the OS stores the PC, registers, and the rest of the process state, picks a different process to run and loads its state, sets the timer, changes to user mode, and jumps to the new PC. Deciding which process to run is called SCHEDULING.

Supervisor mode alone does not fully isolate applications from each other or from the OS: an application could overwrite another application's memory. Programs also typically start at some fixed address, such as `0x8FFFFFFF`, so hundreds of programs would have to share memory at that location, and a program may want to address more memory than the machine has (for sparse data structures, for example). The solution is VIRTUAL MEMORY, which gives each process the illusion of a full memory address space that it has completely to itself.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 28 (Fall 2020).

Lecture 29: Virtual Memory I

29.1 Virtual memory concepts

Virtual memory is the next level in the memory hierarchy, below DRAM. It gives a program the illusion of a very large main memory: a working set of pages resides in main memory and the others are on disk. Demand paging makes it possible to run programs larger than the primary memory (DRAM), and virtual memory hides differences between machine configurations. It also lets the OS share memory and protect programs from each other; today it is more important for protection than as another level of the memory hierarchy. Each process thinks it has all the memory to itself. (Historically, virtual memory predates caches.)

Processes use virtual addresses, for example `0` to `0xffffffff`, so many processes use the same, conflicting addresses; memory uses physical addresses, also for example `0` to `0xffffffff`. A memory manager maps virtual addresses to physical addresses.

Definition: An address space is the set of addresses for all available memory locations. The virtual address space is the set of addresses the user program knows about; the physical address space is the set of addresses that map to actual physical locations in memory, hidden from user applications. The memory manager maps ("translates") between the two.

The two spaces need not have the same size: a laptop's x86-64 processor, for example, reports 39-bit physical and 48-bit virtual addresses.

A library analogy helps. A book title is like a virtual address, and the Library of Congress call number is like a physical address. The card catalogue is like a page table, mapping book titles to call numbers. On the card, "in the local library" versus "in another branch" is like the valid bit indicating main memory versus disk, and "available for 2-hour in-library use" versus "2-week checkout" is like access rights.

The memory hierarchy must let multiple processes occupy memory simultaneously while protecting them, so that one program cannot read or write another's memory, and it must give each program the illusion of its own private address space. If code starts at address `0x40000000`, different processes have different code residing at the same address, so each program needs a different view of memory.

29.2 Physical memory and storage

DRAM is volatile. The latency to access the first word is about 10 ns (30–40 processor cycles), and each successive word takes 0.5–1 ns, since DRAM supports bursts; each access brings 64 bits.

Storage ("disk") is attached as a peripheral I/O device and is non-volatile:

	SSD	HDD
Access time	40–100 µs (~100k processor cycles)	< 5–10 ms (10–20M processor cycles)
Cost	$0.05–0.5/GB	$0.01–0.1/GB

Disks are slow because they are mechanical: at 10,000 revolutions per minute one revolution takes 60/10,000 s = 6 ms, and the average random access time is 3 ms (about 10^7 processor cycles).

An SSD is made of transistors, with nothing mechanical that turns. It is like a "ginormous" register file that does not forget when the power is off (non-volatile), with fast access to all locations regardless of address, though still much slower than registers or DRAM. It reads and writes blocks, not bytes, has potential reliability issues, and has some unusual requirements: it cannot erase single bits, only entire blocks. Flash memory is a 3D array of bit cells, with up to 256 layers.

29.3 The memory manager

In a "bare metal" system without an OS, the addresses issued by loads and stores are real physical addresses, so any process can issue any address and access any part of memory, even areas it does not own, such as the OS data structures. Instead, all addresses should pass through a translation mechanism that the OS controls before they reach DRAM, which also checks that the process has permission to access that part of memory. The OS multiplexes hundreds of processes over the available cores, but there is only one memory, and its contents cannot simply be "saved" on a context switch.

The memory manager has three responsibilities:

1. map virtual to physical addresses;
2. protection: isolate memory between processes, so each process gets dedicated "private" memory, errors in one program do not corrupt the memory of another, and user programs cannot mess with the OS's memory;
3. swap memory to disk: give the illusion of a larger memory by storing some content on disk, which is usually much larger and slower than DRAM, using "clever" caching strategies.

29.4 Paged memory

The concept of PAGED MEMORY dominates. Physical memory (DRAM) is broken into pages; a typical page size on modern operating systems is 4 KiB or more, which needs 12 bits to address. A 32-bit virtual address then splits into a 20-bit page number and a 12-bit offset. Each process has a dedicated PAGE TABLE, and its pages need not be consecutive in physical memory.

To translate (Figure 58), the OS keeps track of which process is active and chooses the correct page table; the memory manager extracts the page number from the virtual address (for example its top 20 bits), looks up the page address in the page table, and computes the physical address from the page address and the offset copied from the virtual address. Physical addresses may, but do not have to, have more or fewer bits than virtual addresses.

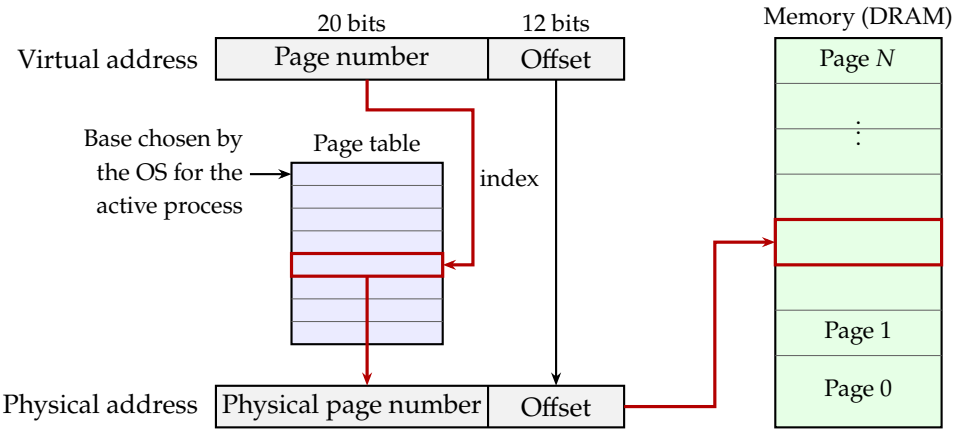


Figure 58: Paged address translation: the page number indexes the active process's page table, the entry gives the physical page, and the offset passes through unchanged.

Assigning different pages in DRAM to different processes also keeps them from accessing each other's memory, which provides isolation; the page tables are handled by the OS in supervisory mode. Sharing is also possible: the OS may assign the same physical page to several processes. A page can also be write protected, and writing to a protected page (for example program code) raises an exception.

Example: With 32-bit virtual addresses and 4 KiB pages there are 2^{20} pages, so with 4-byte entries a single page table takes 4×2^{20} B = 4 MiB, about 0.1% of a 4 GiB memory. That is much too large for a cache, so page tables are stored in memory (DRAM), and a `lw` or `sw` that misses in the cache needs two slow memory accesses: one for the page table entry and one for the data. To minimize the penalty, transfer blocks rather than words between DRAM and the processor cache (exploiting spatial locality), and use a cache for frequently used page table entries.

29.5 Page faults

Caches deal with individual blocks, usually about 64 B on modern systems; virtual memory deals with individual pages, usually about 4 KiB. Bytes, words, blocks, and pages are just different ways of looking at memory. For a 16 KiB DRAM with 4 KiB pages, 128 B cache blocks, and 4 B words, memory is 4 pages, or 128 blocks, or 4096 words, or 16,384 bytes, and a page is 32 blocks or 1024 words.

Each page table entry has a valid bit (the page is allocated) and a bit that says whether the page is in DRAM or on disk. A memory access checks the entry:

- valid and in DRAM: read or write the data;
- valid but on disk: allocate a new page in DRAM (if memory is full, evict a page from DRAM and store it to disk), read the page from disk into memory, then read or write the data;
- not valid: allocate a new page in DRAM (evicting a page if memory is full), then read or write the data.

The last two cases are PAGE FAULTS and need OS intervention. Page faults are treated as exceptions: the page fault handler, yet another function of the interrupt (trap) handler, updates the page table and its status bits and initiates the transfers; if a page must be swapped in from disk, it performs a context switch. After the page fault, the faulting instruction is re-executed.

Example: When there is insufficient free memory, `malloc()` returns `NULL`. A program that keeps allocating 1 GiB blocks without freeing them

```c
int main(void) {
  const int G = 1024*1024*1024;
  for (int n=0; ;n++) {
    char *p = malloc(G*sizeof(char));
    if (p == NULL) {
      fprintf(stderr, "failed to allocate > %g TiBytes\n", n/1000.0);
      return 1;  // abort program
    }
    // no free, keep allocating until out of memory
  }
}
```

reported `failed to allocate > 131 TiBytes`, far more than the machine's DRAM. The allocations only reserve virtual address space; since the pages are never accessed, no page of DRAM is ever allocated for them.

Since DRAM acts like a "cache" for disk, writes could go directly to disk (write-through) or only when the page is evicted (write-back). With disk accesses costing millions of cycles, virtual memory uses write-back.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 29 (Fall 2020).

Lecture 30: Virtual Memory II

30.1 Hierarchical page tables

Example: With 32-bit virtual addresses and 4 KiB pages, a single page table takes 4×2^{20} B = 4 MiB, about 0.1% of a 4 GiB memory. Every process needs its own page table, so 256 processes need $256 \times 4 \times 2^{20}$ B = 256×4 MiB = 1 GiB, 25% of a 4 GiB memory. With 64-bit addresses the problem is far worse.

There are two ways to keep the size of page tables reasonable. One is to increase the page size: doubling the page size cuts the page table in half, at the expense of potentially wasted memory. The other is a hierarchical page table, which keeps small pages. Most programs use only a fraction of their address space (code, static data, and heap at the bottom, the stack at the top, and a large unused region between), so the page table is split into two (or more) parts and the parts that would describe unused memory are not allocated. RISC-V does this.

In the two-level page table of 32-bit RISC-V (Figure 59), the virtual address splits into a 10-bit level-1 index p_1 (bits 31–22), a 10-bit level-2 index p_2 (bits 21–12), and a 12-bit offset. The processor register SPTBR (supervisor page table base register) holds the root of the current page table. Entry p_1 of the level-1 table points to a level-2 table, and entry p_2 of that table gives the data page; the offset selects the byte within the 4096 B page, so each level-1 entry covers 1024×4096 B of the virtual address space. Each table has 1024 entries of 4 bytes, so it fills exactly one page, and pages of the tables themselves, like data pages, may be in primary memory (DRAM) or in secondary memory (disk). An entry may also describe a nonexistent page, so the level-2 tables for unused regions are never needed.

In 32-bit RISC-V the virtual address is VPN[1] (10 bits), VPN[0] (10 bits), and a 12-bit offset, where VPN is the virtual page number; the physical address has 34 bits: PPN[1] (12 bits), PPN[0] (10 bits), and the 12-bit offset, where PPN is the physical page number. A page table entry (PTE) is 32 bits: the physical page number and status bits for protection and usage (read, write, execute), validity, and so on.

Field	PPN[1]	PPN[0]	RSW	D	A	G	U	X	W	R	V
Bits	12	10	2	1	1	1	1	1	1	1	1

If $R = W = X = 0$, the entry points to the next level of the page table; otherwise it is a leaf PTE that maps a page.

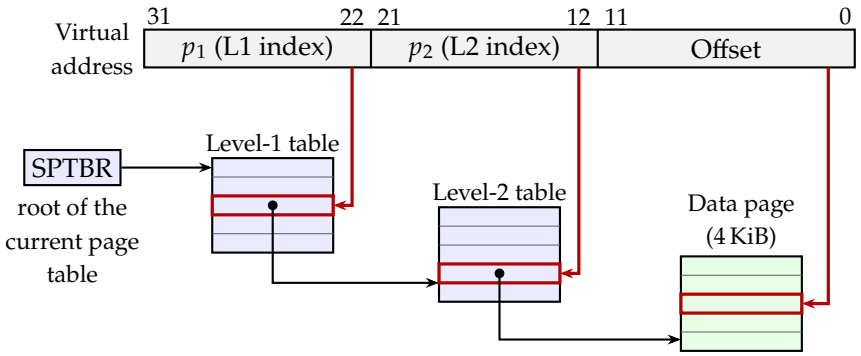


Figure 59: A two-level page table: SPTBR points to the level-1 table, p_1 selects an entry pointing to a level-2 table, p_2 selects the entry for the data page, and the offset selects the byte.

30.2 Translation lookaside buffers

Every instruction and data access needs address translation and protection checks: the virtual page number is translated to a physical page number, the offset is kept, and the check uses the kernel/user mode and whether the access is a read or a write, raising an exception if it fails. Good virtual memory design should be fast (about one cycle) and space efficient. But translation is very expensive: with a single-level page table each reference becomes two memory accesses, and with a two-level page table it becomes three.

Definition: A TRANSLATION LOOKASIDE BUFFER (TLB) caches some translations. Each entry holds a valid bit, a dirty bit, a tag taken from the virtual page number, and the physical page number. A TLB hit gives a single-cycle translation; a TLB miss requires a PAGE-TABLE WALK to refill the TLB.

TLBs typically have 32–128 entries and are usually fully associative. Each entry maps a large page, so there is less spatial locality across pages and two entries are more likely to conflict. Larger TLBs (256–512 entries) are sometimes 4–8-way set associative, and larger systems sometimes have multi-level (L1 and L2) TLBs. Replacement is random or FIFO. The TLB REACH is the size of the largest virtual address space that the TLB can map simultaneously.

The TLB is checked before the cache. The cache cannot hold the requested data if the corresponding page is not in physical memory, and with the TLB first the cache receives physical addresses. On a TLB hit the physical address goes to the cache (and to main memory on a cache miss); on a TLB miss the page table supplies the translation, so it is the TLB, not the page table, that normally translates. The virtual page number splits into a TLB tag and a TLB index, used just as in a cache, while the physical address is split a different way, into the cache's tag, index, and offset; the two splits are unrelated.

In the pipeline, both the instruction fetch and the data access go through a TLB and then a cache, and each can see a TLB miss, a page fault, or a protection violation. A TLB miss needs a hardware or software mechanism to refill the TLB; it is usually done in hardware by a page-table walker, which starts from the page-table base register and reaches main memory through the memory controller,

assuming the page tables are held in untranslated physical memory. A page fault (for example the page is on disk) needs a precise trap so that the software handler can easily resume after retrieving the page. A protection violation may abort the process. Figure 60 puts the pieces together.

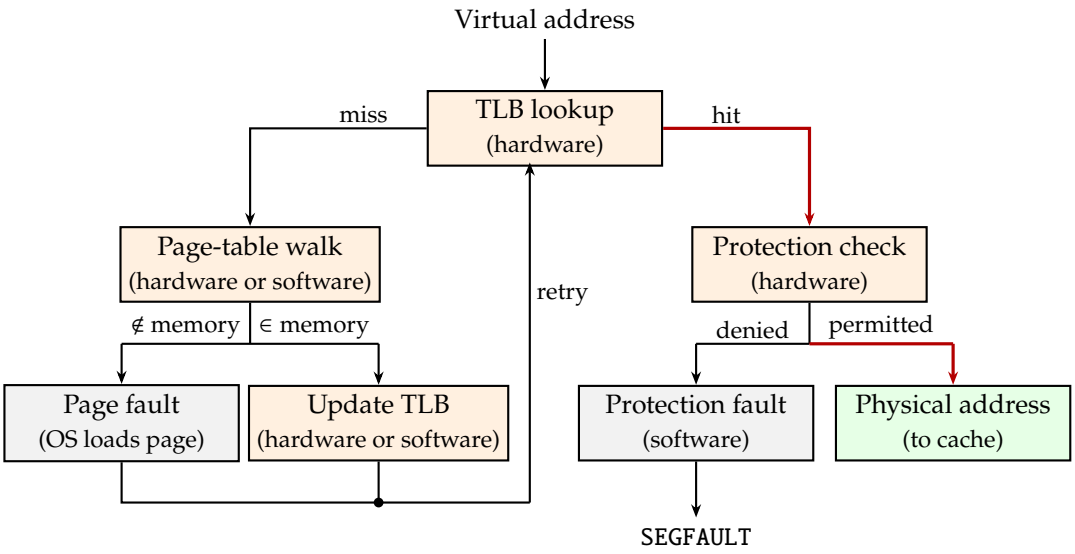


Figure 60: Address translation with a TLB. A hit goes to the protection check; a miss walks the page table, which either updates the TLB or finds the page not in memory and raises a page fault; either way the access is then retried.

Modern virtual memory systems thus provide the illusion of a large, private, uniform store: protection and privacy (several users or processes, each with a private address space), demand paging (programs larger than primary memory, with a swapping space on disk), and independence from machine configurations. The price is address translation on each memory reference, which the TLB makes cheap.

On a CONTEXT SWITCH the processor saves the register values and the PC and changes the value in the supervisor page table base register. The current TLB entries belong to a different process, so all of them are set to invalid.

30.3 Virtual memory performance

Cache	Virtual memory
Block or line	Page
Miss	Page fault
Block size: 32–64 B	Page size: 4–8 KiB
Placement: direct mapped or N-way set associative	Fully associative
Replacement: LRU or random	LRU, FIFO, or random
Write-through or write-back	Write-back

Virtual memory is the level of the memory hierarchy below main memory. The TLB comes before the cache, but paging affects the transfer of data from disk to main memory. The same CPI and AMAT equations apply, but main memory now acts like a mid-level cache in front of the disk. Typical values:

Caching	Demand paging
Cache entry	Page frame
Cache block (≈32–64 bytes)	Page (≈4Ki bytes)
Cache miss rate (1% to 20%)	Page miss rate (< 0.001%)
Cache hit (≈1 cycle)	Page hit (≈100 cycles)
Cache miss (≈100 cycles)	Page miss (≈5M cycles)

Example: Suppose an L1 hit takes 1 clock cycle and hits 95% of accesses, an L2 hit takes 10 cycles and hits 60% of L1 misses, DRAM takes 200 cycles (about 100 ns), and disk takes 20,000,000 cycles (about 10 ms). Without paging,

$$\text{AMAT} = 1 + 5\% \times 10 + 5\% \times 40\% \times 200 = 5.5 \text{ cycles}.$$

With paging, a fraction $1 - \text{HR}_{\text{mem}}$ of the accesses that reach DRAM go to disk, where HR_{mem} is the memory hit rate:

$$\text{AMAT} = 5.5 + 5\% \times 40\% \times (1 - \text{HR}_{\text{mem}}) \times 20{,}000{,}000.$$

For $\text{HR}_{\text{mem}} = 99\%$ this is $5.5 + 0.02 \times 0.01 \times 20{,}000{,}000 = 4005.5$ cycles, about 728× slower: 1 in 5000 memory accesses goes to disk, and a 10-second program takes about 2 hours. For $\text{HR}_{\text{mem}} = 99.9\%$ it is 405.5 cycles, and for $\text{HR}_{\text{mem}} = 99.9999\%$ it is 5.9 cycles. Page faults must be very rare.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 30 (Fall 2020).

LECTURE 31: I/O

31.1 Interacting with devices

A program running on a CPU interacts with the outside world through an I/O interface for keyboards, networks, mice, displays, and so on. The interface must connect to many types of devices (over buses such as PCI, SATA, SAS, and USB), control them, respond to them, transfer data, and present them to user programs so that they are useful; this is the job of the operating system. Each device exposes registers, typically a command (control) register and a data register.

For I/O the processor must read a sequence of bytes (input) and write a sequence of bytes (output). There are two interface options: special input/output instructions and hardware, or MEMORY-MAPPED I/O, in which a portion of the address space is dedicated to I/O. The I/O device registers live there (no memory does), and normal load and store instructions such as `lw` and `sw` access them. Memory-mapped I/O is very common and is used by RISC-V. For example, addresses `0x00000000`–`0x7FFFFFFF` might be memory-mapped I/O, holding device control and data registers, and `0x80000000`–`0xFFFFFFFF` program and data memory.

31.2 Processor and I/O speeds

A 1 GHz microprocessor can move 4 GiB/s with `lw` and `sw`. Typical peak I/O data rates (actual throughput is lower) are:

Device	Rate	Device	Rate
Keyboard	10 B/s	SATA3 HDD	480 MiB/s
Bluetooth 3.0	3 MiB/s	Cutting-edge SSD	560 MiB/s
USB 2/3.1	0.06–1.25 GiB/s	Thunderbolt 3	5 GiB/s
WiFi (depends on standard)	7–250 MiB/s	High-end DDR4 DRAM	32 GiB/s
Gigabit Ethernet	125 MiB/s	HBM2 DRAM	64 GiB/s

Common I/O devices neither deliver nor accept data at a rate matching the processor's speed.

31.3 Polling

Device registers generally serve two functions: a CONTROL REGISTER says it is OK to read or write (the device is ready), like a flagman on a road, and a DATA REGISTER contains the data. In POLLING, the processor reads the control register in a loop, waiting for the device to set the ready bit ($0 \rightarrow 1$), which indicates "data available" or "ready to accept data". The processor then loads from the data register (input) or writes to it (output), and the device resets the control bit ($1 \rightarrow 0$).

Example: With the memory map `0x7ffff000` input control register, `0x7ffff004` input data register, `0x7ffff008` output control register, and `0x7ffff00c` output data register, reading from the keyboard into `a0` is

```
        lui  t0, 0x7ffff        # 7ffff000 (io addr)
Waitloop: lw   t1, 0(t0)        # read control
        andi t1, t1, 0x1        # ready bit
        beq  t1, zero, Waitloop
        lw   a0, 4(t0)          # data
```

and writing `a1` to the display is

```
        lui  t0, 0x7ffff        # 7ffff000
Waitloop: lw   t1, 8(t0)        # write control
        andi t1, t1, 0x1        # ready bit
        beq  t1, zero, Waitloop
        sw   a1, 12(t0)         # data
```

The "ready" bit is from the processor's point of view: for output it means the device is ready to accept data.

Example: Take a 1 GHz processor for which a polling operation (call the polling routine, check the device, return) takes 400 clock cycles. A mouse must be polled 30 times per second so that no motion is missed (which would make the cursor move choppily). Polling costs $30 \times 400 = 12{,}000$ clocks/s, and $12 \times 10^3/(1 \times 10^9) = 0.0012\%$ of the processor: little impact (except that we need to know we should be polling). A hard disk delivering 16 MB/s in 16-byte chunks must be polled 16 MB/s/16 B = 1M times per second, costing 1M×400 = 400M clocks/s, or $400{\times}10^6/(1{\times}10^9) = 40\%$ of the processor, which is unacceptable. (Polling is only part of the problem: accessing data in small chunks is inefficient too.)

31.4 Interrupts and DMA

Polling wastes processor resources, like waiting at the door for guests to show up instead of having a bell. The computer's bell is the INTERRUPT: when an I/O device is ready or needs attention, it interrupts the current program and transfers control to the trap handler in the operating system. With no I/O activity there is nothing to do, but with lots of I/O, interrupts are expensive: they thrash caches and virtual memory and require saving and restoring state. So low-data-rate devices (mouse, keyboard) use interrupts, whose overhead ends up being low; one could poll them from the timer interrupt, but there is no reason to. High-data-rate devices (network, disk) start with interrupts, so that nothing happens when there is no data, and switch to direct memory access once data starts coming.

With PROGRAMMED I/O, the standard for ATA hard-disk drives, the CPU executes `lw`/`sw` instructions for all data movement to and from devices, so it spends its time both getting data from the device

to main memory and computing with it. This is not ideal: the CPU has to execute all transfers when it could be doing other work, device speeds do not align well with CPU speeds, and a beefy general-purpose CPU uses more energy than simpler hardware would. About 5% of the CPU cycles on Google servers are spent in the `memcpy()` and `memmove()` library routines.

Definition: Direct memory access (DMA) lets I/O devices read and write main memory directly, which until now only the CPU could do. It needs new hardware, the DMA engine, which contains registers written by the CPU: the memory address at which to place the data, the number of bytes, the I/O device number and direction of transfer, and the unit of transfer (the amount to transfer per burst).

For incoming data: (1) the device raises an interrupt; (2) the CPU takes it and initiates the transfer, instructing the DMA engine or device to place the data at a certain address; (3) the device and DMA engine handle the transfer while the CPU is free to execute other things; (4) on completion, the device or DMA engine interrupts the CPU again. For outgoing data: (1) the CPU decides to initiate a transfer and confirms that the external device is ready; (2) it begins the transfer, instructing the DMA engine or device that the data is available at a certain address; (3) the device and DMA engine handle the transfer while the CPU does other work; (4) the device or DMA engine interrupts the CPU to signal completion. Figure 61 shows a disk read.

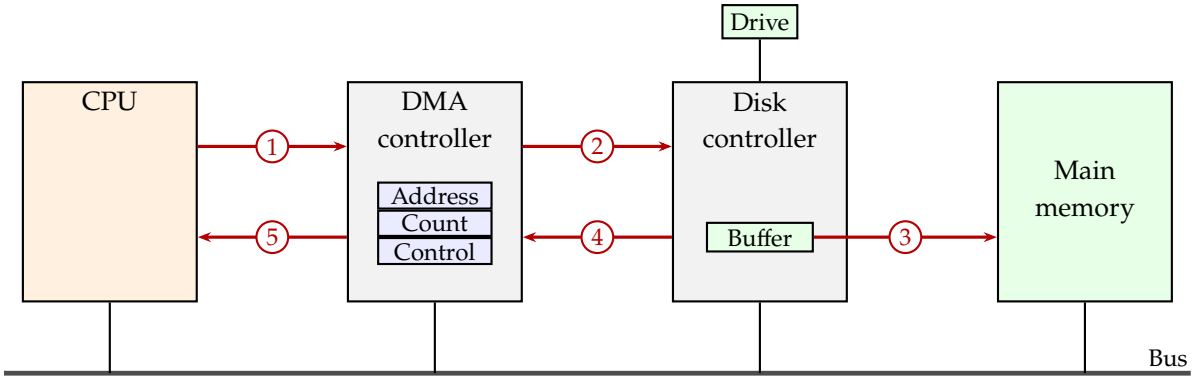


Figure 61: A DMA transfer from disk: (1) the CPU programs the DMA controller's registers; (2) the DMA controller asks the disk controller to transfer data to memory; (3) the data moves from the disk controller's buffer to main memory; (4) the disk controller acknowledges; (5) the DMA controller interrupts the CPU when the whole transfer is done. All transfers use the bus.

Where in the memory hierarchy should the DMA engine be plugged in? At one extreme, between the L1 cache and the CPU, coherency comes for free, but the transferred data trashes the CPU's working set. At the other, between the last-level cache and main memory, the caches are left alone, but coherency must be managed explicitly.

31.5 Networks

Networks originally shared I/O devices such as printers between computers; then they let computers communicate (file transfer protocols), then people (e-mail), then networks of computers (file sharing, the World Wide Web). In 1963 J. C. R. Licklider, at the Department of Defense's ARPA, wrote a memo describing the desire to connect the computers at research universities (Stanford, Berkeley, UCLA, ...); in 1969 ARPA deployed four nodes at UCLA, SRI, Utah, and UCSB; and in 1973 Robert Kahn and Vint Cerf invented TCP, now part of the Internet Protocol Suite. Internet growth has been exponential since the start. The World Wide Web, a "system of interlinked hypertext documents on the Internet", goes back to Vannevar Bush's 1945 description of a hypertext system called the "memex"; in 1989 Sir Tim Berners-Lee proposed and implemented the first successful communication between a Hypertext Transfer Protocol (HTTP) client and server over the Internet (the first web server ran in 1990). Dot-com entrepreneurs rushed in around 2000, and the bubble burst in 2001.

A software protocol sends and receives messages. To send: (1) the application copies the data to an OS buffer; (2) the OS calculates a checksum and starts a timer; (3) the OS sends the data to the network interface hardware and says start. To receive, the steps run in reverse: (3) the OS copies the data from the network interface hardware to an OS buffer; (2) it calculates the checksum, and if it is OK sends an ACK, and otherwise deletes the message (the sender resends when its timer expires); (1) if OK, it copies the data to the user address space and signals the application to continue. A message has a header (destination and source network IDs, length, and ACK information), a payload (a command, address, or data), and a trailer (the checksum). The hardware is traditionally a NETWORK INTERFACE CARD (NIC), wired or wireless, which transfers data using programmed I/O (old) or DMA (new).

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 31 (Fall 2020).

Lecture 32: Flynn Taxonomy, SIMD Instructions

32.1 Matrix multiplication as a reference problem

Parallelism appears at every level of a machine: parallel requests assigned to computers, parallel threads assigned to cores, parallel instructions (pipelining), parallel data (more than one data item at a time, such as adding four pairs of words), and logic gates that all work at the same time. This unit is about DATA-LEVEL PARALLELISM.

Matrix multiplication is a basic operation in many engineering, data, and image-processing tasks (image filtering, noise reduction, machine learning, . . .); for example, inference in a machine-learning model passes the inputs through layers $L_0, \ldots, L_{n-1}$ with weights $W_0, \ldots, W_{n-1}$, each a matrix-vector multiplication. The routine `dgemm` (double-precision floating-point matrix multiplication, originally in FORTRAN) computes $C = A \cdot B$ for $N \times N$ matrices,

$$C_{ij} = \sum_k A_{ik} B_{kj}.$$

For example,

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 2 & 1 \cdot 3 + 0 \cdot 4 \\ 0 \cdot 1 + 1 \cdot 2 & 0 \cdot 3 + 1 \cdot 4 \end{pmatrix} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}.$$

Multiplying two $N \times N$ matrices takes $2N^3$ floating-point operations (FLOPs), and performance is measured in FLOP/s (1 MFLOP/s is a million floating-point additions or multiplications per second). With the matrices stored in column order (element (i, k) of a at `a[i+k*N]`), the scalar C version is

```c
void dgemm_scalar(int N, double *a, double *b, double *c) {
  for (int i=0; i<N; i++)
    for (int j=0; j<N; j++) {
      double cij = 0;
      for (int k=0; k<N; k++)
        //      a[i][k]  * b[k][j]
        cij += a[i+k*N] * b[k+j*N];
      // c[i][j]
      c[i+j*N] = cij;
    }
}
```

and the same triple loop in Python runs at about 5.4 MFLOP/s. Timing uses `clock()` from `<time.h>`: record `clock_t start = clock()` before the task and `end` after it, and compute `(double)(end-start)/CLOCKS_PER_SEC` seconds. (`clock()` measures execution time, not real

time; they differ a lot in shared environments and under heavy system load.)

N	Python (GFLOP/s)	C (GFLOP/s)	AVX (GFLOP/s)
32	0.0054	1.30	4.56
160	0.0055	1.30	5.47
480	0.0054	1.32	5.27
960	0.0053	0.91	3.64

C is about 240× faster than Python, and the SIMD (AVX) version described below is up to about 4× faster again. The theoretical peak of the Intel i7-5557U used is about 25 GFLOP/s: 3.1 GHz × 2 instructions/cycle × 4 multiplications per instruction = 24.8 GFLOP/s.

32.2 Flynn's taxonomy

The choice of hardware parallelism and of software parallelism are independent: concurrent software can run on serial hardware and sequential software can run on parallel hardware. For example, matrix multiply written in MATLAB is sequential software and the Windows Vista operating system is concurrent software, and either can run on a serial Intel Pentium 4 or on a parallel Intel Core i7.

Flynn's taxonomy classifies parallel hardware by the number of instruction streams and data streams:

	Single data stream	Multiple data streams
Single instruction stream	SISD: Intel Pentium 4	SIMD: SSE instructions of x86
Multiple instruction streams	MISD: no examples today	MIMD: Intel Xeon e5345 (Clovertown)

- SISD: a sequential computer that exploits no parallelism in either the instruction or the data streams, such as a traditional uniprocessor; one processing unit takes one instruction stream and works on one data stream.
- SIMD: a computer that applies a single instruction stream to multiple data streams, for operations that may be naturally parallelized, such as SIMD instruction extensions or a graphics processing unit (GPU).
- MIMD: multiple autonomous processors simultaneously executing different instructions on different data, such as multicore processors and warehouse-scale computers.
- MISD: multiple instruction streams against a single data stream (certain kinds of array processors); no longer commonly encountered and mainly of historical interest.

SIMD and MIMD are the most common today. The most common parallel programming style is single program multiple data (SPMD): a single program runs on all processors of an MIMD

machine, and execution is coordinated across processors through conditional expressions. SIMD uses specialized function units for lock-step calculations on arrays, in scientific computing, machine learning, signal processing, and multimedia (audio and video processing).

32.3 SIMD architectures

Data-level parallelism (DLP) executes one operation on multiple data streams, for example multiplying a coefficient vector by a data vector, as in filtering: $y[i] := c[i] \times x[i]$ for $0 \le i < n$. SIMD improves performance because one instruction is fetched and decoded for the entire operation, the multiplications are known to be independent, and memory accesses can also be pipelined or concurrent. A packed SIMD instruction applies an operation to every lane of its source registers at once (Figure 62). The first SIMD extensions appeared in the MIT Lincoln Labs TX-2 in 1957, which could treat its 36-bit word as one 36-bit operand, two 18-bit operands, a 27-bit and a 9-bit operand, or four 9-bit operands.

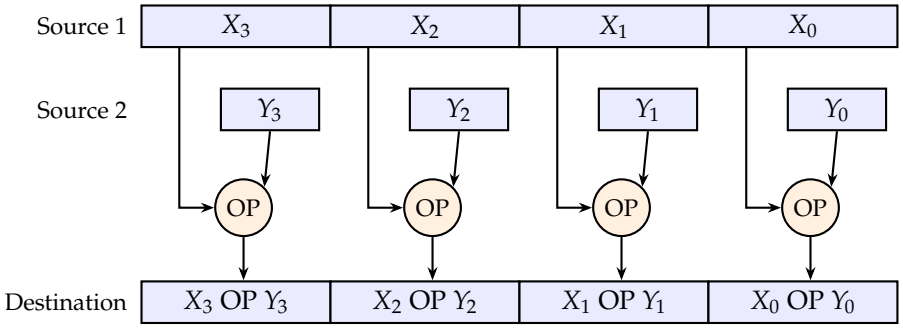


Figure 62: A packed SIMD instruction: one instruction applies OP to the four lanes of two source registers and writes four results.

Intel's SIMD instructions fetch one instruction and do the work of several. They started with MMX (MultiMedia eXtension, Pentium II processor family), followed by SSE (Streaming SIMD Extension, Pentium III and beyond) and later AVX, adding new instructions every few years, with new and wider registers (64, 128, 256, and 512 bits) and more parallelism.

SSE extended the architecture with eight 128-bit data registers, `XMM0`–`XMM7`; the 64-bit architecture has 16 of them (adding `XMM8`–`XMM15`). A 128-bit register can hold 16 packed bytes, 8 packed words, 4 packed doublewords, or 2 packed quadwords. In the Intel architecture, unlike RISC-V, a word is 16 bits, so a single-precision float is a doubleword (32 bits) and a double-precision float is a quadword (64 bits); the 128-bit packed single-precision type allows four single-precision operations at once. AVX-512 has 32 registers `zmm0`–`zmm31` of 512 bits; the 256-bit AVX registers `ymm0`–`ymm15` are the lower halves of the first 16, and the SSE registers are their lower quarters. It adds mask registers `k0`–`k7`, and it has 8× the register state of SSE, commensurate with its 8× FLOP rate. On Linux, `lscpu` lists the supported SIMD extensions (`sse`, `sse2`, `avx2`, `avx512f`, ...) among the CPU flags.

Example: To take the square root of every element of an array, a SISD loop loads each element into

a floating-point register, computes the square root, and writes the result back to memory. A SIMD loop instead loads 4 elements into an SSE register, computes 4 square roots in one operation, and writes the 4 results back.

Example: Adding two vectors of four single-precision floats (vec_res.x = v1.x + v2.x, and likewise for `y`, `z`, `w`) takes three SSE instructions:

```
movaps address-of-v1, %xmm0       # v1.w | v1.z | v1.y | v1.x -> xmm0
addps  address-of-v2, %xmm0       # v1.w+v2.w | ... | v1.x+v2.x -> xmm0
movaps %xmm0, address-of-vec_res
```

`movaps` moves from memory to an XMM register (memory aligned, packed single precision), `addps` adds from memory to an XMM register (packed single precision), and the final `movaps` moves the XMM register back to memory.

32.4 Intel SSE intrinsics

Intrinsics are C functions and procedures that insert assembly language, including SSE instructions, so these instructions can be used indirectly from C; there is a one-to-one correspondence between SSE instructions and intrinsics. For packed doubles: the vector data type is `__m128d`; `_mm_load_pd` and `_mm_store_pd` correspond to `MOVAPD` (aligned, packed double); `_mm_add_pd` to `ADDPD`; and `_mm_mul_pd` to `MULPD`. In addition, `_mm_load1_pd` loads one double and stores it in both the high and the low halves of an XMM register.

Example: Multiply 2×2 matrices stored in column order, with $A = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$ and $B = \left(\begin{smallmatrix} 1 & 3 \\ 2 & 4 \end{smallmatrix}\right)$. Writing a vector register as [low | high], `c1` holds the first column of C, $[C_{11} \mid C_{21}]$, and `c2` the second, $[C_{12} \mid C_{22}]$. In iteration i of the loop (counting from 0), `a` holds column $i+1$ of A, `b1` holds $B_{i+1,1}$ in both halves, and `b2` holds $B_{i+1,2}$ in both halves; one parallel multiply and one parallel add then update both entries of each column:

$$\begin{aligned} i = 0: &\quad \texttt{c1} = [C_{11} + A_{11}B_{11} \mid C_{21} + A_{21}B_{11}], \quad \texttt{c2} = [C_{12} + A_{11}B_{12} \mid C_{22} + A_{21}B_{12}], \\ i = 1: &\quad \texttt{c1} \mathrel{+}= [A_{12}B_{21} \mid A_{22}B_{21}], \quad \texttt{c2} \mathrel{+}= [A_{12}B_{22} \mid A_{22}B_{22}]. \end{aligned}$$

```
#include <stdio.h>
#include <emmintrin.h>  // header file for SSE compiler intrinsics

int main(void) {
  // allocate A, B, C aligned on 16-byte boundaries (column order)
  double A[4] __attribute__ ((aligned (16))) = {1.0, 0.0, 0.0, 1.0};
  double B[4] __attribute__ ((aligned (16))) = {1.0, 2.0, 3.0, 4.0};
  double C[4] __attribute__ ((aligned (16))) = {0.0, 0.0, 0.0, 0.0};
  int lda = 2;
  __m128d c1, c2, a, b1, b2;  // 128-bit vector variables
```

```
  c1 = _mm_load_pd(C+0*lda);  // c1 = [c_11 | c_21]
  c2 = _mm_load_pd(C+1*lda);  // c2 = [c_12 | c_22]
  for (int i = 0; i < 2; i++) {
    a  = _mm_load_pd(A+i*lda);     // i=0: [a_11 | a_21]; i=1: [a_12 | a_22]
    b1 = _mm_load1_pd(B+i+0*lda);  // i=0: [b_11 | b_11]; i=1: [b_21 | b_21]
    b2 = _mm_load1_pd(B+i+1*lda);  // i=0: [b_12 | b_12]; i=1: [b_22 | b_22]
    c1 = _mm_add_pd(c1, _mm_mul_pd(a, b1));
    c2 = _mm_add_pd(c2, _mm_mul_pd(a, b2));
  }
  // store c1, c2 back into C for completion
  _mm_store_pd(C+0*lda, c1);
  _mm_store_pd(C+1*lda, c2);
  printf("%g,%g\n%g,%g\n", C[0], C[2], C[1], C[3]);  // print C
  return 0;
}
```

The program prints `1,3` and `2,4`, the rows of $C = B$.

RISC-V has a draft vector extension, the V extension, which likewise adds SIMD instructions and hardware so that one fetched instruction does the work of many. Vector instructions carry the prefix `v` and operate on vector registers; for example `vadd vd, vs1, vs2` adds two vectors stored in vector registers, which might be 512 bits wide.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 32 (Fall 2020).

Lecture 33: Thread-Level Parallelism I

33.1 Improving performance

There are four ways to improve performance:

1. increase the clock rate f_s, which has reached a practical maximum for today's technology ($< 5\,\text{GHz}$ for general-purpose computers);
2. lower the CPI (cycles per instruction) with SIMD and "instruction-level parallelism";
3. perform multiple tasks simultaneously on multiple CPUs, each executing a different program; the tasks may be related (each CPU performs part of a big matrix multiplication) or unrelated (different web requests distributed over different computers, or a presentation viewer and a browser running at the same time);
4. do all of the above: high f_s, SIMD, and multiple parallel tasks.

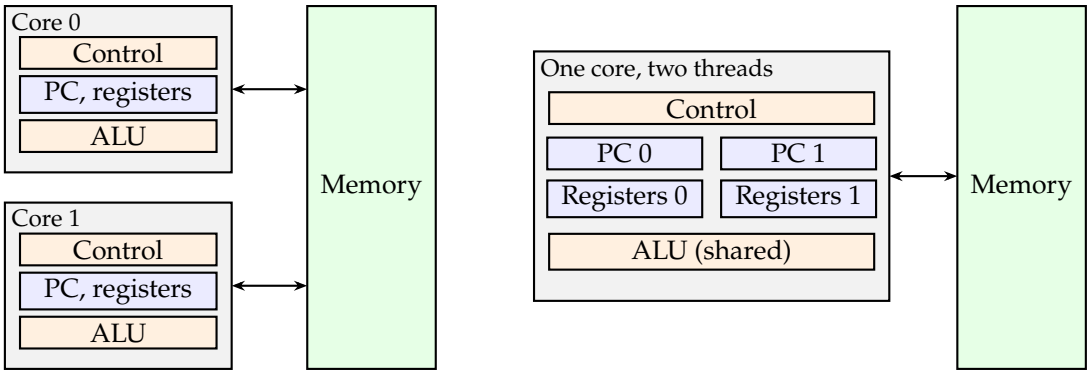
This unit is about the third.

33.2 Multicore processors

Parallel computer architectures range from a massive array of computers with fast communication between processors, through several separate computers with some means of communication (such as Ethernet), to GPUs and multicore CPUs: several datapaths on a single chip sharing the L3 cache, memory, and peripherals.

Definition: In the multiprocessor execution model each processor (core) executes its own instructions. Each core has separate resources: its datapath (PC, registers, ALU) and its highest-level caches (the first and second levels). Memory (DRAM) and often the third-level cache are shared, often on the same silicon chip, though that is not a requirement. Such a chip is a "multiprocessor microprocessor" or multicore processor; a four-core CPU executes four different instruction streams simultaneously.

Figure 63a shows two cores sharing one memory. Over the decades transistor counts kept growing exponentially while frequency, typical power, and sequential application performance leveled off, and the number of cores began to grow: the shift to multicore. The Apple A14 chip, for example, has 11.8 billion transistors in a 5 nm process, a 6-core CPU, a 4-core GPU, and a 16-core neural engine performing 11 trillion operations per second.



(a) two cores (b) hardware multithreading

Figure 63: (a) A CPU with two cores, each with its own control, PC, registers, and ALU, sharing memory. (b) One core with two hardware threads: two PCs and two register files share one datapath.

With SHARED MEMORY, each core has access to the entire memory, and special hardware keeps the caches consistent. Shared memory simplifies communication in a program through shared variables, but it does not scale well: the "slow" memory is shared by many "customers" (cores) and may become a bottleneck (Amdahl's law). There are two ways to use a multiprocessor: JOB-LEVEL PARALLELISM, in which processors work on unrelated problems with no communication between programs, and partitioning the work of a single task between several cores, for example each computing part of a large matrix multiplication.

Parallel processing is difficult, but it is inevitable: it is the only path to higher performance and to lower energy consumption (longer battery life). Mobile systems such as smartphones and tablets have multiple cores and dedicated processors (motion, image, and neural processors, and a GPU). Warehouse-scale computers have multiple "nodes", boxes with several CPUs and disks each, with MIMD (multicore) and SIMD (such as AVX) parallelism in each node.

Example: If software can use it, the potential parallel performance grows quickly: the number of cores grows by 2 every 2 years and the SIMD width doubles every 4 years. With 64-bit operations, FLOPs per cycle are cores × SIMD bits/64:

Year	Cores	SIMD bits/core	Cores × SIMD bits	FLOPs/cycle
2003	2	128	256	4
2005	4	128	512	8
2007	6	128	768	12
2009	8	128	1024	16
2011	10	256	2560	40
2013	12	256	3072	48
2015	14	512	7168	112
2017	16	512	8192	128
2019	18	1024	18432	288
2021	20	1024	20480	320

From 2009 to 2021 the cores grow 2.5× and the SIMD width 8×, so the total grows 20× in 12 years: $20^{1/12} \approx 1.28$, which is 28% per year, or 2× every 3 years.

33.3 Threads

A typical laptop runs well over a hundred processes at once (156 in one `ps -x` listing). It does so with threads.

Definition: A THREAD ("thread of execution") is a single stream of instructions, a sequential flow of instructions that performs some task (until now simply called a "program"). A program or process can split, or FORK, itself into separate threads, which can (in theory) execute simultaneously. Each thread has a dedicated PC (program counter) and separate registers, and accesses the shared memory.

Threads are an easy way to describe and think about parallelism. With a single core, one CPU can execute many threads by TIME SHARING, running each for a while in turn. Each physical core provides one or more HARDWARE THREADS that actively execute instructions. The operating system multiplexes multiple SOFTWARE THREADS onto the available hardware threads; all threads except those mapped to hardware threads are waiting. As Edward A. Lee (UC Berkeley, 2006) put it, threads discard the most essential and appealing properties of sequential computation (understandability, predictability, and determinism); they are wildly non-deterministic, and the programmer's job becomes pruning that nondeterminism.

The operating system gives the illusion of many "simultaneously" active threads:

1. it multiplexes software threads onto hardware threads, switching out blocked threads (on a cache miss, user input, or network access) and switching on a timer (for example, every 1 ms);
2. it removes a software thread from a hardware thread by interrupting its execution and saving its registers and PC to memory;
3. it starts executing a different software thread by loading its previously saved registers into a hardware thread's registers and jumping to its saved PC.

The THREAD POOL is the list of threads competing for the processor; the OS maps threads to cores and schedules the logical (software) threads, and each core actively runs one instruction stream at a time.

33.4 Multithreading

In a typical scenario the active thread encounters a cache miss and waits about 1000 cycles for data from DRAM, so the processor should switch it out and run a different thread until the data is available. But a switch must save the current thread's state and load the new thread's state (the PC

and all registers, which could be many, such as the AVX registers), and this must take much less than 1000 cycles. Hardware can help, since Moore's law makes transistors plentiful.

In HARDWARE-ASSISTED MULTITHREADING the processor holds two copies of the PC and the registers (Figure 63b), so one core looks identical to two processors to software (hardware thread 0 and hardware thread 1).

Definition: SIMULTANEOUS MULTITHREADING (SMT), which Intel calls HYPER-THREADING, runs multiple threads at the same time on one core, so that there are more logical CPUs than physical CPUs. Each thread has its own architectural state (PC, registers, and so on), and the threads share the core's resources (cache, instruction unit, execution units). Without SMT, each cycle's execution resources are filled from one thread; with SMT, resources left idle by one thread can be used by the other, which raises throughput.

Logical threads cost about 1% more hardware for perhaps 10% better performance: the registers are separate, but the datapath, ALUs, and caches are shared. Multicore duplicates processors, which costs about 50% more hardware for perhaps 2× better performance. Modern machines do both, with multiple cores and multiple threads per core. A laptop reporting `hw.physicalcpu: 4` and `hw.logicalcpu: 8` has 4 cores and 8 threads in total; the Intel Xeon W-3275M has 28 cores and 56 threads (2.5 GHz base and 4.4 GHz maximum turbo frequency, a 38.5 MB cache, and a 205 W thermal design power). With 6 cores and 4 logical threads per core, the OS schedules software threads from the pool onto 24 logical threads.

To summarize the terms: a THREAD is a sequence of instructions with its own program counter and processor state (such as a register file). In a multicore, a PHYSICAL CPU runs one thread at a time, and the OS switches threads in software, typically in response to I/O events such as disk reads and writes; a LOGICAL CPU switches threads at a fine grain, in hardware, when a thread blocks on a cache miss or memory access; and hyper-threading (SMT) exploits a superscalar architecture to launch instructions from different threads at the same time.

Sequential execution speed is limited, since clock rates are flat or declining, so parallelism is the only path to higher performance. SIMD is implemented in all high-performance CPUs today (x86, ARM, . . .), is partially supported by compilers, and doubles in width every 3–4 years. MIMD, thread-level parallelism, comes from multicore processors, which are supported by operating systems but require programmer intervention to exploit within a single program, and add 2 cores every 2 years. The key challenge is to write parallel programs that keep high performance as the number of processors increases, that is, programs that scale; the obstacles are scheduling, load balancing, time for synchronization, and communication overhead.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 33 (Fall 2020).

Lecture 34: Thread-Level Parallelism II

34.1 Parallel programming languages

There are dozens of languages that support parallel programming (Ada, Cilk, Chapel, CUDA, Erlang, Go, Java, Occam, Scala, VHDL, and many more). The number of choices shows that there is no universal solution: needs are very problem specific. Scientific computing and machine learning need fast matrix multiplication, a web server must handle many unrelated requests simultaneously, and in input and output everything happens at once. Specialized languages for different tasks are easier to use for some problems, but none is particularly easy. Compilers keep adding SIMD features (Intel, gcc), but translating C into good, fast parallel code is an intense area of research: it took more than 20 years to translate C into good, fast assembly, the general problem is very hard, and the present state is specialized solutions for specific cases (hence intrinsics). For high-performance computing, a widely used example is OpenMP.

34.2 OpenMP

A serial loop `for (int i=0; i<100; i++)` can be executed in parallel by splitting its iterations into chunks, for example 0–24, 25–49, 50–74, and 75–99, each run by a different thread. OpenMP does this with a single directive:

```
#include <omp.h>
#pragma omp parallel for
for (int i=0; i<100; i++) {
  ...
}
```

Definition: OpenMP is an extension of C (no new language to learn) for multi-threaded, shared-memory parallelism. It consists of compiler directives (`#pragma`) and runtime library routines (`#include <omp.h>`). Compilers unaware of OpenMP ignore the pragmas, so the same source works on multiple architectures (for example, the same program for 1 and 16 cores). It works only with shared memory.

OpenMP follows the fork-join model. A program begins as a single process, the master thread, executing sequentially. When it reaches a parallel region, the master thread forks into a team of parallel threads that execute simultaneously; at the end of the region the threads join, leaving only the master thread. The process repeats for each parallel region, and the sequential parts between regions limit the speedup (Amdahl's law). OpenMP threads are operating system (software) threads, which the OS multiplexes onto the available hardware threads. Ideally each gets a real hardware thread, so there is no OS-level time multiplexing, but other tasks on the machine compete

for hardware threads, so timing results must be interpreted with care.

Example: The program

```c
#include <stdio.h>
#include <omp.h>
int main()
{
  omp_set_num_threads(4);
  int a[] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
  int N = sizeof(a)/sizeof(int);

  #pragma omp parallel for
  for (int i=0; i<N; i++) {
    printf("thread %d, i = %2d\n",
      omp_get_thread_num(), i);
    a[i] = a[i] + 10 * omp_get_thread_num();
  }

  for (int i=0; i<N; i++) printf("%02d ", a[i]);
  printf("\n");
}
```

compiled with `gcc -fopenmp` uses 4 threads. Thread 0 ran iterations 0–2, thread 1 iterations 3–5, thread 2 iterations 6–7, and thread 3 iterations 8–9, with the `printf` lines from different threads interleaved, and the final array is `00 01 02 13 14 15 26 27 38 39`. The maximum number of threads available to do work is returned by `omp_get_max_threads()`.

34.3 Example: computing π

Since $\int_0^1 4/(1+x^2)\,dx = \pi$, the number π can be approximated by the midpoint rule with n steps of width $\Delta = 1/n$: $\pi \approx \sum_{i=0}^{n-1} 4\Delta/(1+x_i^2)$ with $x_i = (i+0.5)\Delta$. Sequentially:

```c
#include <stdio.h>

void main () {
  const long num_steps = 10;
  double step = 1.0/((double)num_steps);
  double sum = 0.0;
  for (int i=0; i<num_steps; i++) {
    double x = (i+0.5)*step;
    sum += 4.0*step/(1.0+x*x);
  }
  printf ("pi = %6.12f\n", sum);
}
```

prints `pi = 3.142425985001`, which resembles $\pi = 3.14159265358979\ldots$ but is not very accurate. Simply putting `#pragma omp parallel for` before the loop does not work: every thread needs access to the shared variable `sum`. Instead, each thread accumulates its own partial sum in `sum[id]`, in parallel, and the partial sums are then added sequentially:

```
#include <stdio.h>
#include <omp.h>

void main () {
  const int NUM_THREADS = 4;
  const long num_steps = 10;
  double step = 1.0/((double)num_steps);
  double sum[NUM_THREADS];
  for (int i=0; i<NUM_THREADS; i++) sum[i] = 0;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int id = omp_get_thread_num();
    for (int i=id; i<num_steps; i+=NUM_THREADS) {
      double x = (i+0.5)*step;
      sum[id] += 4.0*step/(1.0+x*x);
    }
  }
  double pi = 0;
  for (int i=0; i<NUM_THREADS; i++) pi += sum[i];
  printf ("pi = %6.12f\n", pi);
}
```

`#pragma omp parallel` runs the block in every thread, and `omp_get_thread_num()` identifies the thread, so thread `id` handles iterations `id`, `id+4`, `id+8`, ... A trial run printing `i` and `id` in the loop shows thread 1 handling 1, 5, 9, thread 0 handling 0, 4, 8, thread 2 handling 2, 6, and thread 3 handling 3, 7, in an unpredictable interleaving, and the result is again `pi = 3.142425985001`. With `num_steps = 1000000` the program prints `pi = 3.141592653590`, correct to all 12 printed decimal places.

34.4 Data races

To beat Amdahl's law, one might also move the final summation into the parallel region, with each thread executing `pi += sum[id];` after its loop. With 1000 threads and 100,000 steps this gives only an insignificant speedup, and the result is wrong (`pi = 3.138450662641`) and changes between runs. The operation is really `pi = pi + sum[id]`: if more than one thread reads the current (same) value of `pi`, computes its sum, and stores the result back, each overwrites the others' updates. The result depends on who gets there when; this is a RACE, and the result is not deterministic.

34.5 Synchronization and locks

The problem is to limit access to a shared resource to one actor at a time, as when only one person may edit a file at a time or only one person holding the microphone may talk. Computers use LOCKS (also called SEMAPHORES) to control access to shared resources. A lock is usually a variable, `int lock;`, that is 0 when unlocked and 1 when locked:

```
// wait for lock released
while (lock != 0) ;
// lock == 0 now (unlocked)

// set lock
lock = 1;
  // access shared resource ...
  // e.g. pi
  // sequential execution! (Amdahl ...)
// release lock
lock = 0;
```

This does not work. Thread 1 and thread 2 can both execute `while (lock != 0) ;` and find the lock not set before either sets it; both then set the lock and enter the critical section, each believing it holds the lock. No arrangement of ordinary loads and stores solves this problem, even at the assembly level; it needs new instructions.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 34 (Fall 2020).

Lecture 35: Thread-Level Parallelism III

35.1 OpenMP loops and data races

The OpenMP `parallel for` pragma breaks a `for` loop such as `for (i=0; i<max; i++) zero[i] = 0;` into chunks and allocates each to a separate thread: with `max` = 100 and 2 threads, thread 0 gets iterations 0–49 and thread 1 gets 50–99. The loop must have a relatively simple "shape" for an OpenMP-aware compiler to parallelize it, because the run-time system must be able to determine how many iterations to assign to each thread. No premature exits from the loop are allowed (no `break`, `return`, `exit`, or `goto`); in general, never jump outside a pragma block.

Definition: Two memory accesses form a DATA RACE if they come from different threads, access the same location, at least one is a write, and they occur one after another.

If there is a data race, the result of the program varies depending on chance (which thread goes first). Data races are avoided by synchronizing writing and reading to get deterministic behavior; synchronization is done by user-level routines that rely on hardware synchronization instructions.

35.2 Hardware synchronization

The solution is an atomic read and write: a read and a write in a single instruction, with no other access permitted between them. This requires shared memory (multiprocessing). Common implementations are an atomic swap of a register with memory, and a pair of instructions for a "linked" read and write, where the write fails if the memory location has been "tampered" with after the linked read. RISC-V has variations of both; here we use the former.

Definition: RISC-V ATOMIC MEMORY OPERATIONS (AMOs) atomically perform an operation on an operand in memory and set the destination register to the original memory value. They are R-type instructions with the operations add, and, or, swap, xor, max, max unsigned, min, and min unsigned. For example, `amoadd.w rd, rs2, (rs1)` performs

$$t = M[x[rs1]]; \quad x[rd] = t; \quad M[x[rs1]] = t + x[rs2].$$

The `aq` (acquire) and `rl` (release) bits ensure in-order execution.

The instruction format is `funct5` (operation, bits 31–27), `aq` (bit 26), `rl` (bit 25), `rs2` (source, bits 24–20), `rs1` (address, bits 19–15), `funct3` (width, bits 14–12), `rd` (destination, bits 11–7), and the AMO opcode (bits 6–0).

Example: Assume the lock is in the memory location whose address is in register `a0`; the lock is "set" if it is 1 and "free" if it is 0 (its initial value). A critical section is

```
      li           t0, 1        # Get 1 to set lock
Try:  amoswap.w.aq t1, t0, (a0) # t1 gets old lock value
                                # while we set it to 1
      bnez         t1, Try      # if it was already 1, another
                                # thread has the lock,
                                # so we need to try again
      # ... critical section goes here ...
      amoswap.w.rl x0, x0, (a0) # store 0 in lock to release
```

Unlike the broken software lock (`while (lock != 0) ;` followed by `lock = 1;`), the swap reads the old value and sets the lock in one indivisible step, so two threads can never both see the lock free and both enter the critical section.

35.3 Synchronization in OpenMP

Hardware synchronization instructions are typically used in libraries of higher-level parallel programming constructs. OpenMP provides locks:

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(void) {
  omp_lock_t lock;
  omp_init_lock(&lock);
  #pragma omp parallel
  {
    int id = omp_get_thread_num();
    // parallel section
    // ...
    omp_set_lock(&lock);
    // start sequential section
    // ...
    printf("id = %d\n", id);
    // end sequential section
    omp_unset_lock(&lock);
    // parallel section
    // ...
  }
  omp_destroy_lock(&lock);
}
```

OpenMP also offers pragmas for common cases (`critical`, `atomic`, `barrier`, `ordered`) and many more features, such as private variables and reductions.

Definition: A CRITICAL SECTION provides mutual exclusion: only one thread at a time can enter a `critical` region, and the other threads wait their turn.

This fixes the racy π program: in the parallel region each thread computes its partial sum `sum[id]` and then executes

```
#pragma omp critical
  pi += sum[id];
```

so the updates of the shared variable `pi` no longer interfere.

Definition: DEADLOCK is a system state in which no progress is possible.

In the DINING PHILOSOPHERS PROBLEM, philosophers sit around a table with one fork between each pair. Each thinks until the left fork is available and picks it up, thinks until the right fork is available and picks it up, eats for a fixed time once both forks are held, puts the right fork down, then the left, and repeats. If every philosopher picks up the left fork at the same time, each waits forever for the right one: a deadlock.

To time OpenMP code, `double omp_get_wtime(void);` returns the elapsed wall-clock time in seconds. The time is measured per thread, and there is no guarantee that two distinct threads measure the same time. It is measured from "some time in the past", so the elapsed time is the difference of two calls to `omp_get_wtime`.

35.4 Shared memory and caches

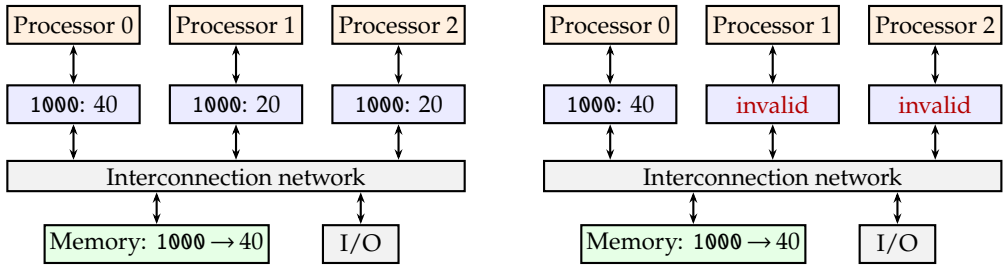
Definition: A SYMMETRIC MULTIPROCESSOR (SMP), or shared-memory multiprocessor, has two or more identical CPUs (cores) and a single shared coherent memory.

A multiprocessor raises three key questions: how do the processors share data, how do they coordinate, and how many processors can be supported? In an SMP, all processors (cores) share a single address space, and they coordinate and communicate through shared variables in memory, via loads and stores. Use of shared data must be coordinated through synchronization primitives (locks) that allow access to the data to only one processor at a time. All multicore computers today are SMPs.

Memory is a performance bottleneck even with one processor, so caches are used to reduce the bandwidth demands on main memory: each core has a local private cache holding data it has accessed recently, and only cache misses have to access the shared memory over the interconnection network. But suppose processors 1 and 2 read `Memory[1000]` (value 20), so both caches hold 20, and then processor 0 writes 40 to `Memory[1000]`. Processor 0's cache and memory now hold 40, while the caches of processors 1 and 2 still hold the stale value 20 (Figure 64a).

35.5 Cache coherence

The architect's job is to make shared memory work by keeping cache values COHERENT. When any processor has a cache miss or writes, it notifies the other processors through the interconnection network. If processors only read, many can have copies; if a processor writes, it invalidates any other copies. On write transactions from one processor, the other caches SNOOP the common interconnect, checking for tags they hold, and invalidate any copies of the same address modified in another cache (Figure 64b).



(a) without coherence: stale copies

(b) the write invalidates other copies

Figure 64: Processors 1 and 2 have read `Memory[1000]` = 20 and processor 0 writes 40. (a) Without coherence, two caches keep the stale value. (b) With a snooping protocol, the write invalidates the other copies.

Each cache tracks the state of each block it holds:

1. SHARED: up-to-date data; other caches may have a copy.
2. MODIFIED: up-to-date data, changed (dirty); no other cache has a copy, it is OK to write, and memory is out of date (write back).
3. EXCLUSIVE (optional): up-to-date data; no other cache has a copy, it is OK to write, and memory is up to date. This avoids writing to memory if the block is replaced, and the cache supplies the data on a read instead of going to memory.
4. OWNER (optional): up-to-date data; other caches may have a copy, and they must be in the Shared state. This cache is one of several with a valid copy, but it has the exclusive right to change the line and must broadcast those changes to all other caches sharing it. The Owned state allows dirty sharing: a modified block can move among caches without updating main memory. An Owned line may become Modified after invalidating all shared copies, or Shared by writing the modifications back to memory, and it must respond to a snoop request with data.

Exclusive and Owner are performance optimizations. Together with INVALID (not in the cache) these states form the common snooping protocol MOESI, such as the Berkeley Ownership Protocol.

Two caches may hold the same block in the following pairs of states:

	M	O	E	S	I
M	no	no	no	no	yes
O	no	no	no	yes	yes
E	no	no	no	no	yes
S	no	yes	no	yes	yes
I	yes	yes	yes	yes	yes

35.6 False sharing and coherence misses

Coherence is tracked by cache block. Suppose the block size is 32 bytes, processor 0 reads and writes a variable `X` at address 4000, and processor 1 reads and writes a variable `Y` at address 4012. Both variables lie in the same block (4000–4031), so every write by one processor invalidates the other's copy, and the block PING-PONGS between the two caches even though the processors access disjoint variables. This effect is called FALSE SHARING; it is avoided by placing data used by different processors in different blocks, so the block size must be watched.

Recall the three Cs of cache misses: COMPULSORY misses (cold start or process migration; the first access to a block, impossible to avoid and a small effect for long-running programs; reduced by larger blocks, which increase the miss penalty and, if very large, the miss rate), CAPACITY misses (the cache cannot hold all blocks accessed by the program even with perfect replacement in a fully associative cache; reduced by a larger cache, which may increase access time), and CONFLICT misses (multiple memory locations map to the same cache location; reduced by a larger cache, more associativity, which may increase access time, or a better replacement policy such as LRU). Multiprocessors add a fourth C: COHERENCE MISSES, caused by coherence traffic with other processors. They are also called communication misses because they represent data moving between processors working together on a parallel program, and for some parallel programs they dominate the total misses.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 35 (Fall 2020).

Lecture 36: MapReduce, Spark

36.1 Amdahl's law

Theorem 36.1 [Amdahl's law]. *Suppose an enhancement E does not affect a portion s ($s < 1$) of a task and accelerates the remainder $1 - s$ by a factor P ($P > 1$). Then*

$$\text{exec time with } E = \text{exec time without } E \times \left[s + \frac{1-s}{P} \right],$$
$$\text{speedup with } E = \frac{1}{s + \frac{1-s}{P}} \leq \frac{1}{s},$$

and the bound is approached as $P \to \infty$.

Here s is the non-sped-up part and $(1 - s)/P$ the sped-up part. For example, if 4/5 of a program's execution time can be accelerated by a factor of 16, the overall speedup is

$$\frac{1}{0.2 + \frac{0.8}{16}} = \frac{1}{0.2 + 0.05} = \frac{1}{0.25} = 4.$$

The consequence is that the speedup achievable through parallelism is limited by the serial portion s of the program: speedup $\leq 1/s$. As processors are added, the parallel portion of the execution time shrinks but the serial portion does not, so the speedup curve levels off at $1/s$: for parallel portions of 50%, 75%, 90%, and 95% the speedup levels off at 2, 4, 10, and 20, even with tens of thousands of processors.

36.2 Request-level and data-level parallelism

Definition: Request-level parallelism (RLP) handles hundreds or thousands of requests per second. These are not requests on a laptop or a phone but on popular Internet services such as web search and social networking. Such requests are largely independent: they often involve read-mostly databases and rarely involve strict read-write data sharing or synchronization across requests. Computation is easily partitioned both within a request and across different requests.

In Google's query-serving architecture, for example, a Google web server receives a query and fans it out to many index servers and document servers, and also consults a spell checker and an ad server.

Data-level parallelism (DLP) comes in two kinds: lots of data in memory that can be operated on in parallel (such as adding two arrays), and lots of data on many disks that can be operated on in parallel (such as searching for documents). The second kind, data-level parallelism across many

servers and disks, is what MapReduce provides.

36.3 MapReduce

Definition: MapReduce is a simple data-parallel programming model designed for scalability and fault tolerance. It was pioneered by Google, which processes more than 25 petabytes of data per day with it, and the open-source Hadoop project implements it (used at Yahoo!, Facebook, Amazon, . . .).

At Google, MapReduce has been used for index construction for Google Search, article clustering for Google News, statistical machine translation, and computing multi-layer street maps; at Yahoo! for the "web map" powering Yahoo! Search and spam detection for Yahoo! Mail; and at Facebook for data mining, ad optimization, and spam detection (for example Facebook Lexicon, which plotted how often phrases such as "party tonight" and "hangover" appeared over time).

The design goals (Dean and Ghemawat, OSDI 2004) are scalability to large data volumes (1000s of machines and 10,000s of disks) and cost efficiency: commodity machines (cheap but unreliable), a commodity network, automatic fault tolerance through re-execution (fewer administrators), and ease of use (fewer programmers). A typical Hadoop cluster has 40 nodes per rack and 1000–4000 nodes in the cluster, with 1 Gbps of bandwidth within a rack and 8 Gbps out of the rack (each rack switch connects to an aggregation switch); a node in Yahoo's terasort setup had 8 cores at 2 GHz, 8 GB of RAM, and 4 disks.

The idea comes from functional programming: in Scheme, `(reduce + (map square '(1 20 3 10)))` squares each input to get 1, 400, 9, 100 and sums them pairwise (1 + 400 = 401, 9 + 100 = 109, 401 + 109 = 510) to get 510; Python's `reduce(plus, map(square, (1,20,3,10)))` does the same.

Definition: In the MapReduce programming model the input and the output are each a set of key/value pairs, and the programmer specifies two functions:

- map(*in_key*, *in_value*) → list(*interm_key*, *interm_value*) processes an input key/value pair and produces a set of intermediate pairs; the data is sliced into "shards" or "splits" that are distributed to workers;
- reduce(*interm_key*, list(*interm_value*)) → list(*out_value*) combines all intermediate values for a particular key and produces a set of merged output values (usually just one).

Example: Word counting uses "mapper" nodes for the map function and "reducer" nodes for the reduce function, with the data on a distributed file system (DFS). The mapper turns `"I do I learn"` into `("I",1), ("do",1), ("I",1), ("learn",1)`, and the reducer turns `("I",[1,1])` into `("I",2)`:

```
map(String input_key, String input_value):
  // input_key  : document name (or line of text)
  // input_value: document contents
  for each word w in input_value:
    EmitIntermediate(w, "1");

reduce(String output_key, Iterator intermediate_values):
  // output_key   : a word
  // output_values: a list of counts
  int result = 0;
  for each v in intermediate_values:
    result += ParseInt(v);
  Emit(AsString(result));
```

Figure 65 runs it on seven small files.

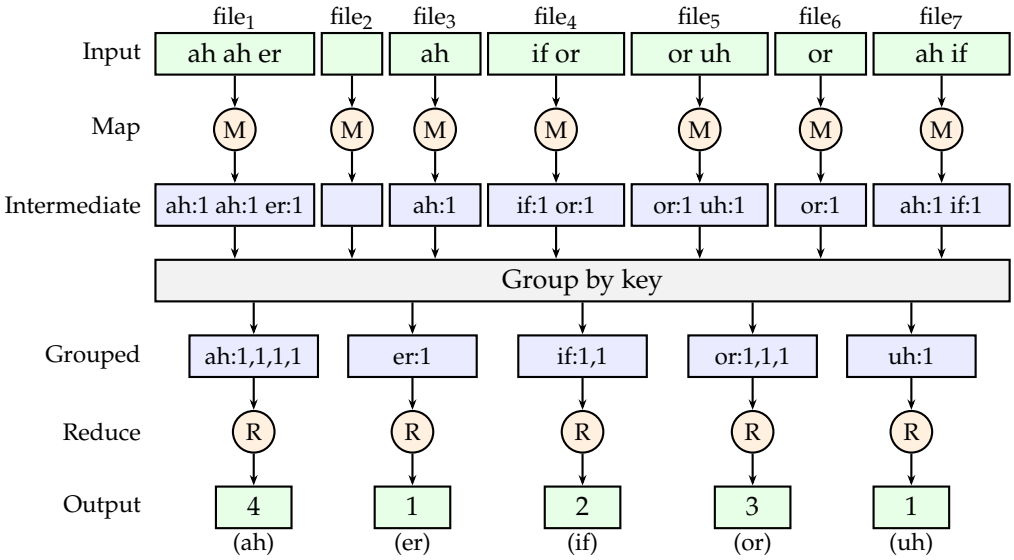


Figure 65: MapReduce word count on seven files ($file_2$ is empty): each map task emits a pair (word, 1) per word, the pairs are grouped by key, and each reduce task sums the counts for one word.

A master assigns map and reduce tasks to "worker" servers. As soon as a map task finishes, its worker can be assigned a new map or reduce task. The data shuffle for a map begins as soon as that map finishes, and a reduce task begins as soon as all data shuffles finish. To tolerate faults, the master reassigns a task if a worker server "dies". In Hadoop the word count is written in Java as a job configuration, a mapper class, and a reducer class.

36.4 Spark

Definition: Apache Spark is a fast and general engine for large-scale data processing. It runs programs up to 100× faster than Hadoop MapReduce in memory, or 10× faster on disk, using

an advanced DAG execution engine that supports cyclic data flow and in-memory computing. Applications can be written quickly in Java, Scala, or Python, with over 80 high-level operators, and used interactively from the Scala and Python shells.

Besides `map`, Spark has `flatMap`, which maps each element to a list and concatenates the lists:

```
>>> def neighbor(n):
...   return [n-1,n,n+1]
>>> R = sc.parallelize(range(5))
>>> R.collect()
[0, 1, 2, 3, 4]
>>> R.map(neighbor).collect()
[[-1, 0, 1], [0, 1, 2], [1, 2, 3], [2, 3, 4], [3, 4, 5]]
>>> R.flatMap(neighbor).collect()
[-1, 0, 1, 0, 1, 2, 1, 2, 3, 2, 3, 4, 3, 4, 5]
```

Example: Word count in Spark's Python API is a single expression,

```
file.flatMap(lambda line: line.split())
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a+b)
```

(compare a page of Java for Hadoop). With `file.txt` holding the lines `ah ah er`, `ah`, `if or`, `or uh`, `or`, `ah if`:

```
>>> W = sc.textFile("file.txt")
>>> W.flatMap(lambda line: line.split()).collect()
['ah', 'ah', 'er', 'ah', 'if', 'or', 'or', 'uh', 'or', 'ah', 'if']
>>> W.flatMap(lambda line: line.split()).map(lambda word: (word,1)).collect()
[('ah', 1), ('ah', 1), ('er', 1), ('ah', 1), ('if', 1), ('or', 1),
 ('or', 1), ('uh', 1), ('or', 1), ('ah', 1), ('if', 1)]
>>> W.flatMap(lambda line: line.split()).map(lambda word:
... (word,1)).reduceByKey(lambda a,b: a+b).collect()
[('er', 1), ('ah', 4), ('if', 2), ('or', 3), ('uh', 1)]
```

To check that Spark really runs in parallel, define a function that simulates five seconds of number crunching:

```
>>> def crunch(n):
...     time.sleep(5)  ## to simulate number crunching
...     return n*n
...
>>> crunch(10)                    ## 5 seconds later
100
>>> list(map(crunch,range(4)))    ## 20 seconds later
[0, 1, 4, 9]
>>> R = sc.parallelize(range(4))
>>> R.map(crunch).collect()       ## 5 seconds later
[0, 1, 4, 9]
```

The sequential `map` takes $4 \times 5 = 20$ seconds, while Spark runs the four calls in parallel in 5 seconds. MapReduce is a wonderful abstraction for programming thousands of machines: it hides the details of machine failures and slow machines, and it is file based. Spark does even better: it is memory based and uses lazy evaluation.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 36 (Fall 2020).

Lecture 37: Data Centers, Cloud Computing (WSC)

37.1 Eras of computer hardware

The great ideas of computer architecture are layers of representation and interpretation, Moore's law, the principle of locality and the memory hierarchy, parallelism, performance measurement and improvement, and dependability via redundancy; this unit continues with parallelism. Computer hardware has passed through several eras:

- *mainframes* (1950s–60s): "big iron" from IBM, UNIVAC, and others, $1M computers for businesses, programmed in COBOL and Fortran under timesharing operating systems;
- *minicomputers* (1970s): built from integrated circuits by Digital, HP, and others, $10k computers for labs and universities, with C and the UNIX OS;
- *PCs* (mid 1980s to mid 2000s): built from microprocessors by Apple, IBM, and others, $1k computers for one person, with Basic, Java, and the Windows OS;
- the *PostPC era* (late 2000s onward): personal mobile devices (PMDs) relying on wireless networking, $500 smartphones and tablets from Apple, Nokia, and others (Objective C, Swift, Java, Android and iOS), together with *cloud computing*: using local area networks, Amazon, Google, and others build $200M warehouse-scale computers with 100,000 servers to provide Internet services for PMDs (MapReduce, Ruby on Rails).

37.2 Warehouse-scale computers

Cloud computing took off because of "the Web space race": the build-out of extremely large datacenters (10,000s of commodity PCs), driven by growth in demand (more users) and enabled by infrastructure software and operational expertise. Such datacenters showed an economy of scale: they are 5–7× cheaper than provisioning a medium-sized facility of 1000 servers. More pervasive broadband Internet made remote computers efficient to access, and hardware and software became commodities with standardized software stacks.

Definition: A warehouse-scale computer (WSC) is a massive datacenter of 10,000 to 100,000 servers plus the networks that connect them. WSCs emphasize cost efficiency and pay attention to power distribution and cooling. Their hardware and software are (relatively) homogeneous, they offer very large applications (Internet services such as search, social networking, and video sharing), and they are very highly available (less than 1 hour down per year), so they must cope with the failures that are common at scale.

As Barroso and Hölzle (2009) put it, "WSCs are no less worthy of the expertise of computer systems

architects than any other class of machines". Design goals unique to warehouse scale are:

- *ample parallelism*: batch applications have a large number of independent data sets with independent processing (data-level parallelism);
- *scale and its opportunities and problems*: the relatively small number of WSCs makes design cost expensive and difficult to amortize, but price breaks are possible from purchasing very large numbers of commodity servers, and a WSC must prepare for a high number of component failures;
- *operational costs count*: the cost of equipment purchases is much less than the cost of ownership.

Google's Oregon WSC is an example; some WSCs are built from shipping containers packed with servers. A *server* in rack format is $1\frac{3}{4}$ inches high ("1U"), 19 inches wide, and 16–20 inches deep, with 8 cores, 16 GB of DRAM, and 4×1 TB of disk. A 7-foot *rack* holds 40–80 servers and an Ethernet local area network switch (1–10 Gbps) in the middle (the "rack switch"). An *array* (cluster) is 16–32 server racks plus a larger local area network switch (the "array switch"); a switch 10× faster costs about 100× as much, since switch cost grows quadratically, as $f(N^2)$.

Example: On-demand Amazon EC2 prices in November 2020:

Instance	Name	$/hour	Ratio	ECUs	vCPUs	GiB	Disk
Standard Small	t3.small	0.021	1	variable	2	2	EBS
Standard Large	t3.large	0.083	4	variable	2	8	EBS
Standard 2x Extra Large	t3.2xlarge	0.333	16	variable	8	32	EBS
High-Mem Large	r5.large	0.126	6	10	2	16	EBS
High-Mem Double XLarge	r5.2xlarge	0.504	24	37	8	64	EBS
High-Mem 24x Large	r5.24xlarge	6.048	288	337	96	768	EBS
High-CPU Large	c5.large	0.085	4	10	2	4	EBS
High-CPU 18x Large	c5.18xlarge	3.060	146	281	72	144	EBS

Here "ratio" is the price relative to the smallest instance, ECUs are EC2 compute units, and GiB is memory. The closest match to the server described above is the Standard 2x Extra Large. At these low rates Amazon EC2 can make money, even if servers are utilized only 50% of the time. EBS is the Elastic Block Store (SSD at $0.10/GB-month, HDD at $0.045/GB-month), and each instance can also come with a dedicated attached SSD for an extra fee.

37.3 Performance in a WSC

To say that X is faster than Y can mean two things. A 2009 Ferrari 599 GTB carries 2 passengers a quarter mile in 11.1 seconds (call it 10 s); a 2009 Type D school bus carries 54 passengers and might take a minute.

Definition: Response time or latency is the time between the start and completion of a task (for example, the time to move a vehicle a quarter mile). Throughput or bandwidth is the total amount

of work done in a given time (for example, passenger-miles in one hour).

Example: For servers with 8 cores, 16 GB of DRAM, and 4 TB of disk, 80 servers per rack, and 30 racks per array:

	Local	Rack	Array
Racks	–	1	30
Servers	1	80	2400
Cores (processors)	8	640	19,200
DRAM capacity (GB)	16	1,280	38,400
Disk capacity (TB)	4	320	9,600
DRAM latency (µs)	0.1	100	300
Disk latency (µs)	10,000	11,000	12,000
DRAM bandwidth (MB/s)	20,000	100	10
Disk bandwidth (MB/s)	200	100	10

The latency to DRAM in another server, even across the array (300 µs), is lower than the latency to the local disk (10,000 µs), but the bandwidth to the local disk (200 MB/s) is higher than the bandwidth to DRAM in another server in the rack (100 MB/s).

Workloads also vary: an online service's peak usage, around noon, is about 2× its off-peak usage around midnight. So WSC software must take care where it places data within an array to get good performance, must cope with failures gracefully, and must scale up and down gracefully in response to varying demand. This more elaborate hierarchy of memories, failure tolerance, and workload accommodation makes WSC software development more challenging than software for a single computer.

37.4 Power and power usage effectiveness

As the load on a server varies from idle to 100%, its power does not scale proportionally: a server uses $\frac{1}{2}$ of its peak power when idle, $\frac{2}{3}$ of peak power when 10% utilized, and 90% of peak power at 50% utilization. Most servers in a WSC are utilized 10% to 50% of the time. The goal should be ENERGY PROPORTIONALITY: % of peak load = % of peak energy.

Definition: The overall energy efficiency of a WSC is the amount of computational work performed divided by the total energy used in the process. The POWER USAGE EFFECTIVENESS (PUE) is

$$\text{PUE} = \frac{\text{total building power}}{\text{IT equipment power}}.$$

It measures the power efficiency of the WSC as a facility, not the efficiency of the servers and networking gear; 1.0 is perfection.

A 2007 Lawrence Berkeley National Laboratory survey of 24 datacenters found PUEs ranging up to about 3.0, with an average of 1.83. In a datacenter with a high PUE, the power goes to:

Consumer	Share of building power
Chiller (cools warm water from the air conditioner)	33%
IT equipment (servers and networking)	30%
Uninterruptible power supply (UPS, battery)	18%
Computer room air conditioner (CRAC)	9%
Power distribution unit (PDU)	5%
Lighting	1%
Transformer and switchgear	1%
Humidifier	remaining ≈3%

With only 30% of the power reaching IT equipment, this datacenter's PUE is about $1/0.30 \approx 3.3$.

Google's WSC A has a PUE of 1.24, achieved by

- careful air-flow handling: not mixing hot server exhaust with cold air (separating the warm aisle from the cold aisle), keeping the path to cooling short so little energy is spent moving cold or hot air long distances, and keeping servers inside containers to help control air flow;
- elevated cold-aisle temperatures: 81 °F instead of the traditional 65–68 °F, after finding that reliability is acceptable when servers run hotter;
- free cooling: cooling warm water outside by evaporation in cooling towers, and locating the WSC in a moderate climate, neither too hot nor too cold;
- a per-server 12 V DC UPS: a single battery on each server board instead of a WSC-wide UPS, which increases WSC efficiency from 90% to 99%;
- measuring rather than estimating PUE, publishing it, and improving operation.

In 2011 Google disclosed that it continuously uses enough electricity to power 200,000 homes, with a search costing per day (per person) about as much as running a 60 W bulb for 3 hours; in 2018 it announced that for every kilowatt-hour it consumed during 2017 it had purchased a kilowatt-hour of renewable energy from a wind or solar farm built specifically for Google.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 37 (Fall 2020).

Lecture 38: Dependability: Parity, ECC, RAID

38.1 Dependability via redundancy

Dependability via redundancy is the sixth of the great ideas of computer architecture, after abstraction, Moore's law, the principle of locality and the memory hierarchy, parallelism, and performance measurement and improvement. Computers fail, transiently or permanently; here we consider hardware failures and how to mitigate them. Redundancy keeps a failing piece from making the whole system fail: if three units compute 1 + 1 and one answers 1 while two answer 2, the two that agree outvote the failure. Increasing transistor density reduces the cost of redundancy. Redundancy applies at every level: redundant datacenters, so that one datacenter can be lost while the Internet service stays online; redundant disks, so that a disk can be lost without losing data (Redundant Arrays of Independent Disks, RAID); and redundant memory bits, so that a bit can be lost without losing data (error correcting code, ECC, memory).

38.2 Dependability metrics

Definition: A FAULT is the failure of a component; it may or may not lead to a system failure. A system alternates between SERVICE ACCOMPLISHMENT, in which service is delivered as specified, and SERVICE INTERRUPTION, a deviation from the specified service: a failure moves it from accomplishment to interruption, and a restoration moves it back.

Redundancy can be in space or in time. SPATIAL REDUNDANCY replicates data, check information, or hardware to handle hard and soft (transient) failures; TEMPORAL REDUNDANCY repeats in time (retries) to handle soft failures.

Definition: Reliability is measured by the MEAN TIME TO FAILURE (MTTF) and service interruption by the MEAN TIME TO REPAIR (MTTR). The MEAN TIME BETWEEN FAILURES is MTBF = MTTF + MTTR, and

$$\text{availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}.$$

Availability improves by increasing MTTF (more reliable hardware and software, plus fault tolerance) or by reducing MTTR (better tools and processes for diagnosis and repair). MTTF and MTBF are usually measured in hours, and availability is quoted as a percentage. Since systems are rarely down, the shorthand is the "number of nines" of availability per year:

Nines	1	2	3	4	5
Availability	90%	99%	99.9%	99.99%	99.999%
Repair time per year	36 days	3.6 days	526 minutes	53 minutes	5 minutes

(Each nine divides the downtime by 10: 0.1% of a year is $0.001 \times 365 \times 24 \times 60 \approx 526$ minutes.)

Example: Another reliability measure is the ANNUALIZED FAILURE RATE (AFR), the average number of failures per year. For 1000 disks with a 100,000-hour MTTF, a year has $365 \times 24 = 8760$ hours, so on average $1000 \times 8760/100{,}000 = 87.6$ disks fail per year, an annual failure rate of $87.6/1000 = 8.76\%$. Google's 2007 study found actual AFRs for individual drives ranging from 1.7% for first-year drives to over 8.6% for three-year-old drives. Backblaze's lifetime statistics for April 2013 to September 2020 cover 150,757 drives and 175,830,635 drive-days with 7,628 failures, an AFR of 1.58%.

The FAILURES IN TIME (FIT) rate of a device is the number of failures expected in one billion (10^9) device-hours of operation, for example 1000 devices for 1 million hours or 1 million devices for 1000 hours each; so MTBF = 10^9/FIT hours. The automotive safety integrity level (ASIL) defines FIT rates for different classes of components in vehicles.

The design principle is *no single points of failure*: a chain is only as strong as its weakest link. This is the dependability corollary of Amdahl's law: it does not matter how dependable one portion of the system is made, since dependability is limited by the part that is not improved.

38.3 Error detection and parity

Memory systems generate errors (accidentally flipped bits). DRAMs store very little charge per bit; *soft* errors occur occasionally when cells are struck by alpha particles or other environmental upsets, and *hard* errors occur when chips fail permanently. The problem gets worse as memories get denser and larger. Memories are protected against soft errors with error detection and correction codes (EDC/ECC): extra bits are added to each data word to detect and/or correct faults. Each data word value is mapped to a unique CODE WORD, and a fault changes a valid code word to an invalid one, which can be detected.

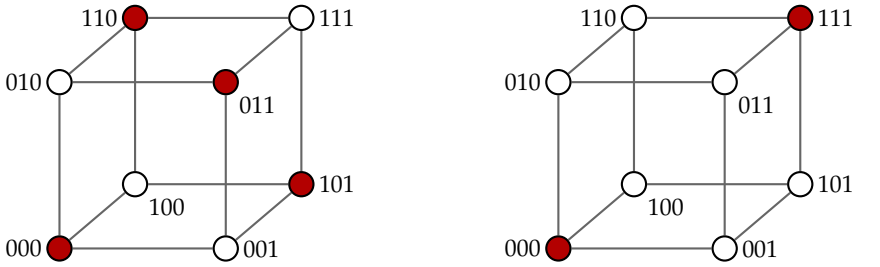
Definition: The HAMMING DISTANCE between two bit strings is the number of positions in which they differ.

For example, 011011 and 001111 have distance 2, and 011011 and 110001 have distance 3. The extra bits can be thought of as creating a code with the data. The code words are a sparse population of the 2^N possible bit patterns ($2^M \ll 2^N$ code words), with an identifiable signature; an error changes a code word into a non-code pattern. DETECTION means the bit pattern fails the code-word check; CORRECTION means mapping it to the nearest valid code word. The eight 3-bit words are the corners of a cube, adjacent corners differing in one bit (Figure 66). If the minimum distance between code words is 2, half the words are valid and no 1-bit error turns one code word into another, so single-bit errors are detected. If it is 3, for example with code words 000 and 111, only a quarter of the words are valid, and every 1-bit error leaves a word that is still nearest the original code word, so single-bit errors can be corrected.

Definition: In PARITY coding, each data value written to memory is tagged with an extra bit $p = b_7 \oplus \cdots \oplus b_0$ that forces the stored word to have even parity. Each word read from memory is

checked by computing the parity c of all its bits, including the parity bit; a nonzero c indicates an error.

The minimum Hamming distance of a parity code is 2. A parity check detects any odd number of bit errors but no even number: two errors on different bits go undetected.



(a) distance 2: detect single-bit errors

(b) distance 3: correct single-bit errors

Figure 66: The 3-bit words as corners of a cube, with valid code words filled. (a) Even parity, minimum distance 2. (b) The code $\{000, 111\}$, minimum distance 3.

Example: The data 0101 0101 has 4 ones, so it is written as 0101 0101 0; the data 0101 0111 has 5 ones, so it is written as 0101 0111 1 to make the parity even. Reading 0101 0101 0 gives 4 ones, even parity, so no error; reading 1101 0101 0 gives 5 ones, odd parity, so an error. An error in the parity bit itself also makes the parity odd and is detected, though the check cannot tell which bit flipped.

38.4 Hamming ECC

Hamming came up with a simple-to-understand mapping that allows error correction at minimum distance 3: SINGLE ERROR CORRECTION, DOUBLE ERROR DETECTION, called HAMMING ECC. (He worked weekends on a relay computer with an unreliable card reader, got frustrated with manual restarting, became interested in error correction, and published "Error Detecting and Correcting Codes" in the Bell System Technical Journal in April 1950.)

Hamming ECC interleaves data and parity bits, placing the parity bits at the positions that are powers of two (1, 10, 100, ... in binary). Parity bit p_1 covers all positions whose least significant bit is 1, p_2 all positions whose next bit is 1, and so on indefinitely. Each parity bit is set to make its group even. For a byte:

Position	1	2	3	4	5	6	7	8	9	10	11	12
Bit	p_1	p_2	d_1	p_4	d_2	d_3	d_4	p_8	d_5	d_6	d_7	d_8
p_1 covers	✓		✓		✓		✓		✓		✓	
p_2 covers		✓	✓			✓	✓			✓	✓	
p_4 covers				✓	✓	✓	✓					✓
p_8 covers								✓	✓	✓	✓	✓

Example: Encode the data byte 10011010. Leaving spaces for the parity bits, the coded word is `__1_001_1010` (positions 1 to 12). Then

- position 1 checks bits $1, 3, 5, 7, 9, 11$: `?_1_001_1010` has data bits $1, 0, 1, 1, 1$ there, four ones, so $p_1 = 0$;
- position 2 checks bits $2, 3, 6, 7, 10, 11$: data bits $1, 0, 1, 0, 1$, three ones, so $p_2 = 1$;
- position 4 checks bits $4, 5, 6, 7, 12$: data bits $0, 0, 1, 0$, one one, so $p_4 = 1$;
- position 8 checks bits $8, 9, 10, 11, 12$: data bits $1, 0, 1, 0$, two ones, so $p_8 = 0$.

The final code word is 011100101010, with data bits 1 001 1010.

Suppose 011100101110 is received. The group of position 1 $(0, 1, 0, 1, 1, 1)$ has even parity, the group of position 2 $(1, 1, 0, 1, 1, 1)$ odd, the group of position 4 $(1, 0, 0, 1, 0)$ even, and the group of position 8 $(0, 1, 1, 1, 0)$ odd. So parity groups 2 and 8 are in error, which implies that bit $8 + 2 = 10$ is in error. Flipping it gives 011100101010, and all four checks now pass.

What if there are more than two bit errors? Double-error correction, triple-error detection (DECTED) codes exist. In network transmissions, disks, and distributed storage the common failure mode is a BURST of bit errors rather than one or two: a contiguous sequence of B bits in which the first, the last, and any number of intermediate bits are in error, caused by impulse noise or by fading in wireless, with a greater effect at higher data rates. These are handled with a cyclic redundancy check (CRC), interleaving, or other more advanced codes.

38.5 RAID

Definition: A REDUNDANT ARRAY OF INEXPENSIVE DISKS (RAID) stores data across multiple disks: files are "striped" across the disks, and redundancy yields high data availability, meaning service is still provided to the user even if some components fail.

Disks will still fail, but their contents can be reconstructed from data stored redundantly in the array. This costs a capacity penalty to store the redundant information and a bandwidth penalty to update it.

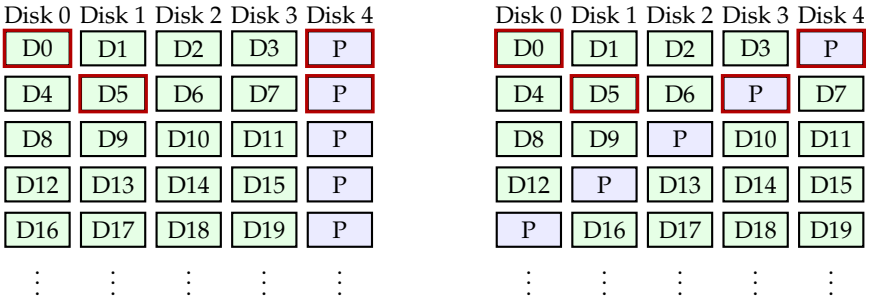
- RAID 1 (disk mirroring or shadowing): each disk is fully duplicated onto its "mirror", forming a recovery group. Very high availability can be achieved; writes are limited by single-disk speed, and reads may be optimized. It is the most expensive solution, with 100% capacity overhead.
- RAID 3 (parity disk): a logical record is striped bit by bit across the data disks, and a parity disk P contains the sum of the other disks in each stripe mod 2 (the parity). If a disk fails, subtracting P from the sum of the other disks recovers the missing information. For example, if a stripe holds $1, 0, 1$ on three data disks, then $P = 0$; if the second disk fails, its bit is recovered

as the parity of the surviving bits 1, 1, and $P = 0$, which is 0.

- RAID 4 (high I/O rate parity): the stripes are made of blocks rather than bits, with all parity blocks on one disk (Figure 67a). A small read such as D_0 or D_5 needs only one disk, and a large write such as the whole stripe D_{12}–D_{15} writes all its blocks and the parity together.

RAID 4 works well for small reads. A small write (to one disk) can either read the other data disks, compute the new sum, and write it to the parity disk, or, since P already holds the old sum, compare the old data with the new data and add the difference to P. Either way every small write also writes the parity disk, so small writes are limited by the parity disk: the writes to D_0 and D_5 both also write the P disk and cannot proceed in parallel.

RAID 5 (high I/O rate interleaved parity) rotates the parity block across the disks from stripe to stripe (Figure 67b), so independent writes are possible: writing D_0 and D_5 uses disks 0 and 4 (data and parity of the first stripe) and disks 1 and 3 (data and parity of the second), four different disks.



(a) RAID 4: one parity disk (b) RAID 5: interleaved parity

Figure 67: Block layouts on five disks. (a) RAID 4 keeps every parity block on disk 4, so small writes to D_0 and D_5 (outlined) both write that disk. (b) RAID 5 rotates the parity, so the same two writes use four different disks.

Disks can be treated like memory, except that we know when a disk has failed: this erasure makes parity an error correcting code, which is why a single parity block suffices to rebuild a failed disk. RAID levels 2 to 5 (and 6 and 10) interleave data and parity.

Source. Dan Garcia and Borivoje Nikolić, *CS 61C: Great Ideas in Computer Architecture (Machine Structures)*, Lecture 38 (Fall 2020).